

O'REILLY®

2nd Edition

Designing Data-Intensive Applications

The Big Ideas Behind Reliable, Scalable,
and Maintainable Systems



**Free
Chapters**

Compliments of



ScyllaDB

Martin Kleppmann
& Chris Riccomini



ScyllaDB

Predictable Low Latencies @ Extreme Throughput

START INSTANTLY

LEARN MORE

Signs you need ScyllaDB for your data-intensive application...

- > Cassandra migraines
- > Scaling impairments
- > Inability to meet SLAs
- > DynamoDB / Bigtable bill shock
- > Painful latency spikes
- > Cloud vendor lock-in



Medium



hulu



STRAVA

Rakuten



SECOND EDITION

Designing Data-Intensive Applications

*The Big Ideas Behind Reliable, Scalable,
and Maintainable Systems*

This excerpt contains Chapters 1, 2, and 9 of *Designing Data-Intensive Applications*. The complete book is available on the O'Reilly Online Learning Platform and through other retailers.

Martin Kleppmann and Chris Riccomini

O'REILLY®

Designing Data-Intensive Applications

by Martin Kleppmann and Chris Riccomini

Copyright © 2026 Martin Kleppmann and Chris Riccomini. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<https://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Aaron Black

Development Editor: Melissa Potter

Production Editor: Katherine Tozer

Copyeditor: Rachel Wheeler

Proofreader: Sharon Wilkey

Indexer: Potomac Indexing, LLC

Cover Designer: Susan Brown

Cover Illustrator: Monica Kamsvaag

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

March 2017: First Edition

February 2026: Second Edition

Revision History for the Second Edition

2026-02-18: First Release

See <https://oreilly.com/catalog/errata.csp?isbn=9781098119065> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Designing Data-Intensive Applications*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O'Reilly and ScyllaDB. See our [statement of editorial independence](#).

978-1-098-11906-5

[LSI]

Table of Contents

1. Trade-Offs in Data Systems Architecture.....	1
Operational Versus Analytical Systems	3
Characterizing Transaction Processing and Analytics	5
Data Warehousing	7
Systems of Record and Derived Data	10
Cloud Versus Self-Hosting	12
Pros and Cons of Cloud Services	13
Cloud Native System Architecture	14
Operations in the Cloud Era	17
Distributed Versus Single-Node Systems	19
Problems with Distributed Systems	20
Microservices and Serverless	21
Cloud Computing Versus Supercomputing	22
Data Systems, Law, and Society	23
Summary	25
2. Defining Nonfunctional Requirements.....	33
Case Study: Social Network Home Timelines	34
Representing Users, Posts, and Follows	34
Materializing and Updating Timelines	35
Describing Performance	37
Latency and Response Time	38
Average, Median, and Percentiles	40
Use of Response Time Metrics	41
Reliability and Fault Tolerance	43
Fault Tolerance	43
Hardware and Software Faults	44

Humans and Reliability	47
Scalability	49
Understanding Load	50
Shared-Memory, Shared-Disk, and Shared-Nothing Architectures	51
Principles for Scalability	52
Maintainability	52
Operability: Making Life Easy for Operations	53
Simplicity: Managing Complexity	54
Evolvability: Making Change Easy	55
Summary	56
9. The Trouble with Distributed Systems.....	65
Faults and Partial Failures	66
Unreliable Networks	67
The Limitations of TCP	68
Network Faults in Practice	70
Fault Detection	71
Timeouts and Unbounded Delays	72
Synchronous Versus Asynchronous Networks	75
Unreliable Clocks	78
Monotonic Versus Time-of-Day Clocks	79
Clock Synchronization and Accuracy	80
Relying on Synchronized Clocks	82
Process Pauses	86
Knowledge, Truth, and Lies	91
The Majority Rules	92
Distributed Locks and Leases	93
Byzantine Faults	97
System Model and Reality	100
Formal Methods and Randomized Testing	104
Summary	108

Trade-Offs in Data Systems Architecture

There are no solutions; there are only trade-offs. [...] But you try to get the best trade-off you can get, and that's all you can hope for.

—Thomas Sowell, interview with Fred Barnes (2005)

Data is central to much application development today. With web and mobile apps, software as a service (SaaS), and cloud services, it has become normal to store data from many different users in a shared server-based data infrastructure. Data from user activity, business transactions, devices, and sensors needs to be stored and made available for analysis. As users interact with an application, they both read the data that is stored and generate more data.

Small amounts of data, which can be stored and processed on a single machine, are often fairly easy to deal with. However, as the data volume or the rate of queries grows, it needs to be distributed across multiple machines, which introduces many challenges. As the needs of the application become more complex, it is no longer sufficient to store everything in one system, and it might be necessary to combine multiple storage or processing systems that provide different capabilities.

We call an application *data-intensive* if data management is one of the primary challenges in developing the application [1]. While in *compute-intensive* systems the challenge is parallelizing a very large computation, in data-intensive applications we usually worry more about things like storing and processing large data volumes, managing changes to data, ensuring consistency in the face of failures and concurrency, and making sure services are highly available.

Such applications are typically built from standard building blocks that provide commonly needed functionality. For example, many applications need to do the following:

- Store data so that they, or another application, can find it again later (*databases*)
- Remember the result of an expensive operation, to speed up reads (*caches*)
- Allow users to search data by keyword or filter it in various ways (*search indexes*)
- Handle events and data changes as soon as they occur (*stream processing*)
- Periodically crunch a large amount of accumulated data (*batch processing*)

In building an application we typically take several software systems or services, such as databases or APIs, and glue them together with application code. If you are doing exactly what the data systems were designed for, this process can be quite easy.

However, as your application becomes more ambitious, challenges arise. There are many database systems with different characteristics, suitable for different purposes—how do you choose which one to use? There are various approaches to caching, several ways of building search indexes, and so on—how do you reason about their trade-offs? You need to figure out which tools and which approaches are the most appropriate for the task at hand, and it can be difficult to combine tools when you need to do something that a single tool cannot do alone.

This book is a guide to help you make decisions about which technologies to use and how to combine them. As you will see, no one approach is fundamentally better than others; everything has pros and cons. With this book, you will learn to ask the right questions to evaluate and compare data systems so that you can figure out which approach will best serve the needs of your particular application.

We will start our journey by looking at some of the ways that data is typically used in organizations today. Many of the ideas here have their origin in *enterprise software* (i.e., the software needs and engineering practices of large organizations, such as big corporations and governments), since historically, only large organizations had the large data volumes that required sophisticated technical solutions. If your data volume is small enough, you can simply keep it in a spreadsheet! However, more recently it has also become common for smaller companies and startups to manage large data volumes and build data-intensive systems.

One of the key challenges with data systems is that different people need to do very different things with data. If you are working at a company, you and your team will have one set of priorities, while another team may have entirely different goals, even though you might be working with the same dataset! Moreover, those goals might not be explicitly articulated, which can lead to misunderstandings and disagreement about the right approach.

To help you understand your choices, this chapter compares several contrasting concepts and explores their trade-offs. We will consider the following topics:

- The difference between operational and analytical systems (“[Operational Versus Analytical Systems](#)” on page 3)
- The pros and cons of cloud services and self-hosted systems (“[Cloud Versus Self-Hosting](#)” on page 12)
- When to move from single-node systems to distributed systems (“[Distributed Versus Single-Node Systems](#)” on page 19)
- Balancing the needs of the business and the rights of the user (“[Data Systems, Law, and Society](#)” on page 23)

This chapter also defines terminology that you will need for the rest of the book.

Terminology: Frontends and Backends

Much of what we will discuss in this book relates to *backend development*. To explain that term: for web applications, the client-side code (which runs in a web browser) is called the *frontend*, and the server-side code that handles user requests is known as the *backend*. Mobile apps are similar to frontends in that they provide user interfaces, which often communicate over the internet with a server-side backend. Frontends sometimes manage data locally on the user’s device [2], but the greatest data infrastructure challenges commonly lie in the backend: a frontend needs to handle only one user’s data, whereas the backend manages data on behalf of *all* the users.

A backend service is often reachable via HTTP (or sometimes WebSocket); it usually consists of application code that reads and writes data in one or more databases and sometimes interfaces with additional data systems, such as caches or message queues (which we might collectively call *data infrastructure*). The application code is often *stateless* (i.e., when it finishes handling one HTTP request, it forgets everything about that request), and any information that needs to persist from one request to another needs to be stored either on the client or in the server-side data infrastructure.

Operational Versus Analytical Systems

If you are working on data systems in an enterprise, you are likely to encounter several different types of people who work with data. The first type are *backend engineers* who build services that handle requests for reading and updating data; these services often serve external users, either directly or indirectly via other services (see “[Microservices and Serverless](#)” on page 21). Sometimes services are for internal use by other parts of the organization.

In addition to the teams managing backend services, two other groups of people typically require access to an organization's data: *business analysts*, who generate reports about the activities of the organization to help management make better decisions (*business intelligence*, or BI), and *data scientists*, who look for novel insights in data or who create user-facing product features that are enabled by data analysis and machine learning (ML)/AI (e.g., "people who bought X also bought Y" recommendations on an ecommerce website, predictive analytics such as risk scoring or spam filtering, and ranking of search results).

Although business analysts and data scientists tend to use different tools and operate in different ways, they have some practices in common. First, both perform *analytics*, which means they look at the data that the users and backend services have generated. Second, they generally do not modify this data (except perhaps for fixing mistakes), although they might create derived datasets in which the original data has been processed in some way.

This has led to a split between two types of systems—a distinction that we will use throughout this book:

- *Operational systems* consist of the backend services and data infrastructure where data is created—for example, by serving external users. Here, the application code both reads and modifies the data in its databases, based on the actions performed by the users.
- *Analytical systems* serve the needs of business analysts and data scientists. They contain a read-only copy of the data from the operational systems, and they are optimized for the types of data processing that are needed for analytics.

As we shall see in the next section, operational and analytical systems are often kept separate, for good reasons. As these systems have matured, two new specialized roles have emerged: data engineers and analytics engineers. *Data engineers* are the people who know how to integrate the operational and analytical systems and who take responsibility for the organization's data infrastructure more widely [3]. *Analytics engineers* model and transform data to make it more useful for the business analysts and data scientists in an organization [4].

Many engineers specialize in either the operational or the analytical side. However, this book covers both operational and analytical data systems, since both play an important role in the lifecycle of data within an organization. We will explore in depth the data infrastructure that is used to deliver services to both internal and external users so that you can work better with your colleagues on the other side of this divide.

Characterizing Transaction Processing and Analytics

In the early days of business data processing, a write to the database typically corresponded to a commercial transaction taking place: making a sale, placing an order with a supplier, paying an employee's salary, etc. As databases expanded into areas that didn't involve money changing hands, the term *transaction* nevertheless stuck, referring to a group of reads and writes that form a logical unit.



Chapter 8 explores in detail what we mean by a transaction. This chapter uses the term loosely to refer to low-latency reads and writes.

Even though databases started being used for many kinds of data—posts on social media, moves in a game, contacts in an address book, and much, much more—the basic access pattern remained similar to processing business transactions. An operational system typically looks up a small number of records by a key (this is called a *point query*). Records are inserted, updated, or deleted based on the user's input. Because these applications are interactive, this access pattern became known as *online transaction processing* (OLTP).

However, databases also started being increasingly used for analytics, which has very different access patterns compared to OLTP. Usually, an analytical query scans over a huge number of records and calculates aggregate statistics (such as count, sum, or average) rather than returning the individual records to the user. For example, a business analyst at a supermarket chain may want to answer analytical queries such as these:

- What was the total revenue of each of our stores in January?
- How many more bananas than usual did we sell during our latest promotion?
- Which brand of baby food is most often purchased together with brand *X* diapers?

The reports that result from these types of queries are important for BI, helping management decide what to do next. To differentiate this pattern of using databases from transaction processing, it has been called *online analytical processing* (OLAP) [5]. The difference between OLTP and analytics is not always clear-cut, but some typical characteristics are listed in [Table 1-1](#).

Table 1-1. Comparing characteristics of operational and analytical systems

Property	Operational systems (OLTP)	Analytical systems (OLAP)
Main read pattern	Point queries (fetch individual records by key)	Aggregate over large number of records
Main write pattern	Create, update, and delete individual records	Bulk import (ETL) or event stream
Human user example	End user of web/mobile application	Internal analyst, for decision support
Machine use example	Checking if an action is authorized	Detecting fraud/abuse patterns
Type of queries	Fixed, predefined by application	Arbitrary, ad-hoc exploration by analysts
Query volume	Lots of small queries	Few queries, each is complex
Data represents	Latest state of data (current point in time)	History of events that happened over time
Dataset size	Gigabytes to terabytes	Terabytes to petabytes



The meaning of *online* in *OLAP* is unclear; it probably indicates that queries are not just for predefined reports, but that analysts use the OLAP system interactively for explorative queries.

With operational systems, users are generally not allowed to construct custom SQL queries and run them on the database, since that would potentially allow them to read or modify data that they do not have permission to access. They might also write queries that are expensive to execute and hence affect the database performance for other users. For these reasons, OLTP systems mostly run fixed sets of queries that are baked into the application code, with one-off custom queries used only occasionally for maintenance or troubleshooting. On the other hand, analytical databases usually give their users the freedom to write arbitrary SQL queries by hand, or to generate queries automatically using a data visualization or dashboard tool such as Tableau, Looker, or Microsoft Power BI.

Another type of system is designed for analytical workloads (queries that aggregate over many records) but embedded into user-facing products. Systems designed for this type of use, known as *product analytics* or *real-time analytics*, include Pinot, Druid, and ClickHouse [6]. Such systems ingest data in real time and are optimized for low-latency query responses. In contrast, traditional OLAP systems typically ingest data in batches and are optimized for high-throughput query processing.

Data Warehousing

At first, the same databases were used for both transaction processing and analytical queries. SQL turned out to be quite flexible in this regard; it works well for both types of queries. In the late 1980s and early 1990s, however, a trend arose for companies to stop using their OLTP systems for analytics purposes and to run the analytics on a separate database system instead. This separate database was called a *data warehouse*.

A large enterprise may have dozens, even hundreds, of OLTP systems: systems powering the customer-facing website, controlling point-of-sale (checkout) systems in physical stores, tracking inventory in warehouses, planning routes for vehicles, managing suppliers, administering employees, and performing many other tasks. Each of these systems is complex and needs a team of people to maintain it, so they end up operating mostly independently from one another.

It is usually undesirable for business analysts and data scientists to directly query these OLTP systems, for several reasons:

- The data of interest may be spread across multiple operational systems, making it difficult to combine those datasets in a single query (a problem known as *data silos*).
- The kinds of schemas and data layouts that are good for OLTP are less well suited for analytics (see Chapter 3).
- Analytical queries can be quite expensive, and running them on an OLTP database would impact the performance for other users.
- The OLTP systems might reside in a separate network that users are not allowed to directly access, for security or compliance reasons.

A *data warehouse*, by contrast, is a separate database that analysts can query to their hearts' content, without affecting OLTP operations [7]. As we shall see in Chapter 4, data warehouses often store data very differently from OLTP databases, to optimize for the types of queries that are common in analytics.

The data warehouse contains a read-only copy of the data from all the various OLTP systems in the company. Data is extracted from OLTP databases (using either a periodic data dump or a continuous stream of updates), transformed into an analysis-friendly schema, cleaned up, and then loaded into the data warehouse. This process of getting data into the data warehouse is known as *extract-transform-load* (ETL) and is illustrated in [Figure 1-1](#). Sometimes the order of the *transform* and *load* steps is swapped (i.e., the transformation is done in the data warehouse, after loading), resulting in *ELT*.

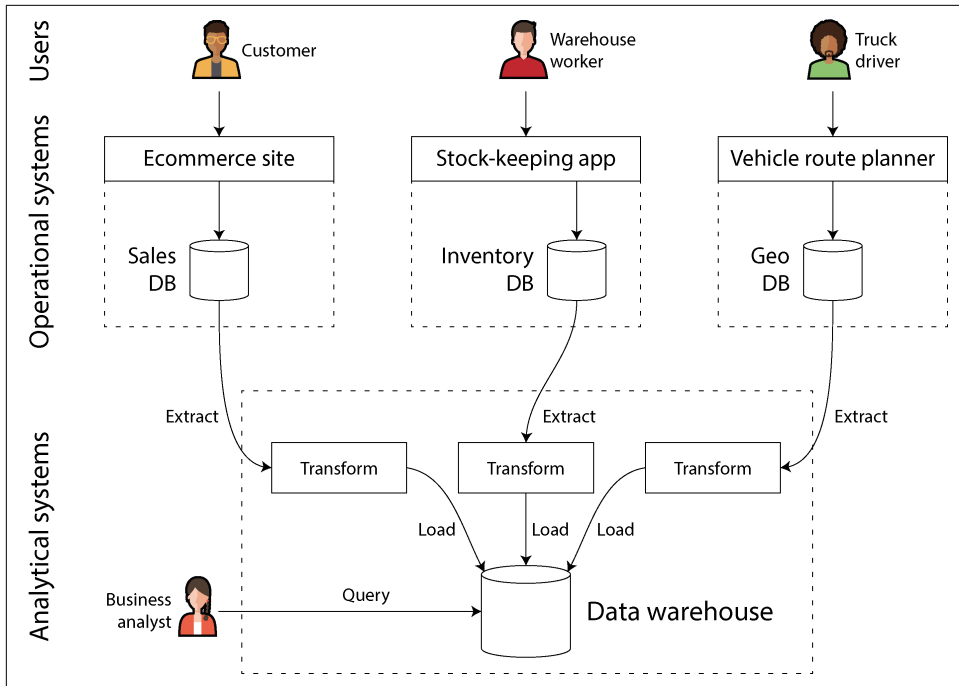


Figure 1-1. A simplified outline of ETL into a data warehouse

In some cases, the data sources of the ETL processes are external SaaS products such as customer relationship management (CRM), email marketing, or credit card processing systems. In those cases, you do not have direct access to the original database, since it is accessible only via the software vendor’s API. Bringing the data from these external systems into your own data warehouse can enable analyses that are not possible via the SaaS API. ETL for SaaS APIs is often implemented by specialist data connector services such as Fivetran, Singer, or Airbyte.

Some database systems offer *hybrid transactional/analytical processing* (HTAP), which aims to enable OLTP and analytics in a single system without requiring ETL from one system into another [8, 9]. However, many HTAP systems internally consist of an OLTP system coupled with a separate analytical system, hidden behind a common interface—so the distinction between the two remains important for understanding how these systems work.

Moreover, even though HTAP exists, it is common to have a separation between transactional and analytical systems because of their different goals and requirements. In particular, it is considered good practice for each operational system to have its own database (see “**Microservices and Serverless**” on page 21), leading to potentially hundreds of separate operational databases; on the other hand, an

enterprise usually has a single data warehouse, so that business analysts can combine data from several operational systems in a single query.

HTAP, therefore, does not replace data warehouses. Rather, it is useful when the same application needs to both perform analytical queries that scan a large number of rows and read and update individual records with low latency. Fraud detection can involve such workloads, for example [10].

The separation between operational and analytical systems is part of a wider trend. As workloads have become more demanding, systems have become more specialized and optimized for particular workloads. General-purpose systems can handle small data volumes comfortably, but the greater the scale, the more specialized systems tend to become [11].

From data warehouse to data lake

A data warehouse often uses a *relational* data model that is queried through SQL (see Chapter 3), perhaps using specialized BI software. This model works well for the types of queries that business analysts need to make, but it is less well suited to the needs of data scientists performing tasks such as these:

- Transforming data into a form that is suitable for training an ML model. This often requires turning the rows and columns of a database table into a vector or matrix of numerical values called *features*. The process of performing this transformation in a way that maximizes the performance of the trained model is called *feature engineering*, and it commonly requires custom code that is difficult to express using SQL.
- Using natural language processing (NLP) techniques on textual data (e.g., reviews of a product) to try to extract structured information from it (e.g., the sentiment of the author, or which topics they mention). Similarly, data scientists might need to extract structured information from photos by using computer vision techniques.

Although there have been efforts to add ML operators to a SQL data model [12] and to build efficient ML systems on top of a relational foundation [13], many data scientists prefer not to work in a relational database such as a data warehouse. Instead, many prefer to use Python data analysis libraries such as Pandas and scikit-learn, statistical analysis languages such as R, and distributed analytics frameworks such as Spark [14]. We discuss these further in Chapter 3.

Consequently, organizations face a need to make data available in a form that is suitable for use by data scientists. The answer is a *data lake*: a centralized data repository that holds a copy of any data that might be useful for analysis, obtained from operational systems via ETL processes. The difference from a data warehouse is that a data lake simply contains files, without imposing any particular file format, data

model, or schema [15]. Files in a data lake might be collections of database records, encoded using a file format such as Avro or Parquet (see Chapter 5), but a data lake can equally well contain text, images, videos, sensor readings, sparse matrices, feature vectors, genome sequences, or any other kind of data [16]. Besides being more flexible, a data lake is also often cheaper than relational data storage, since it can use commoditized file storage such as object stores (see “[Cloud Native System Architecture](#)” on page 14).

ETL processes have been generalized to *data pipelines*, and in some cases the data lake has become an intermediate stop on the path from the operational systems to the data warehouse. The data lake contains data in the “raw” form produced by the operational systems, without the transformation into a relational data warehouse schema. This approach has the advantage that each consumer of the data can transform the raw data into the form that best suits their needs. It’s sometimes called the *sushi principle*: “raw data is better” [17].

Beyond the data lake

As analytics practices have matured, organizations have been increasingly paying attention to the management and operations of analytical systems and data pipelines, as captured, for example, in the DataOps Manifesto [18]. This has been driven partly by issues of governance, privacy, and compliance with regulations such as the General Data Protection Regulation (GDPR) and California Consumer Privacy Act (CCPA), which we discuss in “[Data Systems, Law, and Society](#)” on page 23 and in Chapter 14.

Another important factor is that data for analytics is increasingly made available not only as files and relational tables, but as streams of events (see Chapter 12). With file-based data analysis, you can rerun the analysis periodically (e.g., daily) to respond to changes in the data, but stream processing allows analytical systems to respond to events much faster, on the order of seconds. Depending on the application and its time-sensitivity, a stream processing approach can be valuable, for example, to identify and block potentially fraudulent or abusive activity.

In some cases the outputs of analytical systems are made available to operational systems (a process sometimes known as *reverse ETL* [19]). For example, an ML model that was trained on data in an analytical system may be deployed to production so that it can generate recommendations for end users, such as “people who bought *X* also bought *Y*.” Machine learning models can be deployed to operational systems by using specialized tools such as TFX, Kubeflow, or MLflow.

Systems of Record and Derived Data

Related to the distinction between operational and analytical systems, this book also distinguishes between *systems of record* and *derived data systems*. These terms are useful because they help clarify the flow of data through a system:

Systems of record

A system of record, also known as a *source of truth*, holds the authoritative or *canonical* version of data. When new data comes in—for example, as user input—it is first written here. Each fact is represented exactly once (the representation is typically *normalized*; see Chapter 3). If there is any discrepancy between another system and the system of record, the value in the system of record is (by definition) the correct one.

Derived data systems

Data in a derived system is the result of taking existing data from another system and transforming or processing it in some way. If you lose derived data, you can re-create it from the original source. A classic example is a cache: data can be served from the cache if present, but if the cache doesn't contain what you need, you can fall back to the underlying database. Denormalized values, indexes, materialized views, transformed data representations, and models trained on a dataset also fall into this category.

Technically speaking, derived data is *redundant*, in the sense that it duplicates existing information. However, this data is often essential for getting good performance on read queries. You can derive several datasets from a single source, enabling you to look at the data from different points of view.

Analytical systems are usually derived data systems, because they are consumers of data created elsewhere. Operational services may contain a mixture of systems of record and derived data systems. The systems of record are the primary databases to which data is first written, whereas the derived data systems are the indexes and caches that speed up common read operations, especially for queries that the system of record cannot answer efficiently.

Most databases, storage engines, and query languages are not inherently systems of record or derived systems. A database is just a tool; how you use it is up to you. The distinction between a system of record and a derived data system depends not on the tool, but on the way you use it in your application. By being clear about which data is derived from which other data, you can bring clarity to an otherwise confusing system architecture.

When the data in one system is derived from the data in another, you need a process for updating the derived data when the original in the system of record changes. Unfortunately, many databases are designed based on the assumption that your application will always need to use only that one database, and they do not make it easy to integrate multiple systems in order to propagate such updates. In Chapter 11 we will discuss data pipelines as an approach to *data integration*, which allows us to compose multiple data systems to achieve things that one system alone cannot do.

That brings us to the end of our comparison of analytics and transaction processing. In the next section we will examine another trade-off that you might have already seen debated multiple times.

Cloud Versus Self-Hosting

With anything that an organization needs to do, one of the first questions is whether it should be done in-house or outsourced. That is, should you build or should you buy?

Ultimately, this is a question about business priorities. A common rule of thumb is that things that are a core competency or a competitive advantage of your organization should be done in-house, whereas things that are non-core, routine, or commonplace should be left to a vendor [20]. To give an extreme example, most companies do not fabricate their own CPUs, since it is cheaper to buy them from the semiconductor manufacturers.

With software, two important decisions to be made are who builds the software and who deploys it. The spectrum of possibilities is illustrated in Figure 1-2. At one extreme is bespoke software that you write and run in-house; at the other extreme are widely used cloud services or SaaS products that are implemented and operated by an external vendor and that you access only through a web interface or API.

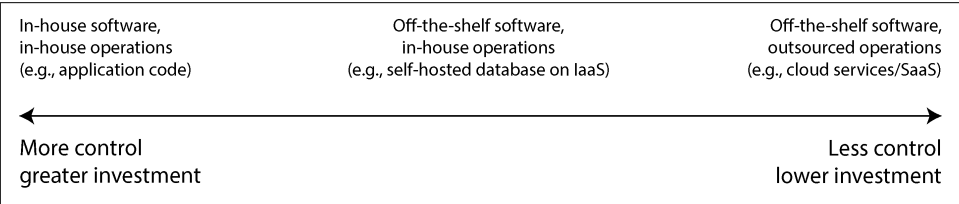


Figure 1-2. The spectrum of decisions on outsourcing software and its operations

The middle ground is off-the-shelf software (open source or commercial) that you *self-host*, or deploy yourself—for example, if you download MySQL and install it on a server you control. This could be on your own hardware (often called *on premises*, even if the server is in a rented datacenter rack and not literally on your own premises), or on a virtual machine (VM) in the cloud (*infrastructure as a service*, or IaaS). There are more points along this spectrum, such as taking open source software and running a modified version of it.

A related question is *how* you deploy services, either in the cloud or on premises—for example, whether you use an orchestration framework such as Kubernetes. However, choice of deployment tooling is beyond the scope for this book, since other factors have a greater influence on the architecture of data systems.

Pros and Cons of Cloud Services

Using a cloud service, rather than running comparable software yourself, essentially outsources the operation of that software to the cloud provider. There are good arguments for and against this approach. Cloud providers claim that using their services saves you time and money and allows you to move faster compared to setting up your own infrastructure.

Whether using a cloud service is actually cheaper and easier than self-hosting depends very much on your skills and the workload on your systems, however. If you already have experience setting up and operating the systems you need, and if your load is quite predictable (i.e., the number of machines you need does not fluctuate wildly), then it's often cheaper to buy your own machines and run the software on them yourself [21, 22].

On the other hand, if you need a system that you don't already know how to deploy and operate, adopting a cloud service is often easier and quicker than learning to manage the system. Hiring and training staff specifically to maintain and operate the system can get very expensive. You still need an operations team when you're using the cloud (see [“Operations in the Cloud Era” on page 17](#)), but outsourcing the basic system administration can free up your team to focus on higher-level concerns.

Outsourcing the operation of a system to a company that specializes in running it can potentially result in better service, since the provider gains operational expertise from providing the service to many customers. On the other hand, if you run the service, you can configure and tune it to perform well on your particular workload. A cloud service would not likely be willing to make such customizations on your behalf.

Cloud services are particularly valuable if the load on your systems varies a lot over time. If you provision your machines to be able to handle peak load, but those computing resources are idle most of the time, the system becomes less cost-effective. In this situation, cloud services have the advantage that they can make it easier to scale your computing resources up or down in response to changes in demand.

For example, analytical systems often have extremely variable load. Running a large analytical query quickly requires a lot of computing resources in parallel, but once the query completes, those resources sit idle until a user makes the next query. Predefined queries (e.g., for daily reports) can be enqueued and scheduled to smooth out the load, but for interactive queries, the faster you want them to complete, the more variable the workload becomes. If your dataset is so large that querying it quickly requires significant computing resources, using the cloud can save money, since you can return unused resources to the provider rather than leaving them idle. For smaller datasets, this difference is less significant.

The biggest downside of a cloud service is that you have no control over it:

- If it is lacking a feature you need, all you can do is politely ask the vendor whether they will add it; you generally cannot implement it yourself.
- If the service goes down, all you can do is to wait for it to recover.
- If you are using the service in a way that triggers a bug or causes performance problems, diagnosing the issue will be difficult. With software that you run yourself, you can get performance metrics and debugging information from the operating system to help you understand its behavior, and you can look at the server logs. With a service hosted by a vendor, you usually do not have access to these internals.
- If the service shuts down or becomes unacceptably expensive, or if the vendor changes their product in a way you don't like, you are at their mercy; continuing to run an old version of the software is usually not an option, so you'll be forced to migrate to an alternative service [23]. This risk is mitigated if alternative services expose a compatible API, but for many cloud services there are no standard APIs, which raises the cost of switching, making vendor lock-in a problem.
- If the cloud provider is in another country and a political conflict arises between that country and your own, you risk being locked out of the service due to imposed sanctions.
- The cloud provider needs to be trusted to keep the data secure, which can complicate the process of complying with privacy and security regulations.

Despite all these risks, it has become more and more popular for organizations to build new applications on top of cloud services, or to adopt a hybrid approach in which cloud services are used for some aspects of a system. However, cloud services will not subsume all in-house data systems. Many older systems predate the cloud, and for any services that have specialist requirements that existing cloud services cannot meet, in-house systems remain necessary. For example, very latency-sensitive applications such as high-frequency trading require full control of the hardware.

Cloud Native System Architecture

Besides having a different economic model (subscribing to a service instead of buying hardware and licensing software to run on it), the rise of the cloud has also had a profound effect on how data systems are implemented on a technical level. The term *cloud native* is used to describe an architecture that is designed to take advantage of cloud services.

In principle, almost any software that you can self-host could also be provided as a cloud service, and indeed such managed services are now available for many popular data systems. However, systems that have been designed from the ground up to be

cloud native have been shown to have several advantages: better performance on the same hardware, faster recovery from failures, being able to quickly scale computing resources to match the load, and supporting larger datasets [24, 25, 26]. [Table 1-2](#) lists some examples of both types of systems.

Table 1-2. Examples of self-hosted and cloud native database systems

Category	Self-hosted systems	Cloud native systems
Operational/OLTP	MySQL, PostgreSQL, MongoDB	AWS Aurora [24], Azure SQL DB Hyperscale [25], Google Cloud Spanner
Analytical/OLAP	Teradata, ClickHouse, Spark	Snowflake [26], Google BigQuery, Azure Synapse Analytics

Layering of cloud services

Many self-hosted data systems have simple system requirements; they run on a conventional operating system such as Linux or Windows, they store their data as files on the filesystem, and they communicate via standard network protocols such as TCP/IP. A few systems depend on special hardware such as GPUs (for ML) or remote direct memory access (RDMA) network interfaces, but on the whole, self-hosted software tends to use generic computing resources: CPUs, RAM, a filesystem, and an IP network.

In a cloud, this type of software can be run in an IaaS environment, using one or more VMs (or *instances*) with a certain allocation of CPUs, memory, disk, and network bandwidth. Compared to physical machines, cloud instances can be provisioned faster and come in a greater variety of sizes, but otherwise they are similar to traditional computers: you can run any software you like on them, but you are responsible for administering it yourself.

In contrast, the key idea of cloud native services is not only to use the computing resources managed by your operating system, but also to build upon lower-level cloud services to create higher-level services. For example:

- Object storage services such as Amazon S3, Azure Blob Storage, and Cloudflare R2 store large files. They provide more limited APIs than a typical filesystem (basic file reads and writes), but they have the advantage that they hide the underlying physical machines; the service automatically distributes the data across many machines so that you don't have to worry about running out of disk space on any one machine. Even if some machines or their disks fail entirely, no data is lost.
- Many other services are, in turn, built upon object storage and other cloud services. For instance, Snowflake is a cloud-based analytical database (data warehouse) that relies on S3 for data storage [26], and some other services, in turn, build upon Snowflake.

As always with abstractions in computing, there is no one right answer to what you should use. As a general rule, higher-level abstractions tend to be more oriented toward particular use cases. If your needs match the situations for which a higher-level system is designed, using the existing higher-level system will probably meet your needs with much less hassle than building it yourself from lower-level systems. On the other hand, if no high-level system meets your needs, building it yourself from lower-level components is the only option.

Separation of storage and compute

In traditional computing, disk storage is regarded as durable (we assume that once something is written to disk, it will not be lost). To tolerate the failure of an individual hard disk, RAID (redundant array of independent disks) is often used to maintain copies of the data on several disks attached to the same machine. RAID can be implemented either in hardware or in software by the operating system, and it is transparent to the applications accessing the filesystem.

In the cloud, compute instances (VMs) may also have local disks attached, but cloud native systems typically treat these disks more like an ephemeral cache and less like long-term storage. This is because the local disk becomes inaccessible if the associated instance fails, or if the instance is replaced with a bigger or a smaller one (on a different physical machine) to adapt to changes in load.

As an alternative to local disks, cloud services also offer virtual disk storage that can be detached from one instance and attached to a different one (e.g., Amazon EBS, Azure managed disks, and persistent disks in Google Cloud). Such a virtual disk is not a physical disk, but rather a cloud service provided by a separate set of machines that emulates the behavior of a disk (a *block device*, where each block is typically 4 KiB in size). This technology makes it possible to run traditional disk-based software in the cloud, but the block device emulation introduces overheads that can be avoided in systems that are designed from the ground up for the cloud [24]. The use of virtual disks also makes the application very sensitive to network glitches, since every I/O operation on the virtual block device is a network call [27].

To address this problem, cloud native services generally avoid using virtual disks and instead build on dedicated storage services that are optimized for particular workloads. Object storage services such as S3 are designed for long-term storage of fairly large files, ranging from hundreds of kilobytes to several gigabytes in size. The individual rows or values stored in a database are typically much smaller than this; cloud databases therefore typically manage smaller values in a separate service and store larger data blocks (containing many individual values) in an object store [25, 28]. We will see ways of doing this in Chapter 4.

In a traditional systems architecture, the same computer is responsible for both storage (disk) and computation (CPU and RAM), but in cloud native systems, these two responsibilities have become somewhat separated, or *disaggregated* [9, 26, 29, 30]: for example, S3 only stores files, and if you want to analyze that data, you will have to run the analysis code somewhere outside of S3. This implies transferring the data over the network, which we will discuss further in “[Distributed Versus Single-Node Systems](#)” on page 19.

Furthermore, cloud native systems are often *multitenant*, which means that rather than having a separate machine for each customer, data and computation from several customers are handled on the same shared hardware by the same service [31]. Multitenancy can enable better hardware utilization, easier scalability, and easier management by the cloud provider, but it also requires careful engineering to ensure that one customer’s activity does not affect the performance or security of the system for other customers [32].

Operations in the Cloud Era

Traditionally, the people managing an organization’s server-side data infrastructure were known as *database administrators* (DBAs) or *system administrators* (sysadmins). More recently, many organizations have tried to integrate the roles of software development and operations into teams with a shared responsibility for both backend services and data infrastructure; the *DevOps* philosophy has guided this trend. *Site reliability engineers* (SREs) are Google’s implementation of this idea [33].

The role of operations is to ensure that services are reliably delivered to users (including configuring infrastructure and deploying applications) and to ensure a stable production environment (including monitoring and diagnosing any problems that may affect reliability). For self-hosted systems, operations traditionally involves a significant amount of work at the level of individual machines, such as capacity planning (e.g., monitoring available disk space and adding more disks before you run out of space), provisioning new machines, moving services from one machine to another, and installing operating system patches.

Many cloud services present an API that hides the individual machines implementing the service. For example, cloud storage replaces fixed-size disks with *metered billing*, where you can store data without planning your capacity needs in advance, and you are then charged based on the space used. Moreover, many cloud services remain highly available, even when individual machines have failed (see “[Reliability and Fault Tolerance](#)” on page 43).

This shift in emphasis from individual machines to services has been accompanied by a change in the role of operations. The high-level goal of providing a reliable service remains the same, but the processes and tools have evolved.

The DevOps/SRE philosophy places greater emphasis on the following:

- Setting up automation, preferring repeatable processes over manual one-off jobs
- Using ephemeral VMs and services rather than long-running servers
- Enabling frequent application updates
- Learning from incidents
- Preserving the organization's knowledge about the system, even as individual people come and go [34]

With the rise of cloud services, a bifurcation of roles has occurred. Operations teams at infrastructure companies specialize in the details of providing a reliable service to a large number of customers, while the customers of the service spend as little time and effort as possible on infrastructure [35].

Customers of cloud services still require operations, but they focus on different aspects, such as choosing the most appropriate service for a given task, integrating services with each other, and migrating from one service to another. Even though metered billing removes the need for capacity planning in the traditional sense, it's still important to know what resources you are using for which purpose so that you don't waste money on cloud resources that are not needed. Capacity planning becomes financial planning, and performance optimization becomes cost optimization [36]. Additionally, cloud services do have resource limits or *quotas* (such as the maximum number of processes you can run concurrently), which you need to know about and plan for before you run into them [37].

Adopting a cloud service can be easier and quicker than provisioning and running your own infrastructure, although you still have to learn how to use the cloud service and perhaps work around its limitations. Integration among services becomes a particular challenge as a growing number of vendors offer an ever broader range of cloud services targeting different use cases [38, 39]. ETL (see “[Data Warehousing](#)” on [page 7](#)) is only part of the story; operational cloud services also need to be integrated with each other. At present, we lack standards to facilitate this sort of integration, so it often involves significant manual effort.

Other operational aspects that cannot fully be outsourced to cloud services include maintaining the security of an application and the libraries it uses, managing the interactions between your own services, monitoring the load on your services, and tracking down the cause of problems such as performance degradations or outages. While the cloud is changing the role of operations, the need for operations is as great as ever.

Distributed Versus Single-Node Systems

A system that involves several machines communicating via a network is called a *distributed system*. Each of the processes participating in a distributed system is called a *node*. You might want to use this type of system for various reasons:

Inherent distribution

If an application involves two or more interacting users, each using their own device, then the system is unavoidably distributed: the communication between the devices will have to occur via a network.

Requests between cloud services

If data is stored in one service but processed in another, that data must be transferred over the network from one service to the other. Cloud native systems and microservices (see “[Microservices and Serverless](#)” on page 21) are therefore distributed.

Fault tolerance/high availability

If your application needs to continue working even if one machine (or several machines, or the network, or an entire datacenter) goes down, you can use multiple machines to give you redundancy. When one fails, another one can take over. See “[Reliability and Fault Tolerance](#)” on page 43 and Chapter 6.

Scalability

If your data volume or computing requirements grow bigger than a single machine can handle, you can potentially spread the load across multiple machines. See “[Scalability](#)” on page 49.

Latency

If you have users around the world, you might want to have servers in various regions worldwide so that each user can be served from a server that is geographically close to them. That avoids the users having to wait for network packets to travel halfway around the world to answer their requests. See “[Describing Performance](#)” on page 37.

Elasticity

If your application is busy at some times and idle at others, a cloud deployment can scale up or down to meet the demand so that you pay only for resources you are actively using. This is more difficult on a single machine, which needs to be provisioned to handle the maximum load, even at times when it is barely used.

Specialized hardware

Different parts of the system can take advantage of different types of hardware to match their workload. For example, an object store may use machines with many disks but few CPUs, whereas a data analysis system may use machines with

lots of CPU and memory but no disks, and a machine learning system may use machines with GPUs (which are much more efficient than CPUs for training deep neural networks and other ML tasks).

Legal compliance

Some countries have data residency laws that require data about people in their jurisdiction to be stored and processed geographically within that country [40]. The scope of these rules varies—for example, in some cases it applies only to medical or financial data, while other cases are broader. A service with users in several such jurisdictions will therefore have to distribute their data across servers in several locations.

Sustainability

If you have flexibility on where and when to run your jobs, you might be able to run them at a time and in a place where plenty of renewable electricity is available and avoid running them when the power grid is under strain. This can reduce your carbon emissions and allow you to take advantage of cheap power when it is available [41, 42].

These reasons apply to both services that you write yourself (application code) and services consisting of off-the-shelf software (such as databases).

Problems with Distributed Systems

Distributed systems also have downsides. Every request and API call that traverses the network needs to deal with the possibility of failure. The network may be interrupted, or the service may be overloaded or crash, and therefore any request may time out without receiving a response. In this case, we don't know whether the service received the request, and simply retrying it might not be safe. We will discuss these problems in detail in Chapter 9.

Although datacenter networks are fast, making a call to another service is still vastly slower than calling a function in the same process [43]. When operating on large volumes of data, rather than transferring the data from storage to a separate machine that processes it, it can be faster to bring the computation to the machine that already has the data [44]. More nodes are not always faster; in some cases, a simple single-threaded program on one computer can perform significantly better than a cluster with over 100 CPU cores [45].

Troubleshooting a distributed system is often difficult—if the system is slow to respond, how do you figure out where the problem lies? Techniques for diagnosing problems in distributed systems are developed under the heading of *observability* [46, 47], which involves collecting data about the execution of a system and allowing that data to be queried in ways that allow both high-level metrics and individual events to be analyzed. *Tracing* tools such as OpenTelemetry, Zipkin, and Jaeger allow you to

track which client called which server for which operation and how long each call took [48].

Databases provide various mechanisms for ensuring data consistency, as we shall see in Chapters 6 and 8. However, when each service has its own database, maintaining consistency of data across those different services becomes the application's problem. Distributed transactions, which we explore in Chapter 8, are a possible technique for ensuring consistency, but they are rarely used in a microservices context because they run counter to the goal of making services independent from each other, and many databases don't support them [49].

For all these reasons, performing a task on a single machine is often much simpler and cheaper than setting up a distributed system [22, 45, 50]. CPUs, memory, and disks have grown larger, faster, and more reliable. When combined with single-node databases such as DuckDB, SQLite, and KùzuDB, many workloads can now run on a single node. We will explore this topic further in Chapter 4.

Microservices and Serverless

The most common way of distributing a system across multiple machines is to divide them into clients and servers and let the clients make requests to the servers. Most commonly, HTTP is used for this communication, as we will discuss in Chapter 5. The same process may be both a server (handling incoming requests) and a client (making outbound requests to other services).

This way of building applications has traditionally been called a *service-oriented architecture* (SOA); more recently, the idea has been refined into a *microservices architecture* [51, 52]. In a microservices architecture, a service has one well-defined purpose (e.g., in the case of S3, this is file storage); each service exposes an API that can be called by clients via the network, and each service has one team that is responsible for its maintenance. A complex application can thus be decomposed into multiple interacting services, each managed by a separate team. Cloud native systems make heavy use of decomposition into services, but on-premises systems can use a service-oriented approach too.

Dividing a complex piece of software into multiple services has several advantages: each service can be updated independently, reducing coordination effort among teams; each service can be assigned the hardware resources it needs; and hiding the implementation details behind an API means the service owners are free to change the implementation without affecting clients. In terms of data storage, it is common for each service to have its own databases and not to share databases between services. Sharing a database would effectively make the entire database structure a part of the service's API, and then that structure would be difficult to change. Shared databases could also cause one service's queries to negatively impact the performance of other services.

On the other hand, having many services can itself breed complexity. Testing a service during development can be complicated, since you also need to run all the other services that it depends on. What's more, each service requires infrastructure for deploying new releases, adjusting the allocated hardware resources to match the load, collecting logs, monitoring service health, and alerting an on-call engineer in the case of a problem. Orchestration frameworks such as Kubernetes have become a popular way of deploying services, since they provide a foundation for this infrastructure.

In addition, microservice APIs can be challenging to evolve. Clients that call an API expect it to have certain fields. Developers might wish to add or remove fields in an API as business needs change, but doing so can cause clients to fail. Worse still, such failures are often not discovered until late in the development cycle, when the updated service API is deployed to a staging or production environment. API description standards such as OpenAPI and gRPC help manage the relationship between client and server APIs; we discuss these further in Chapter 5.

Microservices are primarily a technical solution to a people problem: allowing different teams to make progress independently without having to coordinate with each other. This is valuable in a large company, but in a small company with fewer teams, using microservices is likely to be unnecessary overhead, and implementing the application in the simplest way possible is preferable [51].

Serverless, or *function as a service* (FaaS), is another approach to deploying services, in which the management of the infrastructure is outsourced to a cloud vendor [32]. When using VMs, you have to explicitly choose when to start up or shut down an instance; in contrast, with the serverless model, the cloud provider automatically allocates and frees hardware resources as needed, based on the incoming requests to your service [53]. Just as cloud storage replaced capacity planning (deciding in advance how many disks to buy) with a metered billing model, the serverless approach is bringing metered billing to code execution: you pay only for the time that your application code is running rather than having to provision resources in advance.

To offer such benefits, many serverless infrastructure providers impose a time limit on function execution and limit runtime environments, and services might suffer from slow start times when a function is first invoked. The term “serverless” can also be misleading; each serverless function execution still runs on a server, but subsequent executions might run on a different one. What's more, infrastructure services such as BigQuery and various Kafka offerings have adopted “serverless” terminology to signal that their services autoscale and that they bill by usage rather than machine instances.

Cloud Computing Versus Supercomputing

Cloud computing is not the only way of building large-scale computing systems; an alternative is *high-performance computing* (HPC), also known as *supercomputing*.

Although there are overlaps, HPC often has different priorities and uses different techniques than cloud computing and enterprise datacenter systems. Here are some of the main differences:

- Supercomputers are typically used for computationally intensive scientific computing tasks, such as weather forecasting, climate modeling, molecular dynamics (simulating the movement of atoms and molecules), complex optimization problems, and solving partial differential equations. On the other hand, cloud computing tends to be used for online services, business data systems, and similar systems that need to serve user requests with high availability.
- A supercomputer typically runs large batch jobs that checkpoint the state of their computation to disk from time to time. If a node fails, a common solution is to simply stop the entire cluster workload, repair the faulty node, and then restart the computation from the last checkpoint [54, 55]. With cloud services, stopping the entire cluster is usually not desirable, since the services need to continually serve users with minimal interruptions.
- Supercomputer nodes typically communicate through shared memory and RDMA, which support high bandwidth and low latency but assume a high level of trust among the users of the system [56]. In cloud computing, the network and the machines are often shared by mutually untrusting organizations, requiring stronger security mechanisms such as resource isolation (e.g., virtual machines), encryption, and authentication.
- Cloud datacenter networks are often based on IP and Ethernet, arranged in Clos topologies to provide high bisection bandwidth—a commonly used measure of a network’s overall performance [54, 57]. Supercomputers often use specialized network topologies, such as multidimensional meshes and toruses [58], which yield better performance for HPC workloads with known communication patterns.
- Cloud computing allows nodes to be distributed across multiple geographic regions, whereas supercomputers generally assume that all their nodes are close together.

Large-scale analytical systems sometimes share some characteristics with supercomputing, which is why it can be worth knowing about these techniques if you are working in this area. However, this book is mostly concerned with services that need to be continually available, as discussed in [“Reliability and Fault Tolerance” on page 43](#).

Data Systems, Law, and Society

As you’ve seen in this chapter, the architecture of data systems is influenced not only by technical goals and requirements, but also by the human needs of the organiza-

tions that they support. Increasingly, data systems engineers are realizing that serving the needs of their own business is not enough; we also have a responsibility toward society at large.

One particular concern is systems that store data about people and their behavior. Since 2018, the GDPR has given residents of many European countries greater control and legal rights over their personal data, and similar privacy regulations have been adopted in various other countries and states around the world (including, for example, the CCPA). Regulations around AI, such as the EU AI Act, place further restrictions on how personal data can be used.

Moreover, even in areas that are not directly subject to regulation, there is increasing recognition of the effects that computer systems have on people and society. Social media has changed how individuals consume news, which influences their political opinions and hence may affect the outcome of elections. Automated systems increasingly make decisions that have profound consequences for individuals, such as who should be given a loan or insurance coverage, who should be invited to a job interview, or who should be suspected of a crime [59].

Everyone who works on such systems shares a responsibility for considering the ethical impact of their decisions and ensuring that they comply with relevant laws. Not everyone needs to become an expert in law and ethics, but a basic awareness of legal and ethical principles is just as important as, say, some foundational knowledge in distributed systems.

Legal considerations are influencing the very foundations of data system design [60]. For example, the GDPR grants individuals the right to have their data erased on request (sometimes known as the *right to be forgotten*). However, as we shall see in this book, many data systems rely on immutable constructs such as append-only logs as part of their design. How can we ensure deletion of some data in the middle of a file that is supposed to be immutable? How do we handle deletion of data that has been incorporated into derived datasets (see “[Systems of Record and Derived Data](#)” on page 10), such as training data for ML models? Answering these questions creates new engineering challenges.

At present, we don’t have clear guidelines on which particular technologies or system architectures should be considered GDPR compliant. The regulation deliberately does not mandate particular technologies, because these may quickly change as technology progresses. Instead, the legal texts set out high-level principles that are subject to interpretation. Therefore, how to comply with privacy regulations has no simple answer, but we will look at some of the technologies through this lens.

In general, we store data because we think that its value is greater than the costs of storing it. However, it is worth remembering that the costs of storage extend beyond the bill you pay for S3 or another service. The cost-benefit calculation should also

take into account the risks of liability and reputational damage if the data were to be leaked or compromised by adversaries, and the risk of legal costs and fines if the storage and processing of the data is found not to be compliant with the law [50].

Governments or police forces might also compel companies to hand over data. When data could reveal criminalized behaviors (e.g., homosexuality in several Middle Eastern and African countries, or seeking an abortion in several states in the United States), storing that data creates real safety risks for users. Travel to an abortion clinic, for example, could easily be revealed by location data, or perhaps even by a log of the user's IP addresses over time (which indicate approximate location).

Once all the risks are taken into account, it might be reasonable to decide that some data is simply not worth storing, and that it should therefore be deleted. This principle of *data minimization* (sometimes known by the German term *Datensparsamkeit*) runs counter to the “big data” philosophy of storing lots of data speculatively in case it turns out to be useful in the future [61]. But data minimization fits with the GDPR, which mandates that personal data may be collected only for a specified, explicit purpose; cannot later be used for any other purpose; and must not be kept for longer than necessary for the purposes for which it was collected [62].

Businesses have also taken notice of privacy and safety concerns. Credit card companies require payment processing businesses to adhere to strict Payment Card Industry (PCI) standards. Processors undergo frequent evaluations from independent auditors to verify continued compliance. Software vendors have also seen increased scrutiny. Many buyers now require their vendors to comply with Service Organization Control (SOC) Type 2 standards. As with PCI compliance, vendors undergo third-party audits to verify adherence.

Generally, it is important to balance the needs of your business against the needs of the people whose data you are collecting and processing. There is much more to this topic; in Chapter 14 we will go deeper into ethics and legal compliance, including the problems of bias and discrimination.

Summary

The theme of this chapter has been to understand trade-offs—that is, to recognize that many questions do not have one right answer, but several possibilities that each have pros and cons. We explored some of the most important choices that affect the architecture of data systems, and we introduced terminology that will be used throughout the rest of this book.

We started by making a distinction between operational (transaction processing, OLTP) and analytical (OLAP) systems and exploring how they differ, not only in managing different types of data with different access patterns, but also in serving different audiences. Along the way, we encountered the concepts of a data warehouse

and data lake, which receive data feeds from operational systems via ETL. In Chapter 4 we will see that operational and analytical systems often use very different internal data layouts because of the different types of queries they need to serve.

We then compared cloud services, a comparatively recent development, to the traditional paradigm of self-hosted software that has previously dominated data systems architecture. Which of these approaches is more cost-effective depends a lot on your particular situation, but it's undeniable that cloud native approaches are bringing big changes to the way data systems are architected—for example, in the way they separate storage and compute.

Cloud systems are intrinsically distributed, and we briefly examined some of the trade-offs of distributed systems compared to using a single machine. In some situations you can't avoid going distributed, but it's advisable not to rush into making a system distributed if it's possible to keep it on a single machine. In Chapter 9 we will cover the challenges with distributed systems in more detail.

Finally, we saw that a data system's architecture is determined not only by the needs of the business deploying the system, but also by privacy regulations that protect the rights of the people whose data is being processed—an aspect that many engineers are prone to ignoring. How we translate legal requirements into technical implementations has not yet been formalized, but it's important to keep this question in mind as we move through the rest of this book.

References

- [1] Richard T. Kouzes, Gordon A. Anderson, Stephen T. Elbert, Ian Gorton, and Deborah K. Gracio. “The Changing Paradigm of Data-Intensive Computing.” *IEEE Computer*, volume 42, issue 1, pages 26–34, January 2009. [doi:10.1109/MC.2009.26](https://doi.org/10.1109/MC.2009.26)
- [2] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. “Local-First Software: You Own Your Data, in Spite of the Cloud.” At *2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* (Onward!), October 2019. [doi:10.1145/3359591.3359737](https://doi.org/10.1145/3359591.3359737)
- [3] Joe Reis and Matt Housley. *Fundamentals of Data Engineering*. O'Reilly Media, 2022. ISBN: 9781098108304
- [4] Rui Pedro Machado and Helder Russa. *Analytics Engineering with SQL and dbt*. O'Reilly Media, 2023. ISBN: 9781098142384
- [5] Edgar F. Codd, S. B. Codd, and C. T. Salley. “Providing OLAP to User-Analysts: An IT Mandate.” E. F. Codd Associates, 1993. Archived at perma.cc/RKX8-2GEE

- [6] Chinmay Soman and Neha Pawar. “Comparing Three Real-Time OLAP Databases: Apache Pinot, Apache Druid, and ClickHouse.” *startree.ai*, April 2023. Archived at perma.cc/8BZP-VWPA
- [7] Surajit Chaudhuri and Umeshwar Dayal. “An Overview of Data Warehousing and OLAP Technology.” *ACM SIGMOD Record*, volume 26, issue 1, pages 65–74, March 1997. [doi:10.1145/248603.248616](https://doi.org/10.1145/248603.248616)
- [8] Fatma Özcan, Yuanyuan Tian, and Pinar Tözün. “Hybrid Transactional/Analytical Processing: A Survey.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. [doi:10.1145/3035918.3054784](https://doi.org/10.1145/3035918.3054784)
- [9] Adam Prout, Szu-Po Wang, Joseph Victor, Zhou Sun, Yongzhu Li, Jack Chen, Evan Bergeron, Eric Hanson, Robert Walzer, Rodrigo Gomes, and Nikita Shamgunov. “Cloud-Native Transactions and Analytics in SingleStore.” At *International Conference on Management of Data (SIGMOD)*, June 2022. [doi:10.1145/3514221.3526055](https://doi.org/10.1145/3514221.3526055)
- [10] Chao Zhang, Guoliang Li, Jintao Zhang, Xinning Zhang, and Jianhua Feng. “HTAP Databases: A Survey.” *IEEE Transactions on Knowledge and Data Engineering*, volume 36, issue 11, pages 6410–6429, April 2024. [doi:10.1109/TKDE.2024.3389693](https://doi.org/10.1109/TKDE.2024.3389693)
- [11] Michael Stonebraker and Uğur Çetintemel. “One Size Fits All: An Idea Whose Time Has Come and Gone.” At *21st International Conference on Data Engineering (ICDE)*, April 2005. [doi:10.1109/ICDE.2005.1](https://doi.org/10.1109/ICDE.2005.1)
- [12] Jeffrey Cohen, Brian Dolan, Mark Dunlap, Joseph M. Hellerstein, and Caleb Welton. “MAD Skills: New Analysis Practices for Big Data.” *Proceedings of the VLDB Endowment*, volume 2, issue 2, pages 1481–1492, August 2009. [doi:10.14778/1687553.1687576](https://doi.org/10.14778/1687553.1687576)
- [13] Dan Olteanu. “The Relational Data Borg Is Learning.” *Proceedings of the VLDB Endowment*, volume 13, issue 12, pages 3502–3515, August 2020. [doi:10.14778/3415478.3415572](https://doi.org/10.14778/3415478.3415572)
- [14] Matt Bornstein, Martin Casado, and Jennifer Li. “Emerging Architectures for Modern Data Infrastructure: 2020.” *future.a16z.com*, October 2020. Archived at perma.cc/LF8W-KDCC
- [15] Rihan Hai, Christos Koutras, Christoph Quix, and Matthias Jarke. “Data Lakes: A Survey of Functions and Systems.” *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, volume 35, issue 12, pages 12571–12590, December 2023. [doi:10.1109/TKDE.2023.3270101](https://doi.org/10.1109/TKDE.2023.3270101)
- [16] Martin Fowler. “Data Lake.” *martinfowler.com*, February 2015. Archived at perma.cc/4WKN-CZUK
- [17] Bobby Johnson and Joseph Adler. “The Sushi Principle: Raw Data Is Better.” At *Strata+Hadoop World*, February 2015.

- [18] DataKitchen, Inc. “The DataOps Manifesto.” *dataopsmanifesto.org*, 2017. Archived at perma.cc/3F5N-FUQ4
- [19] Tejas Manohar. “What Is Reverse ETL: A Definition & Why It’s Taking Off” *hightouch.io*, November 2021. Archived at perma.cc/A7TN-GLYJ
- [20] Camille Fournier. “Why Is It So Hard to Decide to Buy?” *skamille.medium.com*, July 2021. Archived at perma.cc/6VSG-HQ5X
- [21] David Heinemeier Hansson. “Why We’re Leaving the Cloud.” *world.hey.com*, October 2022. Archived at perma.cc/82E6-UJ65
- [22] Nima Badizadegan. “Use One Big Server.” *specbranch.com*, August 2022. Archived at perma.cc/M8NB-95UK
- [23] Steve Yegge. “Dear Google Cloud: Your Deprecation Policy Is Killing You.” *steve-yegge.medium.com*, August 2020. Archived at perma.cc/KQP9-SPGU
- [24] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmadesam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. “Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases.” At *ACM International Conference on Management of Data (SIGMOD)*, May 2017. [doi:10.1145/3035918.3056101](https://doi.org/10.1145/3035918.3056101)
- [25] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravela, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. “Soc-rates: The New SQL Server in the Cloud.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2019. [doi:10.1145/3299869.3314047](https://doi.org/10.1145/3299869.3314047)
- [26] Midhul Vuppalapati, Justin Miron, Rachit Agarwal, Dan Truong, Ashish Motivala, and Thierry Cruanes. “Building an Elastic Query Engine on Disaggregated Storage.” At *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February 2020.
- [27] Nick Van Wiggeren. “The Real Failure Rate of EBS.” *planetscale.com*, March 2025. Archived at perma.cc/43CR-SAH5
- [28] Colin Breck. “Predicting the Future of Distributed Systems.” *blog.colinbreck.com*, August 2024. Archived at perma.cc/K5FC-4XX2
- [29] Gwen Shapira. “Compute-Storage Separation Explained.” *thenile.dev*, January 2023. Archived at perma.cc/QCV3-XJNZ
- [30] Ravi Murthy and Gurmeet Goindi. “AlloyDB for PostgreSQL Under the Hood: Intelligent, Database-Aware Storage.” *cloud.google.com*, May 2022. Archived at archive.org

- [31] Jack Vanlightly. “The Architecture of Serverless Data Systems.” *jack-vanlightly.com*, November 2023. Archived at perma.cc/UDV4-TNJ5
- [32] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, Joseph E. Gonzalez, Raluca Ada Popa, Ion Stoica, and David A. Patterson. “Cloud Programming Simplified: A Berkeley View on Serverless Computing.” *arXiv:1902.03383*, February 2019.
- [33] Betsy Beyer, Jennifer Petoff, Chris Jones, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. ISBN: 9781491929124
- [34] Thomas Limoncelli. “The Time I Stole \$10,000 from Bell Labs.” *ACM Queue*, volume 18, issue 5, November 2020. [doi:10.1145/3434571.3434773](https://doi.org/10.1145/3434571.3434773)
- [35] Charity Majors. “The Future of Ops Jobs.” *acloudguru.com*, August 2020. Archived at perma.cc/GRU2-CZG3
- [36] Boris Cherkasky. “(Over)Pay as You Go for Your Datastore.” *medium.com*, September 2021. Archived at perma.cc/Q8TV-2AM2
- [37] Shlomi Kushchi. “Serverless Doesn’t Mean DevOpsLess or NoOps.” *thenewstack.io*, February 2023. Archived at perma.cc/3NJR-AYYU
- [38] Erik Bernhardsson. “Storm in the Stratosphere: How the Cloud Will Be Reshuffled.” *erikbern.com*, November 2021. Archived at perma.cc/SYB2-99P3
- [39] Benn Stancil. “The Data OS.” *benn.substack.com*, September 2021. Archived at perma.cc/WQ43-FHS6
- [40] Maria Korolov. “Data Residency Laws Pushing Companies Toward Residency as a Service.” *csoonline.com*, January 2022. Archived at perma.cc/CHE4-XZZ2
- [41] Severin Borenstein. “Can Data Centers Flex Their Power Demand?” *energy-at-haas.wordpress.com*, April 2025. Archived at perma.cc/MUD3-A6FF
- [42] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Aditya Sundarrajan, Kiwan Maeng, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. “Carbon Dependencies in Datacenter Design and Management.” *ACM SIGENERGY Energy Informatics Review*, volume 3, issue 3, pages 21–26, October 2023. [doi:10.1145/3630614.3630619](https://doi.org/10.1145/3630614.3630619)
- [43] Kousik Nath. “These Are the Numbers Every Computer Engineer Should Know.” *freecodecamp.org*, September 2019. Archived at perma.cc/RW73-36RL
- [44] Joseph M. Hellerstein, Jose Faleiro, Joseph E. Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. “Serverless Computing: One Step Forward, Two Steps Back.” *arXiv:1812.03651*, December 2018.

- [45] Frank McSherry, Michael Isard, and Derek G. Murray. “Scalability! But at What COST?” At *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [46] Cindy Sridharan. *Distributed Systems Observability: A Guide to Building Robust Systems*. Report, O’Reilly Media, 2018. Archived at perma.cc/M6JL-XKCM
- [47] Charity Majors. “Observability—A 3-Year Retrospective.” *thenewstack.io*, August 2019. Archived at perma.cc/CG62-TJWL
- [48] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure.” Google Technical Report dapper-2010-1, April 2010. Archived at perma.cc/K7KU-2TMH
- [49] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. “Data Management in Microservices: State of the Practice, Challenges, and Research Directions.” *Proceedings of the VLDB Endowment*, volume 14, issue 13, pages 3348–3361, September 2021. [doi:10.14778/3484224.3484232](https://doi.org/10.14778/3484224.3484232)
- [50] Jordan Tigani. “Big Data Is Dead.” *motherduck.com*, February 2023. Archived at perma.cc/HT4Q-K77U
- [51] Sam Newman. *Building Microservices*, 2nd edition. O’Reilly Media, 2021. ISBN: 9781492034025
- [52] Chris Richardson. “Microservices: Decomposing Applications for Deployability and Scalability.” *infoq.com*, May 2014. Archived at perma.cc/CKN4-YEQ2
- [53] Mohammad Shahrads, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. “Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider.” At *USENIX Annual Technical Conference (ATC)*, July 2020.
- [54] Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, 3rd edition. Springer Nature, 2019. ISBN: 9783031017612
- [55] David Fiala, Frank Mueller, Christian Engelmann, Rolf Riesen, Kurt Ferreira, and Ron Brightwell. “Detection and Correction of Silent Data Corruption for Large-Scale High-Performance Computing.” At *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, November 2012. [doi:10.1109/SC.2012.49](https://doi.org/10.1109/SC.2012.49)
- [56] Anna Kornfeld Simpson, Adriana Szekeres, Jacob Nelson, and Irene Zhang. “Securing RDMA for High-Performance Datacenter Storage Systems.” At *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, July 2020.

- [57] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. “Jupiter Rising: A Decade of Clos Topologies and Centralized Control in Google’s Datacenter Network.” At *Annual Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*, August 2015. doi:10.1145/2785956.2787508
- [58] Glenn K. Lockwood. “Hadoop’s Uncomfortable Fit in HPC.” *glennklockwood.blogspot.co.uk*, May 2014. Archived at perma.cc/S8XX-Y67B
- [59] Cathy O’Neil. *Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy*. Crown Publishing, 2016. ISBN: 9780553418811
- [60] Supreeth Shastri, Vinay Banakar, Melissa Wasserman, Arun Kumar, and Vijay Chidambaram. “Understanding and Benchmarking the Impact of GDPR on Database Systems.” *Proceedings of the VLDB Endowment*, volume 13, issue 7, pages 1064–1077, March 2020. doi:10.14778/3384345.3384354
- [61] Martin Fowler. “Datensparsamkeit.” *martinfowler.com*, December 2013. Archived at perma.cc/R9QX-CME6
- [62] “Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 (General Data Protection Regulation).” *Official Journal of the European Union* L 119/1, May 2016.

Defining Nonfunctional Requirements

The Internet was done so well that most people think of it as a natural resource like the Pacific Ocean, rather than something that was man-made. When was the last time a technology with a scale like that was so error-free?

—Alan Kay, in interview with Dr. Dobbs Journal (2012)

If you are building an application, you will be driven by a list of requirements. At the top of your list is most likely the functionality that the application must offer: what screens and what buttons you need, and what each operation is supposed to do in order to fulfill the purpose of your software. These are your *functional requirements*.

In addition, you probably have *nonfunctional requirements*: for example, the app should be fast, reliable, secure, legally compliant, and easy to maintain. These requirements might not be explicitly written down, because they may seem somewhat obvious, but they are just as important as the app's functionality; an app that is unbearably slow or unreliable might as well not exist.

Many nonfunctional requirements, such as security, fall outside the scope of this book. But we will consider a few, and this chapter will help you articulate them for your own systems. In particular, we will look at the following:

- Defining and measuring the *performance* of a system
- What it means for a service to be *reliable*—namely, continuing to work correctly, even when things go wrong
- Allowing a system to be *scalable* by having efficient ways of adding computing capacity as the load on the system grows
- Making it easier to maintain a system in the long term

The terminology introduced in this chapter will also be useful in the following chapters, when we go into the details of how data-intensive systems are implemented. However, abstract definitions can be quite dry; to make the ideas more concrete, we will start this chapter with a case study of a social networking service, which will provide practical examples of performance and scalability.

Case Study: Social Network Home Timelines

Imagine we have been given the task of implementing a social network in the style of X (formerly Twitter), where users can post messages and follow other users. This will be a huge simplification of how such a service actually works [1, 2, 3], but it will help illustrate some of the issues that arise in large-scale systems.

Let's assume that users make a total of 500 million posts per day, or 5,800 posts per second on average. Occasionally, the rate can spike to as high as 150,000 posts per second [4]. Let's also assume that the average user follows 200 people and has 200 followers (although there is a very wide range: most people have only a handful of followers, and a few celebrities, such as Barack Obama, have over 100 million followers).

Representing Users, Posts, and Follows

We keep all the data in a relational database, as shown in [Figure 2-1](#). We have one table for users, one table for posts, and one table for follow relationships.

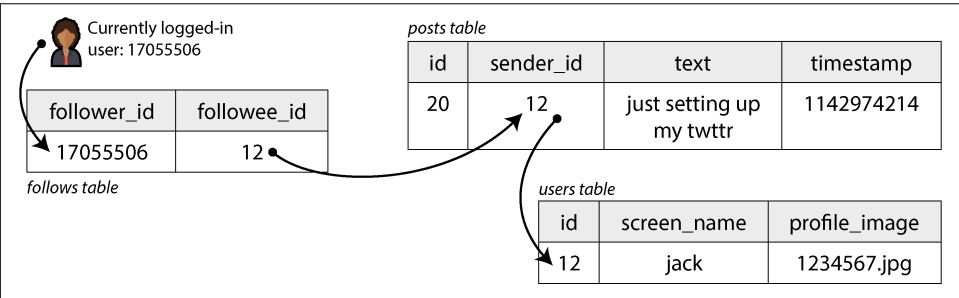


Figure 2-1. A simple relational schema for a social network in which users can follow one another

Let's say the main read operation that our social network must support is the *home timeline*, which displays recent posts by people the user is following (for simplicity we will ignore ads, suggested posts from people they are not following, and other extensions). We could write the following SQL query to get the home timeline for a particular user:

```

SELECT posts.*, users.* FROM posts
JOIN follows ON posts.sender_id = follows.followee_id
JOIN users   ON posts.sender_id = users.id
WHERE follows.follower_id = current_user
ORDER BY posts.timestamp DESC
LIMIT 1000

```

To execute this query, the database will use the `follows` table to find everybody who `current_user` is following, look up recent posts by those users, and sort them by timestamp to get the most recent 1,000 posts by any of the followed users.

Posts are supposed to be timely, so let's assume that after somebody makes a post, we want their followers to be able to see it within five seconds. One approach is for the user's client to repeat the preceding query every five seconds while the user is online (this is known as *polling*). If we assume that 10 million users are online and logged in at the same time, that would mean running the query 2 million times per second. Even if we were to poll less frequently, this is a lot.

This query is also quite expensive: if a user is following 200 people, the query needs to fetch a list of recent posts by each of those 200 people and merge those lists. Two million timeline queries per second times 200 followed accounts makes 400 million lookups per second—a huge number. And that's the average case. Some users follow tens of thousands of accounts; for them, this query is very expensive to execute and difficult to make fast.

Materializing and Updating Timelines

How can we do better? First, instead of polling, it would be better if the server actively pushed new posts to any followers who are currently online. Second, we should precompute the results of the query so that a user's request for their home timeline can be served from a cache.

Imagine that for each user, we store a data structure containing their home timeline (i.e., the recent posts by people they are following). Every time a user makes a post, we look up all their followers and insert that post into the home timeline of each follower—like delivering a message to a mailbox. Now when a user logs in, we can simply give them this precomputed home timeline. Moreover, to receive a notification about any new posts on their timeline, the user's client simply needs to subscribe to the stream of posts being added to their home timeline.

The downside of this approach is that we now need to do more work every time a user makes a post, because the home timelines are derived data that needs to be updated. The process is illustrated in [Figure 2-2](#). When one initial request results in several downstream requests being carried out, we use the term *fan-out* to describe the factor by which the number of requests increases.

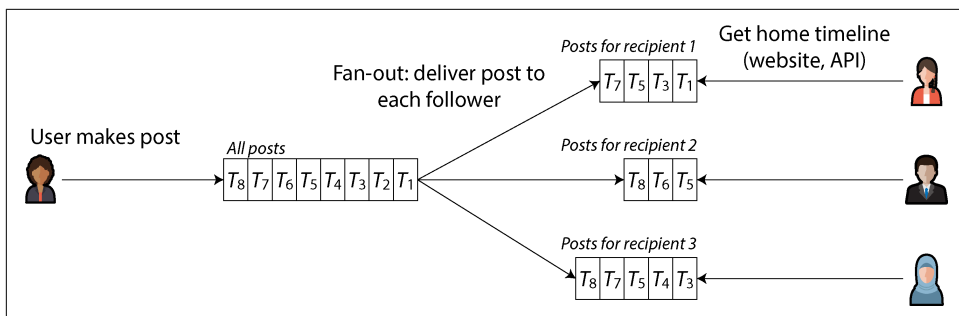


Figure 2-2. Fan-out: delivering new posts to every follower of the user who made the post

At a rate of 5,800 posts per second, if the average post reaches 200 followers (i.e., a fan-out factor of 200), we will need to do just over 1 million home timeline writes per second. This is a lot, but it's still a significant saving compared to the 400 million per-sender post lookups per second that we would otherwise have to do.

If the rate of posts spikes because of a special event, we don't have to do the timeline deliveries immediately—we can enqueue them and accept that it will temporarily take a bit longer for posts to show up in followers' timelines. Even during such load spikes, timelines remain fast to load, since we simply serve them from a cache.

This process of precomputing and updating the results of a query is called *materialization*, and the timeline cache is an example of a *materialized view* (a concept we will discuss further in later chapters). The materialized view speeds up reads, but in return we have to do more work on writes. The cost of writes for most users is modest, but a social network also has to consider some extreme cases:

- If a user is following a very large number of accounts, and those accounts post a lot, that user will have a high rate of writes to their materialized timeline. However, that user is not likely reading all the posts in their timeline, so it's OK to simply drop some of their timeline writes and show the user only a sample of the posts from the accounts they're following [5].
- When a celebrity account with a very large number of followers makes a post, we have to do a lot of work to insert that post into the home timelines of each of their millions of followers. In this case, dropping some of those writes is not OK. One way of solving this problem is to handle celebrity posts separately from everyone else's posts: we can save ourselves the effort of adding celebrity posts to millions of timelines by storing them separately and merging them with the materialized timeline when it is read. Despite such optimizations, handling celebrities on a social network can require a lot of infrastructure [6].

Describing Performance

Most discussions of software performance consider two main types of metric:

Response time

The elapsed time from the moment when a user makes a request until they receive the requested answer. The unit of measurement is seconds (or milliseconds, or microseconds).

Throughput

The number of requests per second, or the data volume per second, that the system is processing. For a given allocation of hardware resources, there is a *maximum throughput* that can be handled. The unit of measurement is “some-things per second.”

In the social network case study, “posts per second” and “timeline writes per second” are throughput metrics, whereas “time it takes to load the home timeline” and “time until a post is delivered to followers” are response time metrics.

Throughput and response time are often related. An example of such a relationship for an online service is sketched in [Figure 2-3](#). The service has a low response time when request throughput is low, but response time increases as load increases. This is because of *queueing*: when a request arrives on a highly loaded system, the CPU is likely already in the process of handling an earlier request, and therefore the incoming request needs to wait until the earlier request has been completed. As throughput approaches the maximum that the hardware can handle, queueing delays increase sharply.

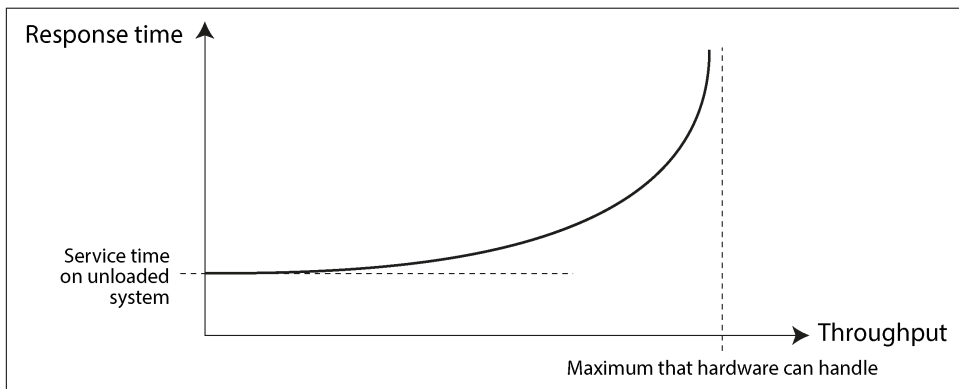


Figure 2-3. As the throughput of a service approaches its capacity, the response time increases dramatically because of queueing.

When an Overloaded System Won't Recover

If a system is close to overload, with throughput pushed close to the limit, it can sometimes enter a vicious cycle where it becomes less efficient and hence even more overloaded. For example, if a long queue of requests is waiting to be handled, response times may increase so much that clients time out and resend their requests. This causes the rate of requests to increase even further, making the problem worse—a *retry storm*. Even when the load is reduced again, such a system may remain in an overloaded state until it is rebooted or otherwise reset. This phenomenon is called a *metastable failure*, and it can cause serious outages in production systems [7, 8, 9].

To avoid retries overloading a service, you can increase and randomize the time between successive retries on the client side (*exponential backoff* [10, 11]) and temporarily stop sending requests to a service that has returned errors or timed out recently (by using a *circuit breaker* [12, 13] or *token bucket* algorithm [14]). The server can also detect when it is approaching overload and start proactively rejecting requests (*load shedding* [15]), or send back responses asking clients to slow down (*backpressure* [1, 16]). The choice of queuing and load balancing algorithms can also make a difference [17].

In terms of performance metrics, the response time is usually what users care about the most, whereas the throughput determines the required computing resources (e.g., how many servers you need) and hence the cost of serving a particular workload. If throughput is likely to increase beyond the current hardware's capability, the capacity needs to be expanded; a system is said to be *scalable* if its maximum throughput can be significantly increased by adding computing resources.

In this section we will focus primarily on response times, and we will return to throughput and scalability in “[Scalability](#)” on page 49.

Latency and Response Time

“Latency” and “response time” are sometimes used interchangeably, but in this book we will use these and a few related terms in a specific way (illustrated in [Figure 2-4](#)):

- The *response time* is what the client sees; it includes all delays incurred anywhere in the system.
- The *service time* is the duration for which the service is actively processing the client's request.
- *Queueing delays* can occur at several points in the flow—for example, after a request is received, it might need to wait until a CPU is available before it can be processed, or a response packet might need to be buffered before it is sent over

the network if other tasks on the same machine are sending a lot of data via the outbound network interface.

- *Latency* is a catchall term for time during which a request is not being actively processed—that is, during which it is *latent*. In particular, *network latency* or *network delay* refers to the time that a request and response spend traveling through the network.

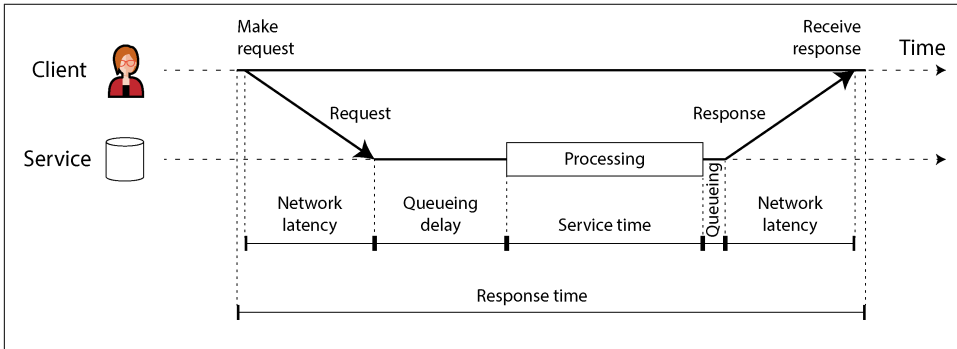


Figure 2-4. Response time, service time, network latency, and queueing delay

In [Figure 2-4](#), time flows from left to right; each communicating node is shown as a horizontal line, and a request or response message is shown as a thick diagonal arrow from one node to another. You will encounter this style of diagram frequently over the course of this book.

The response time can vary significantly from one request to the next, even if you keep making the same request over and over again. Many factors can add random delays—for example, a context switch to a background process, the loss of a network packet and TCP retransmission, a garbage collection pause, a page fault forcing a read from disk, or mechanical vibrations in the server rack [18]. We will discuss this topic in more detail in [“Timeouts and Unbounded Delays” on page 72](#).

Queueing delays often account for a large part of the variability in response times. As a server can process only a small number of things in parallel (limited, for example, by its number of CPU cores), it takes only a small number of slow requests to hold up the processing of subsequent requests—an effect known as *head-of-line blocking*. Even if those subsequent requests have fast service times, the client will see a slow overall response time due to the time waiting for the prior request to complete. The queueing delay is not part of the service time, and for this reason it is important to measure response times on the client side.

Average, Median, and Percentiles

Because the response time varies from one request to the next, we need to think of it not as a single number, but as a *distribution* of values that we can measure. In [Figure 2-5](#), each gray bar represents a request to a service, and its height shows how long that request took. Most requests are reasonably fast, but occasional *outliers* take much longer. Variation in network delay is also known as *jitter*.

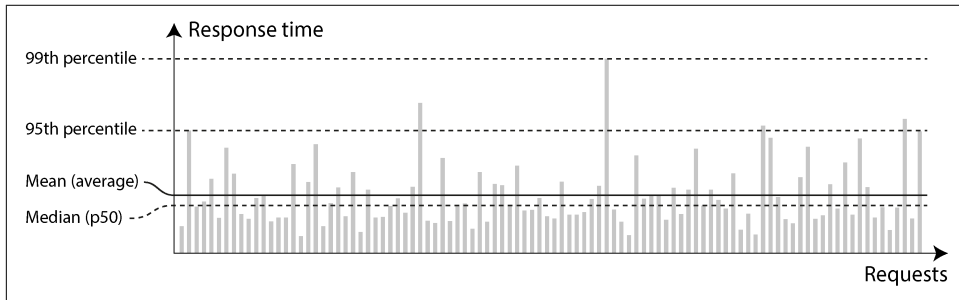


Figure 2-5. Illustrating mean and percentiles: response times for a sample of 100 requests to a service

It's common to report the *average* response time of a service (technically, the *arithmetic mean*, which you find by summing all the response times and dividing by the number of requests). The mean response time is useful for estimating throughput limits [19]. However, the mean is not a very good metric if you want to know your “typical” response time, because it doesn't tell you how many users actually experienced that delay.

Usually it's better to use *percentiles*. If you take your list of response times and sort it from fastest to slowest, the *median* is the halfway point—for example, if your median response time is 200 ms, that means half your requests return in less than 200 milliseconds (ms), and half your requests take longer. This makes the median a good metric if you want to know how long users typically have to wait. The median is also known as the *50th percentile*, sometimes abbreviated as *p50*.

To figure out how bad your outliers are, you can look at higher percentiles: the *95th*, *99th*, and *99.9th percentiles* are common (abbreviated *p95*, *p99*, and *p999*). For example, if the 95th percentile response time is 1.5 seconds, that means 95 out of 100 requests take less than 1.5 seconds, and 5 out of 100 requests take 1.5 seconds or more. This is illustrated in [Figure 2-5](#).

High response-time percentiles, also known as *tail latencies*, are important because they directly affect users' experience of the service. For example, Amazon describes response time requirements for internal services in terms of the 99.9th percentile, even though this affects only 1 in 1,000 requests. This is because the customers with the slowest requests are often those who have the most data on their accounts, as

they have made many purchases—that is, they’re the most valuable customers [20]. It’s important to keep those customers happy by ensuring the website is fast for them.

Optimizing the 99.99th percentile (the slowest 1 in 10,000 requests) was deemed too expensive and found to not yield enough benefit for Amazon’s purposes. Reducing response times at very high percentiles is difficult because they are easily affected by random events outside of your control, and the benefits are diminishing.

The User Impact of Response Times

It seems obvious that a fast service is better for users than a slow service [21]. However, it is surprisingly difficult to get hold of reliable data to quantify the effect that latency has on user behavior.

Some often-cited statistics are unreliable. In 2006, for example, Google reported that a slowdown in search results from 400 ms to 900 ms was associated with a 20% drop in traffic and revenue [22]. However, another Google study from 2009 reported that a 400 ms increase in latency resulted in only 0.6% fewer searches per day [23], and in the same year Bing found that a two-second increase in load time reduced ad revenue by 4.3% [24]. Newer data from these companies appears not to be publicly available.

A more recent Akamai study claims that a 100 ms increase in response time reduced the conversion rate of ecommerce sites by up to 7% [25]; on closer inspection, though, the same study reveals that very *fast* page load times are also correlated with lower conversion rates! This seemingly paradoxical result is explained by the fact that the pages that load fastest are often those that have no useful content (e.g., 404 error pages). However, since the study makes no effort to separate the effects of page content from the effects of load time, its results are probably not meaningful.

A study by Yahoo conducted the following year compared click-through rates on fast-loading versus slow-loading search results, controlling for quality of search results [26]. It reports 20%–30% more clicks on fast searches when the difference between fast and slow responses is 1.25 seconds or more.

Use of Response Time Metrics

High percentiles are especially important in backend services that are called multiple times as part of serving a single end-user request. Even if you make the calls in parallel, the request still needs to wait for the slowest of the parallel calls to complete. It takes just one slow call to make the entire end-user request slow, as illustrated in [Figure 2-6](#). Even if only a small percentage of backend calls are slow, the chance of getting a slow call increases if an end-user request requires multiple backend calls, so a higher proportion of such end-user requests end up being slow (an effect known as *tail latency amplification* [27]).

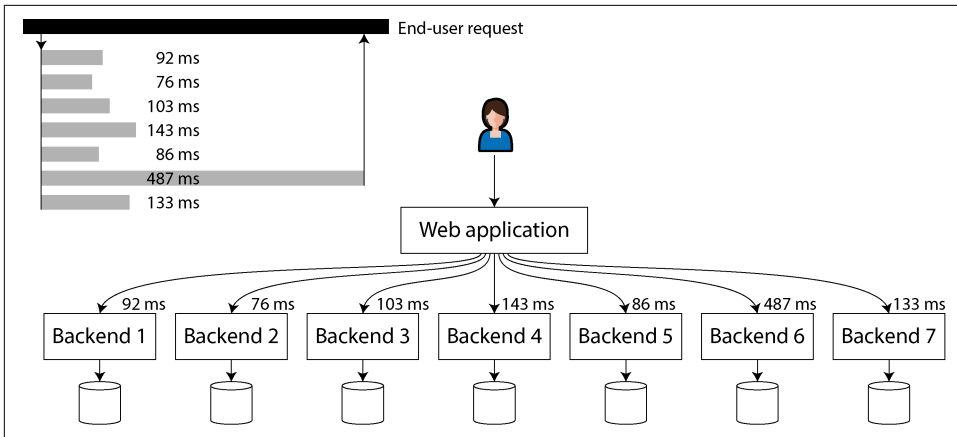


Figure 2-6. When several backend calls are needed to serve a request, just a single slow call can slow down the entire end-user request.

Percentiles are often used in *service level objectives* (SLOs) and *service level agreements* (SLAs) as ways of defining the expected performance and availability of a service [28]. For example, an SLO may set a target for a service to have a median response time of less than 200 ms and a 99th percentile under 1 second, and a target that at least 99.9% of valid requests result in non-error responses. An SLA is a contract that specifies what happens if the SLO is not met (e.g., customers may be entitled to a refund). That's the basic idea, at least; in practice, defining good availability metrics for SLOs and SLAs is not straightforward [29, 30].

Computing Percentiles

If you want to add response time percentiles to the monitoring dashboards for your services, you need to efficiently calculate them on an ongoing basis. For example, you may want to keep a rolling window of response times for requests in the last 10 minutes. Every minute, you calculate the median and various percentiles over the values in that window and plot those metrics on a graph.

The simplest implementation is to keep a list of response times for all requests within the time window and sort that list every minute. If that is too inefficient for you, there are algorithms that can calculate a good approximation of percentiles at minimal CPU and memory cost. Open source percentile estimation libraries include HdrHistogram [31], t-digest [32, 33], OpenHistogram [34], and DDSketch [35].

Beware that averaging percentiles (e.g., to reduce the time resolution or to combine data from several machines) is mathematically meaningless. The right way of aggregating response time data is to add the histograms [36].

Reliability and Fault Tolerance

Everybody has an intuitive idea of what it means for something to be reliable or unreliable. For software, typical expectations include the following:

- The application performs the function that the user expected.
- The application can tolerate the user making mistakes or using the software in unexpected ways.
- Its performance is good enough for the required use case, under the expected load and data volume.
- The system prevents any unauthorized access and abuse.

If all those things together mean “working correctly,” then we can understand *reliability* as meaning, roughly, “continuing to work correctly, even when things go wrong.” To be more precise about things going wrong, we will distinguish between faults and failures [37, 38, 39]:

Fault

A fault occurs when a particular *part* of a system stops working correctly—for example, if a single hard drive malfunctions, or a single machine crashes, or an external service (that the system depends on) has an outage.

Failure

A failure occurs when the system *as a whole* stops providing the required service to the user—in other words, when it does not meet the SLO.

The distinction between faults and failures can be confusing because they are the same thing, just at different levels. For example, if a hard drive stops working, we say that the hard drive has failed; if the system consists of only that one hard drive, it has stopped providing the required service and thus has also failed. However, if the system consists of multiple hard drives, the failure of a single hard drive is only a fault from the point of view of the bigger system, and the bigger system might be able to tolerate that fault by having a copy of the data on another hard drive.

Fault Tolerance

We call a system *fault-tolerant* if it continues providing the required service to users in spite of certain faults occurring. If a system cannot tolerate a certain part becoming faulty, we call that part a *single point of failure* (SPOF), because a fault in that part escalates to cause the failure of the whole system.

For example, in the social network case study, a fault that might happen is that during the fan-out process, a machine involved in updating the materialized timelines crashes or become unavailable. To make this process fault-tolerant, we would need to

ensure that another machine can take over this task without missing any posts that should have been delivered, and without duplicating any posts. (This idea is known as *exactly-once semantics*, and we will examine it in detail in Chapter 12.)

Fault tolerance is always limited to a certain number of certain types of faults. For example, a system might be able to tolerate a maximum of two hard drives failing at the same time, or a maximum of one out of three nodes crashing. It would not make sense to tolerate any number of faults; if all nodes crash, nothing can be done. If the entire planet Earth (and all servers on it) were swallowed by a black hole, tolerance of that fault would require web hosting in space—good luck getting that budget item approved.

Counterintuitively, in such fault-tolerant systems, it can make sense to *increase* the rate of faults by triggering them deliberately—for example, by randomly killing individual processes without warning. This is called *fault injection*. Many critical bugs are actually due to poor error handling [40]; by deliberately inducing faults, you ensure that the fault-tolerance machinery is continually exercised and tested, which can increase your confidence that faults will be handled correctly when they occur naturally. *Chaos engineering* is a discipline that aims to improve confidence in fault-tolerance mechanisms through experiments such as deliberately injecting faults [41].

Although we generally prefer tolerating faults over preventing faults, in some cases where prevention is better than cure (e.g., because no cure exists). This is the case with security matters, for example; if an attacker has compromised a system and gained access to sensitive data, that event cannot be undone. However, this book mostly deals with the kinds of faults that can be cured, as described in the following sections.

Hardware and Software Faults

When we think of causes of system failure, hardware faults quickly come to mind:

- Approximately 2%–5% of magnetic hard drives fail per year [42, 43]; in a storage cluster with 10,000 disks, we should therefore expect on average one disk failure per day. Recent data suggests that disks are getting more reliable, but failure rates remain significant [44].
- Approximately 0.5%–1% of solid state drives (SSDs) fail per year [45]. Small numbers of bit errors are corrected automatically [46], but uncorrectable errors occur approximately once per year per drive, even in drives that are fairly new (i.e., that have experienced little wear). This error rate is higher than that of magnetic hard drives [47, 48].
- Other hardware components (such as power supplies, RAID controllers, and memory modules) also fail, although less frequently than hard drives [49, 50].

- Approximately 1 in 1,000 machines has a CPU core that occasionally computes the wrong result, likely because of manufacturing defects [51, 52, 53]. In some cases an erroneous computation leads to a crash, but in other cases it leads to a program simply returning the wrong result.
- Data in RAM can be corrupted, either because of random events such as cosmic rays or because of permanent physical defects. Even when memory with error-correcting codes (ECC) is used, more than 1% of machines encounter an uncorrectable error in a given year, which typically leads to a crash of the machine and the affected memory module needing to be replaced [54]. Furthermore, certain pathological memory access patterns can flip bits with high probability [55].
- An entire datacenter might become unavailable (e.g., because of a power outage or network misconfiguration) or even be permanently destroyed (e.g., by fire, flood, or earthquake [56]). A solar storm, which induces large electrical currents in long-distance wires when the sun ejects a large mass of charged particles, could damage power grids and undersea network cables [57]. Although such large-scale failures are rare, their impact can be catastrophic if a service cannot tolerate the loss of a datacenter [58].

These events are rare enough that you often don't need to worry about them when working on a small system, as long as you can easily replace hardware that becomes faulty. However, in a large-scale system, hardware faults happen often enough that they become part of normal system operation.

Tolerating hardware faults through redundancy

Our first response to unreliable hardware is usually to add redundancy to the individual hardware components in order to reduce the failure rate of the system. Disks may be set up in a RAID configuration (spreading data across multiple disks in the same machine so that a failed disk does not cause data loss), servers may have dual power supplies and hot-swappable CPUs, and datacenters may have batteries and diesel generators for backup power. Such redundancy can often keep a machine running uninterrupted for years.

Redundancy is most effective when component faults are independent—that is, when the occurrence of one fault does not change the likelihood that another fault will occur. However, experience has shown significant correlations between component failures [43, 59, 60]. Unavailability of an entire server rack or an entire datacenter still happens more often than we would like.

Hardware redundancy increases the uptime of a single machine; however, as discussed in “[Distributed Versus Single-Node Systems](#)” on page 19, using a distributed system has advantages, such as being able to tolerate a complete outage of one datacenter. For this reason, cloud systems tend to focus less on the reliability of individual machines and instead aim to make services highly available by tolerating

faulty nodes at the software level. Cloud providers use *availability zones* to identify which resources are physically co-located; resources in the same place are more likely to fail at the same time than geographically separated resources.

The fault-tolerance techniques we discuss in this book are designed to tolerate the loss of entire machines, racks, or availability zones. They generally work by allowing a machine in one datacenter to take over when a machine in another datacenter fails or becomes unreachable. We will discuss such techniques for fault tolerance in Chapters 6, 10, and various other points in this book.

Systems that can tolerate the loss of entire machines also have operational advantages. A single-server system requires planned downtime if you need to reboot the machine (to apply operating system security patches, for example), whereas a multi-node fault-tolerant system can be patched by restarting one node at a time, without affecting the service for users. This is called a *rolling upgrade*, and we will discuss it further in Chapter 5.

Software faults

Although hardware failures can be weakly correlated, they are still mostly independent—for example, if one disk fails, other disks in the same machine will likely be fine, at least for a while. On the other hand, software faults are often very highly correlated, because it is common for many nodes to run the same software and thus have the same bugs [61, 62]. Such faults are harder to anticipate, and they tend to cause many more system failures than uncorrelated hardware faults [49]. Examples include the following:

- A software bug that causes every node to fail at the same time in particular circumstances. For instance, on June 30, 2012, a leap second caused many Java applications to hang simultaneously because of a bug in the Linux kernel, bringing down several internet services [63]. And because of a firmware bug, all SSDs of certain models suddenly fail after precisely 32,768 hours of operation (less than four years), rendering the data on them unrecoverable [64].
- A runaway process that uses up a shared, limited resource, such as CPU time, memory, disk space, network bandwidth, or threads [65]. For instance, a process that consumes too much memory while processing a large request may be killed by the operating system, or a bug in a client library could cause a much higher request volume than anticipated [66].
- A service that the system depends on slows down, becomes unresponsive, or starts returning corrupted responses.
- An interaction between different systems results in emergent behavior that does not occur when each system is tested in isolation [67].

- Cascading failures, where a problem in one component causes another component to become overloaded and slow down, which in turn brings down another component [68, 69].

The bugs that cause these kinds of software faults often lie dormant for a long time until they are triggered by an unusual set of circumstances. In those circumstances, it is revealed that the software is making some kind of assumption about its environment—and while that assumption is *usually* true, it eventually stops being true for some reason [70, 71].

The problem of systematic faults in software has no quick solution. Lots of small things can help: carefully thinking about assumptions and interactions in the system; thorough testing; ensuring process isolation; allowing processes to crash and restart; avoiding feedback loops such as retry storms (see “[When an Overloaded System Won’t Recover](#)” on page 38); measuring, monitoring, and analyzing system behavior in production.

Humans and Reliability

Humans design and build software systems, and the operators who keep the systems running are also human. Unlike machines, humans don’t just follow rules; one of their strengths is being creative and adaptive in getting their jobs done. However, this characteristic also leads to unpredictability, and sometimes to mistakes that can lead to failures, despite best intentions. For example, one study of large internet services found that configuration changes by operators were the leading cause of outages, whereas hardware faults (servers or network) played a role in only 10%–25% of cases [72].

It is tempting to label such problems as “human error” and to wish that they could be solved by better controlling human behavior through tighter procedures and compliance with rules. However, blaming people for mistakes is counterproductive. What we call “human error” is not really the cause of an incident, but rather a symptom of a problem with the sociotechnical system in which people are trying their best to do their jobs [73]. Often complex systems have emergent behavior, in which unexpected interactions between components may also lead to failures [74].

Various technical measures can help minimize the impact of human mistakes, including thorough testing (both handwritten tests and *property testing* on lots of random inputs) [40], rollback mechanisms for quickly reverting configuration changes, gradual rollouts of new code, detailed and clear monitoring, observability tools for diagnosing production issues (see “[Problems with Distributed Systems](#)” on page 20), and well-designed interfaces that encourage “the right thing” and discourage “the wrong thing.”

However, these all require an investment of time and money, and in the pragmatic reality of everyday business, organizations often prioritize revenue-generating activities over measures that increase their systems' resilience against mistakes. Given a choice between more features and more testing, many organizations understandably choose features. Then, when a preventable mistake inevitably occurs, blaming the person who made the mistake does not make sense; the problem is the organization's priorities.

Increasingly, organizations are adopting a culture of *blameless postmortems*: after an incident, the people involved are encouraged to share full details about what happened, without fear of punishment, since this allows others in the organization to learn how to prevent similar problems in the future [75]. This process may uncover a need to change business priorities, invest in areas that have been neglected, change the incentives for the people involved, or bring another systemic issue to management's attention.

As a general principle, when investigating an incident, you should be suspicious of simplistic answers. "Bob should have been more careful when deploying that change" is not productive, but neither is "We must rewrite the backend in Haskell." Instead, management should take the opportunity to learn the details of how the sociotechnical system works from the point of view of the people who work with it every day, and take steps to improve it based on this feedback [73].

How Important Is Reliability?

Reliability is not just for nuclear power stations and air traffic control; more mundane applications are also expected to work reliably. Bugs in business applications lead to lost productivity (and legal risks if figures are reported incorrectly), and outages of ecommerce sites can have huge costs in terms of lost revenue and damage to reputation.

In many applications, a temporary outage of a few minutes or even a few hours is tolerable [76], but permanent data loss or corruption would be catastrophic. Consider a parent who stores all their pictures and videos of their children in your photo application [77]. How would they feel if that database was suddenly corrupted? Would they know how to restore their collection from a backup?

As another example of how unreliable software can harm people, consider the Post Office Horizon scandal. Between 1999 and 2019, hundreds of people managing Post Office branches in Britain were convicted of theft or fraud because the accounting software showed a shortfall in their accounts. Eventually it became clear that many of these shortfalls were due to bugs in the software, resulting in many of these convictions being overturned [78]. What led to this, probably the largest miscarriage of justice in British history, is an assumption by English law that computers operate correctly (and hence, evidence produced by computers is reliable) unless evidence

exists to the contrary [79]. Software engineers may laugh at the idea that software could ever be bug-free, but this is little solace to the people who were wrongfully imprisoned, declared bankruptcy, or even committed suicide as a result of a wrongful conviction due to an unreliable computer system.

In some situations we may choose to sacrifice reliability in order to reduce development cost (e.g., when developing a prototype product for an unproven market)—but we should be very conscious of when we are cutting corners and keep in mind the potential consequences.

Scalability

Even if a system is working reliably today, that doesn't mean it will necessarily work reliably in the future. One common reason for degradation is increased load. Perhaps the system has grown from 10,000 concurrent users to 100,000 concurrent users, or from 1 million to 10 million. Perhaps it is processing much larger volumes of data than it did before.

Scalability is the term we use to describe a system's ability to cope with increased load. Sometimes, when discussing scalability, people make comments along the lines of, "You're not Google or Amazon. Stop worrying about scale and just use a relational database." Whether this maxim applies to you depends on the type of application you are building.

If you are building a new product that currently has only a small number of users, perhaps at a startup, the overriding engineering goal is usually to keep the system as simple and flexible as possible so that you can easily modify and adapt the features of your product as you learn more about customers' needs [80]. In such an environment, it is counterproductive to worry about hypothetical scale that might be needed in the future. In the best case, investments in scalability are wasted effort and premature optimization; in the worst case, they lock you into an inflexible design and make it harder to evolve your application.

Scalability is not a one-dimensional label—it is meaningless to say "X is scalable" or "Y doesn't scale." Rather, discussing scalability means considering questions like these:

- If the system grows in a particular way, what are our options for coping with the growth?
- How can we add computing resources to handle the additional load?
- Based on current growth projections, when will we hit the limits of our current architecture?

If you succeed in making your application popular, and therefore are handling a growing amount of load, you will learn where your performance bottlenecks lie and along which dimensions you need to scale. At that point, it's time to start worrying about techniques for scalability.

Understanding Load

First, you need a clear understanding of the current load on the system. Only then can you discuss growth questions (“What happens if our load doubles?”). Often this will be a measure of throughput—for example, the number of requests per second to a service, the number of gigabytes of new data arriving per day, or the number of shopping cart checkouts per hour. Sometimes you care about the peak of a variable quantity, such as the number of simultaneously online users in our social network case study.

Often other statistical characteristics of the load affect the access patterns and hence the scalability requirements. For example, you may need to know the ratio of reads to writes in a database, the hit rate on a cache, or the number of data items per user (followers, in our case study). Perhaps the average case is what matters for you, or perhaps your bottleneck is dominated by a small number of extreme cases. It all depends on the details of your particular application.

Once you understand the load on your system, you can investigate what happens when the load increases. You can look at this in two ways:

- When you increase the load in a certain way and keep the system resources (CPUs, memory, network bandwidth, etc.) unchanged, how is the performance of your system affected?
- When you increase the load in a certain way, how much do you need to increase the resources if you want to keep performance unchanged?

Usually the goal is to keep the performance of the system within the requirements of the SLA (see [“Use of Response Time Metrics” on page 41](#)) while also minimizing the cost of running the system. The greater the required computing resources, the higher the cost. Some types of hardware might be more cost-effective than others, and these factors may change over time as new types of hardware become available.

If doubling the resources will enable you to handle twice the load while keeping performance the same, we say that you have *linear scalability*, and this is considered a good thing. Occasionally it is possible to handle twice the load with less than double the resources, because of economies of scale or a better distribution of peak load [81, 82]. Much more likely is that the cost grows faster than linearly. There may be many reasons for the inefficiency; for example, if you have a lot of data, processing a single

write request may involve more work than if you have a small amount of data, even if the size of the request is the same.

Shared-Memory, Shared-Disk, and Shared-Nothing Architectures

The simplest way of increasing the hardware resources of a service is to move it to a more powerful machine. Individual CPU cores are no longer getting significantly faster, but you can buy a machine (or rent a cloud instance) with more CPU cores, more RAM, and more disk space. This approach is called *vertical scaling* or *scaling up*.

You can get parallelism on a single machine by using multiple processes or threads. All the threads belonging to the same process can access the same RAM, and hence this approach is also called a *shared-memory architecture*. The problem with a shared-memory approach is that the cost grows faster than linearly; a high-end machine with twice the hardware resources of a lower-spec machine typically costs significantly more than twice as much. And because of bottlenecks, that machine is unlikely to actually be able to handle twice the load.

Another approach is the *shared-disk architecture*, which uses several machines with independent CPUs and RAM but stores data on an array of disks that is shared among the machines, which are connected via a fast network: *network-attached storage* (NAS) or a *storage area network* (SAN). This architecture has traditionally been used for on-premises data warehousing workloads, but contention and the overhead of locking limit the scalability of the shared-disk approach [83].

By contrast, the *shared-nothing architecture* [84] (also called *horizontal scaling* or *scaling out*) involves a distributed system with multiple nodes, each of which has its own CPUs, RAM, and disks. Any coordination between nodes is done at the software level, via a conventional network.

The advantages of this approach, which has gained popularity in recent years, are that it has the potential to scale linearly, it can use whatever hardware offers the best price/performance ratio (especially in the cloud), it can more easily adjust its hardware resources as load increases or decreases, and it can achieve greater fault tolerance by distributing the system across multiple datacenters and regions. The downsides are that it requires explicit sharding (see Chapter 7) and incurs all the complexity of distributed systems (discussed in Chapter 9).

Some cloud native database systems use separate services for storage and transaction execution (see “[Separation of storage and compute](#)” on page 16), with multiple compute nodes sharing access to the same storage service. This model has some similarity to a shared-disk architecture, but it avoids the scalability problems of older systems. Instead of providing a filesystem (NAS) or block device (SAN) abstraction, the storage service offers a specialized API that is designed for the specific needs of the database [85].

Principles for Scalability

The architecture of systems that operate at large scale is usually highly specific to the application. There is no such thing as a generic, one-size-fits-all scalable architecture (informally known as *magic scaling sauce*). For example, a system designed to handle 100,000 requests per second, each 1 kB in size, looks very different from a system designed for 3 requests per minute, each 2 GB in size—even though the two systems have the same data throughput (100 MB/second).

Moreover, an architecture that is appropriate for one level of load is unlikely to cope with 10 times that load. If you are working on a fast-growing service, it is therefore probable that you will need to rethink your architecture on every order of magnitude load increase. As the needs of the application are likely to evolve, it is usually not worth planning future scaling needs more than one order of magnitude in advance.

A good general principle for scalability is to break a system into smaller components that can operate largely independently from one another. This is the underlying principle behind microservices (see “[Microservices and Serverless](#)” on page 21), sharding (Chapter 7), stream processing (Chapter 12), and shared-nothing architectures. The challenge lies in knowing where to draw the line between things that should be together and things that should be apart. Design guidelines for microservices can be found in other books [86], and we discuss sharding of shared-nothing systems in Chapter 7.

Another good principle is not to make things more complicated than necessary. If a single-machine database will do the job, it’s probably preferable to a complicated distributed setup. Autoscaling systems (which automatically add or remove resources in response to demand) are cool, but if your load is fairly predictable, a manually scaled system may have fewer operational surprises (see Chapter 7). A system with 5 services is simpler than one with 50. Good architectures usually involve a pragmatic mixture of approaches.

Maintainability

Software does not wear out or suffer material fatigue, so it does not break in the same ways as mechanical objects do. But the requirements for an application frequently evolve, the environment that the software runs in changes (such as its dependencies and the underlying platform), and it may have bugs that need fixing.

It is widely recognized that the majority of the cost of software is not in its initial development but in its ongoing maintenance—fixing bugs, keeping its systems operational, investigating failures, adapting it to new platforms, modifying it for new use cases, repaying technical debt, and adding new features [87, 88].

Maintenance can be complex, especially for legacy systems. A system that has been successfully running for a long time may well use outdated technologies that not many engineers understand today (such as mainframes and COBOL code), and institutional knowledge of how and why the system was designed in a certain way may have been lost as people have left the organization. Fixing other people's mistakes might also be necessary. Because computer systems are often intertwined with the human organizations they support, maintenance of such systems is as much a people problem as a technical one [89].

Every system we create today will one day become a legacy system if it is valuable enough to survive for a long time. To minimize the pain for future generations who need to maintain our software, we should design it with maintenance in mind. Although we cannot always predict which decisions might create maintenance headaches in the future, in this book we will pay attention to several principles that are widely applicable:

Operability

Make it easy for the organization to keep the system running smoothly.

Simplicity

Make it easy for new engineers to understand the system, by implementing it using well-understood, consistent patterns and structures and avoiding unnecessary complexity.

Evolvability

Make it easy for engineers to make changes to the system in the future, adapting it and extending it for unanticipated use cases as requirements change.

Operability: Making Life Easy for Operations

We previously discussed the role of operations in “[Operations in the Cloud Era](#)” on [page 17](#), and we saw that human processes are at least as important for reliable operations as software tools. In fact, it has been suggested that “good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations” [62].

In large-scale systems consisting of many thousands of machines, manual maintenance would be unreasonably expensive, and automation is essential. However, automation can be a two-edged sword. There will always be edge cases (such as rare failure scenarios) that require manual intervention from the operations team, and since the cases that cannot be handled automatically tend to be the most complex, greater automation requires a *more* skilled operations team that can resolve those issues [90].

Additionally, an automated system that goes wrong is often harder to troubleshoot than a system that relies on an operator to perform some actions manually. For that

reason, more automation is not always better for operability. However, some amount of automation is important—the sweet spot will depend on the specifics of your particular application and organization.

Good operability means making routine tasks easy, allowing the operations team to focus on high-value activities. Data systems can help by doing the following [91]:

- Allowing monitoring tools to check the system’s key metrics and supporting observability tools (see “[Problems with Distributed Systems](#)” on page 20) to give insights into the system’s runtime behavior. A variety of commercial and open source tools can help here [92].
- Avoiding dependency on individual machines (allowing machines to be taken down for maintenance while the system as a whole continues running uninterrupted).
- Providing good documentation and an easy-to-understand operational model (“If I do X, Y will happen”).
- Providing good default behavior, but also giving administrators the freedom to override defaults when needed.
- Self-healing where appropriate, but also giving administrators manual control over the system state when needed.
- Exhibiting predictable behavior, minimizing surprises.

Simplicity: Managing Complexity

Small software projects can have delightfully simple and expressive code, but as projects get larger, they often become very complex and difficult to understand. This complexity slows down everyone who needs to work on the system, further increasing the cost of maintenance. A software project mired in complexity is sometimes described as a *big ball of mud* [93].

When complexity makes maintenance hard, budgets and schedules are often overrun. In complex software, there is also a greater risk of introducing bugs when making a change. When the system is harder for developers to understand and reason about, hidden assumptions, unintended consequences, and unexpected interactions are more easily overlooked [71]. Conversely, reducing complexity greatly improves the maintainability of software, and thus simplicity should be a key goal for the systems we build.

Simple systems are easier to understand, so we should try to solve a given problem in the simplest way possible. Unfortunately, this is easier said than done. Whether something is simple is often a subjective matter, as there is no objective standard of simplicity [94]. For example, one system may hide a complex implementation behind

a simple interface, whereas another may have a simple implementation that exposes more internal detail to its users—which one is simpler?

One attempt at reasoning about complexity breaks it into two categories: essential and accidental [95]. The idea is that *essential* complexity is inherent in the problem domain of the application, while *accidental* complexity arises only because of limitations of our tooling. Unfortunately, this distinction is also flawed, because boundaries between the essential and the accidental shift as our tooling evolves [96].

One of the best tools we have for managing complexity is *abstraction*. A good abstraction can hide a great deal of implementation detail behind a clean, simple-to-understand façade. A good abstraction can also be used for a wide range of applications. Not only is this reuse more efficient than reimplementing a similar thing multiple times, but it also leads to higher-quality software, as quality improvements in the abstracted component benefit all applications that use it.

For example, high-level programming languages are abstractions that hide machine code, CPU registers, and system calls. SQL is an abstraction that hides complex on-disk and in-memory data structures, concurrent requests from other clients, and inconsistencies after crashes. Of course, when programming in a high-level language, we are still using machine code; we are just not using it *directly*, because the programming language abstraction saves us from having to think about it.

Abstractions for application code that aim to reduce its complexity can be created using methodologies such as *design patterns* [97] and *domain-driven design* (DDD) [98]. This book is not about such application-specific abstractions, but rather about general-purpose abstractions on top of which you can build your applications, such as database transactions, indexes, and event logs. If you want to use techniques such as DDD, you can implement them on top of the foundations described in this book.

Evolvability: Making Change Easy

It's extremely unlikely that your system's requirements will remain unchanged forever. They are much more likely to be in constant flux: you learn new facts, previously unanticipated use cases emerge, business priorities change, users request new features, new platforms replace old platforms, legal or regulatory requirements change, growth of the system forces architectural changes, etc.

In terms of organizational processes, *Agile* working patterns provide a framework for adapting to change. The Agile community has also developed technical tools and processes that are helpful when building software in a frequently changing environment, such as test-driven development (TDD) and refactoring. In this book, we search for ways of increasing agility at the level of a system consisting of several applications or services with different characteristics.

The ease with which you can modify a data system and adapt it to changing requirements is closely linked to its simplicity and its abstractions. Loosely coupled, simple systems are usually easier to modify than tightly coupled, complex ones. Since this is such an important idea, we will use a different word to refer to agility on a data system level: *evolvability* [99].

One major factor that makes change difficult in large systems is irreversibility [100]. For example, say you are migrating from one database to another. If you cannot switch back to the old system in case of problems with the new one, the stakes are much higher than if you can easily go back. Therefore, irreversible actions need to be taken very carefully. Minimizing irreversibility improves flexibility.

Summary

In this chapter we examined several examples of nonfunctional requirements: performance, reliability, scalability, and maintainability. Through these topics, we also encountered principles and terminology that will be relevant throughout the rest of the book.

We started with a case study of implementing home timelines in a social network, which illustrated some of the challenges that arise at scale. We then discussed how to measure performance (e.g., using response time percentiles) and the load on a system (e.g., using throughput metrics), and how these metrics are used in SLAs. Scalability is a closely related concept: it focuses on ensuring that performance stays the same when the load grows. We saw some general principles for scalability, such as breaking a task into smaller parts that can operate independently, and we will dive into greater technical detail on scalability techniques in the following chapters.

To achieve reliability, you can use fault-tolerance techniques, which allow a system to continue providing its services even if a component (e.g., a disk, a machine, or another service) is faulty. We saw examples of hardware faults that can occur and distinguished them from software faults, which can be harder to deal with because they are often strongly correlated. Another aspect of achieving reliability is to build resilience against humans making mistakes, and we saw blameless postmortems as a technique for learning from incidents.

Finally, we examined several facets of maintainability, including supporting the work of operations teams, managing complexity, and making it easy to evolve an application's functionality over time. There are no easy answers to how to achieve these goals, but one approach that can help is to build applications using well-understood building blocks that provide useful abstractions. The rest of this book will cover a selection of building blocks that have proved to be valuable in practice.

References

- [1] Mike Cvet. “How We Learned to Stop Worrying and Love Fan-in at Twitter.” At *QCon San Francisco*, December 2016.
- [2] Raffi Krikorian. “Timelines at Scale.” At *QCon San Francisco*, November 2012. Archived at perma.cc/V9G5-KLYK
- [3] Twitter. “Twitter’s Recommendation Algorithm.” *blog.x.com*, March 2023. Archived at perma.cc/L5GT-229T
- [4] Raffi Krikorian. “New Tweets per Second Record, and How!” *blog.x.com*, August 2013. Archived at perma.cc/6JZN-XJYN
- [5] Jaz Volpert. “When Imperfect Systems Are Good, Actually: Bluesky’s Lossy Timelines.” *jazco.dev*, February 2025. Archived at perma.cc/2PVE-L2MX
- [6] Samuel Axon. “3% of Twitter’s Servers Dedicated to Justin Bieber.” *mashable.com*, September 2010. Archived at perma.cc/F35N-CGVX
- [7] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. “Meta-stable Failures in Distributed Systems.” At *Workshop on Hot Topics in Operating Systems (HotOS)*, May 2021. [doi:10.1145/3458336.3465286](https://doi.org/10.1145/3458336.3465286)
- [8] Marc Brooker. “Metastability and Distributed Systems.” *brooker.co.za*, May 2021. Archived at perma.cc/7FGJ-7XRK
- [9] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. “Metastable Failures in the Wild.” At *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2022.
- [10] Marc Brooker. “Exponential Backoff and Jitter.” *aws.amazon.com*, March 2015. Archived at perma.cc/R6MS-AZKH
- [11] Marc Brooker. “What Is Backoff For?” *brooker.co.za*, August 2022. Archived at perma.cc/PW9N-55Q5
- [12] Michael T. Nygard. *Release It!*, 2nd edition. Pragmatic Bookshelf, 2018. ISBN: 9781680502398
- [13] Frank Chen. “Slowing Down to Speed Up—Circuit Breakers for Slack’s CI/CD.” *slack.engineering*, August 2022. Archived at perma.cc/5FGS-ZPH3
- [14] Marc Brooker. “Fixing Retries with Token Buckets and Circuit Breakers.” *brooker.co.za*, February 2022. Archived at perma.cc/MD6N-GW26
- [15] David Yanacek. “Using Load Shedding to Avoid Overload.” Amazon Builders’ Library, *aws.amazon.com*. Archived at perma.cc/9SAW-68MP

- [16] Matthew Sackman. “Pushing Back.” *wellquite.org*, May 2016. Archived at perma.cc/3KCZ-RUFY
- [17] Dmitry Kopytkov and Patrick Lee. “Meet Bandid, the Dropbox Service Proxy.” *dropbox.tech*, March 2018. Archived at perma.cc/KUU6-YG4S
- [18] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, Gary Grider, Parks M. Fields, Kevin Harms, Robert B. Ross, Andree Jacobson, Robert Ricci, Kirk Webb, Peter Alvaro, H. Biralı Runesha, Mingzhe Hao, and Huaicheng Li. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems.” At *16th USENIX Conference on File and Storage Technologies*, February 2018.
- [19] Marc Brooker. “Is the Mean Really Useless?” *brooker.co.za*, December 2017. Archived at perma.cc/U5AE-CVEM
- [20] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. “Dynamo: Amazon’s Highly Available Key-Value Store.” At *21st ACM Symposium on Operating Systems Principles (SOSP)*, October 2007. [doi:10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281)
- [21] Kathryn Whitenon. “The Need for Speed, 23 Years Later.” *nngroup.com*, May 2020. Archived at perma.cc/C4ER-LZYA
- [22] Greg Linden. “Marissa Mayer at Web 2.0.” *glinden.blogspot.com*, November 2005. Archived at perma.cc/V7EA-3VXB
- [23] Jake Brutlag. “Speed Matters for Google Web Search.” *services.google.com*, June 2009. Archived at perma.cc/BK7R-X7M2
- [24] Eric Schurman and Jake Brutlag. “Performance Related Changes and Their User Impact.” Talk at *Velocity 2009*.
- [25] Akamai Technologies, Inc. “The State of Online Retail Performance.” *akamai.com*, April 2017. Archived at perma.cc/UEK2-HYCS
- [26] Xiao Bai, Ioannis Arapakis, B. Barla Cambazoglu, and Ana Freire. “Understanding and Leveraging the Impact of Response Latency on User Behaviour in Web Search.” *ACM Transactions on Information Systems*, volume 36, issue 2, article 21, April 2018. [doi:10.1145/3106372](https://doi.org/10.1145/3106372)
- [27] Jeffrey Dean and Luiz André Barroso. “The Tail at Scale.” *Communications of the ACM*, volume 56, issue 2, pages 74–80, February 2013. [doi:10.1145/2408776.2408794](https://doi.org/10.1145/2408776.2408794)
- [28] Alex Hidalgo. *Implementing Service Level Objectives: A Practical Guide to SLIs, SLOs, and Error Budgets*. O’Reilly Media, 2020. ISBN: 9781492076813

- [29] Jeffrey C. Mogul and John Wilkes. “Nines Are Not Enough: Meaningful Metrics for Clouds.” At *17th Workshop on Hot Topics in Operating Systems (HotOS)*, May 2019. [doi:10.1145/3317550.3321432](https://doi.org/10.1145/3317550.3321432)
- [30] Tamás Hauer, Philipp Hoffmann, John Lunney, Dan Ardelean, and Amer Diwan. “Meaningful Availability.” At *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February 2020.
- [31] Gil Tene. “HdrHistogram: A High Dynamic Range Histogram.” [hdrhistogram.github.io/HdrHistogram](https://github.com/HdrHistogram)
- [32] Ted Dunning. “The *t*-digest: Efficient Estimates of Distributions.” *Software Impacts*, volume 7, article 100049, February 2021. [doi:10.1016/j.simpa.2020.100049](https://doi.org/10.1016/j.simpa.2020.100049)
- [33] David Kohn. “How Percentile Approximation Works (and Why It’s More Useful than Averages).” *timescale.com*, September 2021. Archived at perma.cc/3PDP-NR8B
- [34] Heinrich Hartmann and Theo Schlossnagle. “Circllhist—A Log-Linear Histogram Data Structure for IT Infrastructure Monitoring.” *arXiv:2001.06561*, January 2020.
- [35] Charles Masson, Jee E. Rim, and Homin K. Lee. “DDSketch: A Fast and Fully-Mergeable Quantile Sketch with Relative-Error Guarantees.” *Proceedings of the VLDB Endowment*, volume 12, issue 12, pages 2195–2205, August 2019. [doi:10.14778/3352063.3352135](https://doi.org/10.14778/3352063.3352135)
- [36] Baron Schwartz. “Why Percentiles Don’t Work the Way You Think.” *solarwinds.com*, November 2016. Archived at perma.cc/469T-6UGB
- [37] Walter L. Heimerdinger and Charles B. Weinstock. “A Conceptual Framework for System Fault Tolerance.” Technical Report CMU/SEI-92-TR-033, Software Engineering Institute, Carnegie Mellon University, October 1992. Archived at perma.cc/GD2V-DMJW
- [38] Felix C. Gärtner. “Fundamentals of Fault-Tolerant Distributed Computing in Asynchronous Environments.” *ACM Computing Surveys*, volume 31, issue 1, pages 1–26, March 1999. [doi:10.1145/311531.311532](https://doi.org/10.1145/311531.311532)
- [39] Algirdas Avizienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. “Basic Concepts and Taxonomy of Dependable and Secure Computing.” *IEEE Transactions on Dependable and Secure Computing*, volume 1, issue 1, pages 11–33, January 2004. [doi:10.1109/TDSC.2004.2](https://doi.org/10.1109/TDSC.2004.2)
- [40] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. “Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems.” At *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2014.

- [41] Casey Rosenthal and Nora Jones. *Chaos Engineering*. O'Reilly Media, 2020. ISBN: 9781492043867
- [42] Eduardo Pinheiro, Wolf-Dietrich Weber, and Luiz Andre Barroso. “Failure Trends in a Large Disk Drive Population.” At *5th USENIX Conference on File and Storage Technologies* (FAST), February 2007.
- [43] Bianca Schroeder and Garth A. Gibson. “Disk Failures in the Real World: What Does an Mttf of 1,000,000 Hours Mean to You?” At *5th USENIX Conference on File and Storage Technologies* (FAST), February 2007.
- [44] Andy Klein. “Backblaze Drive Stats for Q2 2021.” *backblaze.com*, August 2021. Archived at perma.cc/2943-UD5E
- [45] Iyswarya Narayanan, Di Wang, Myeongjae Jeon, Bikash Sharma, Laura Caulfield, Anand Sivasubramaniam, Ben Cutler, Jie Liu, Badriddine Khessib, and Kushagra Vaid. “SSD Failures in Datacenters: What? When? And Why?” At *9th ACM International on Systems and Storage Conference* (SYSTOR), June 2016. [doi:10.1145/2928275.2928278](https://doi.org/10.1145/2928275.2928278)
- [46] Alibaba Cloud Storage Team. “Storage System Design Analysis: Factors Affecting NVMe SSD Performance (1).” *alibabacloud.com*, January 2019. Archived at archive.org
- [47] Bianca Schroeder, Raghav Lagisetty, and Arif Merchant. “Flash Reliability in Production: The Expected and the Unexpected.” At *14th USENIX Conference on File and Storage Technologies* (FAST), February 2016.
- [48] Jacob Alter, Ji Xue, Alma Dimnaku, and Evgenia Smirni. “SSD Failures in the Field: Symptoms, Causes, and Prediction Models.” At *International Conference for High Performance Computing, Networking, Storage and Analysis* (SC), November 2019. [doi:10.1145/3295500.3356172](https://doi.org/10.1145/3295500.3356172)
- [49] Daniel Ford, François Labelle, Florentina I. Popovici, Murray Stokely, Van-Anh Truong, Luiz Barroso, Carrie Grimes, and Sean Quinlan. “Availability in Globally Distributed Storage Systems.” At *9th USENIX Symposium on Operating Systems Design and Implementation* (OSDI), October 2010.
- [50] Kashi Venkatesh Vishwanath and Nachiappan Nagappan. “Characterizing Cloud Computing Hardware Reliability.” At *1st ACM Symposium on Cloud Computing* (SoCC), June 2010. [doi:10.1145/1807128.1807161](https://doi.org/10.1145/1807128.1807161)
- [51] Peter H. Hochschild, Paul Turner, Jeffrey C. Mogul, Rama Govindaraju, Parthasarathy Ranganathan, David E. Culler, and Amin Vahdat. “Cores That Don't Count.” At *Workshop on Hot Topics in Operating Systems* (HotOS), June 2021. [doi:10.1145/3458336.3465297](https://doi.org/10.1145/3458336.3465297)

- [52] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, Chris Mason, Tejasvi Chakravarthy, Bharath Muthiah, and Sriram Sankar. [Silent Data Corruptions at Scale.](#) *arXiv:2102.11245*, February 2021.
- [53] Diogo Behrens, Marco Serafini, Sergei Arnautov, Flavio P. Junqueira, and Christof Fetzer. [“Scalable Error Isolation for Distributed Systems.”](#) At *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, May 2015.
- [54] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. [“DRAM Errors in the Wild: A Large-Scale Field Study”](#) At *11th International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS)*, June 2009. [doi:10.1145/1555349.1555372](#)
- [55] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. [“Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors.”](#) At *41st Annual International Symposium on Computer Architecture (ISCA)*, June 2014. [doi:10.5555/2665671.2665726](#)
- [56] Tim Bray. [“Worst Case.”](#) *tbray.org*, October 2021. Archived at [perma.cc/4QQM-RTHN](#)
- [57] Sangeetha Abdu Jyothi. [“Solar Superstorms: Planning for an Internet Apocalypse.”](#) At *ACM SIGCOMM Conference*, August 2021. [doi:10.1145/3452296.3472916](#)
- [58] Adrian Cockcroft. [“Failure Modes and Continuous Resilience.”](#) *adrianco.medium.com*, November 2019. Archived at [perma.cc/7SYS-BVJP](#)
- [59] Shujie Han, Patrick P. C. Lee, Fan Xu, Yi Liu, Cheng He, and Jiongzhou Liu. [“An In-Depth Study of Correlated Failures in Production SSD-Based Data Centers.”](#) At *19th USENIX Conference on File and Storage Technologies (FAST)*, February 2021.
- [60] Edmund B. Nightingale, John R. Douceur, and Vince Orgovan. [“Cycles, Cells and Platters: An Empirical Analysis of Hardware Failures on a Million Consumer PCs.”](#) At *6th European Conference on Computer Systems (EuroSys)*, April 2011. [doi:10.1145/1966445.1966477](#)
- [61] Haryadi S. Gunawi, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patanajakane, Thanh Do, Jeffry Adityatama, Kurnia J. Eliazar, Agung Laksono, Jeffrey F. Lukman, Vincentius Martin, and Anang D. Satria. [“What Bugs Live in the Cloud? A Study of 3000+ Issues in Cloud Systems.”](#) At *5th ACM Symposium on Cloud Computing (SoCC)*, November 2014. [doi:10.1145/2670979.2670986](#)
- [62] Jay Kreps. [“Getting Real About Distributed System Reliability.”](#) *blog.empathy-box.com*, March 2012. Archived at [perma.cc/9B5Q-AEBW](#)
- [63] Nelson Minar. [“Leap Second Crashes Half the Internet.”](#) *somebits.com*, July 2012. Archived at [perma.cc/2WB8-D6EU](#)

- [64] Hewlett Packard Enterprise. “Support Alerts—Customer Bulletin a00092491en_us.” *support.hpe.com*, November 2019. Archived at perma.cc/S5F6-7ZAC
- [65] Lorin Hochstein. “Awesome Limits.” *github.com*, November 2020. Archived at perma.cc/3R5M-E5Q4
- [66] Caitie McCaffrey. “Clients Are Jerks: AKA How Halo 4 DoSed the Services at Launch & How We Survived.” *caitiem.com*, June 2015. Archived at perma.cc/MXX4-W373
- [67] Lilia Tang, Chaitanya Bhandari, Yongle Zhang, Anna Karanika, Shuyang Ji, Indranil Gupta, and Tianyin Xu. “Fail Through the Cracks: Cross-System Interaction Failures in Modern Cloud Systems.” At *18th European Conference on Computer Systems* (EuroSys), May 2023. [doi:10.1145/3552326.3587448](https://doi.org/10.1145/3552326.3587448)
- [68] Mike Ulrich. *Addressing Cascading Failures*. In Betsy Beyer, Jennifer Petoff, Chris Jones, and Niall Richard Murphy (ed). *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. ISBN: 9781491929124
- [69] Harri Faßbender. “Cascading Failures in Large-Scale Distributed Systems.” *blog.mi.hdm-stuttgart.de*, March 2022. Archived at perma.cc/K7VY-YJRX
- [70] Richard I. Cook. “How Complex Systems Fail.” Cognitive Technologies Laboratory, April 2000. Archived at perma.cc/RDS6-2YVA
- [71] David D. Woods. “STELLA: Report from the SNAFUcatchers Workshop on Coping with Complexity.” *snafucatchers.github.io*, March 2017. Archived at archive.org
- [72] David Oppenheimer, Archana Ganapathi, and David A. Patterson. “Why Do Internet Services Fail, and What Can Be Done About It?” At *4th USENIX Symposium on Internet Technologies and Systems* (USITS), March 2003.
- [73] Sidney Dekker. *The Field Guide to Understanding “Human Error”*, 3rd edition. CRC Press, 2017. ISBN: 9781472439055
- [74] Sidney Dekker. *Drift into Failure: From Hunting Broken Components to Understanding Complex Systems*. CRC Press, 2011. ISBN: 9781315257396
- [75] John Allspaw. “Blameless PostMortems and a Just Culture.” *etsy.com*, May 2012. Archived at perma.cc/YMJ7-NTAP
- [76] Itzy Sabo. “Uptime Guarantees—A Pragmatic Perspective.” *world.hey.com*, March 2023. Archived at perma.cc/F7TU-78JB
- [77] Michael Jurewitz. “The Human Impact of Bugs.” *jury.me*, March 2013. Archived at perma.cc/5KQ4-VDYL

- [78] Mark Halper. “How Software Bugs Led to ‘One of the Greatest Miscarriages of Justice’ in British History.” *Communications of the ACM*, volume 68, issue 3, pages 12–14, January 2025. [doi:10.1145/3703779](https://doi.org/10.1145/3703779)
- [79] Nicholas Bohm, James Christie, Peter Bernard Ladkin, Bev Littlewood, Paul Marshall, Stephen Mason, Martin Newby, Steven J. Murdoch, Harold Thimbleby, and Martyn Thomas. “The Legal Rule That Computers Are Presumed to be Operating Correctly—Unforeseen and Unjust Consequences.” Briefing note, benthamsgaze.org, June 2022. Archived at perma.cc/WQ6X-TMW4
- [80] Dan McKinley. “Choose Boring Technology.” mcfunley.com, March 2015. Archived at perma.cc/7QW7-J4YP
- [81] Andy Warfield. “Building and Operating a Pretty Big Storage System Called S3.” allthingsdistributed.com, July 2023. Archived at perma.cc/7LPK-TP7V
- [82] Marc Brooker. “Surprising Scalability of Multitenancy.” brooker.co.za, March 2023. Archived at perma.cc/ZZD9-VV8T
- [83] Ben Stopford. “Shared Nothing vs. Shared Disk Architectures: An Independent View.” benstopford.com, November 2009. Archived at perma.cc/7BXH-EDUR
- [84] Michael Stonebraker. “The Case for Shared Nothing.” *IEEE Database Engineering Bulletin*, volume 9, issue 1, pages 4–9, March 1986. perma.cc/P9YL-C4PS
- [85] Panagiotis Antonopoulos, Alex Budovski, Cristian Diaconu, Alejandro Hernandez Saenz, Jack Hu, Hanuma Kodavalla, Donald Kossmann, Sandeep Lingam, Umar Farooq Minhas, Naveen Prakash, Vijendra Purohit, Hugh Qu, Chaitanya Sreenivas Ravela, Krystyna Reisteter, Sheetal Shrotri, Dixin Tang, and Vikram Wakade. “Socrates: The New SQL Server in the Cloud.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2019. [doi:10.1145/3299869.3314047](https://doi.org/10.1145/3299869.3314047)
- [86] Sam Newman. *Building Microservices*, 2nd edition. O’Reilly Media, 2021. ISBN: 9781492034025
- [87] Nathan Ensmenger. “When Good Software Goes Bad: The Surprising Durability of an Ephemeral Technology.” At *The Maintainers Conference*, April 2016. Archived at perma.cc/ZXT4-HGZB
- [88] Robert L. Glass. *Facts and Fallacies of Software Engineering*. Addison-Wesley Professional, 2002. ISBN: 9780321117427
- [89] Marianne Bellotti. *Kill It with Fire*. No Starch Press, 2021. ISBN: 9781718501188
- [90] Lisanne Bainbridge. “Ironies of Automation.” *Automatica*, volume 19, issue 6, pages 775–779, November 1983. [doi:10.1016/0005-1098\(83\)90046-8](https://doi.org/10.1016/0005-1098(83)90046-8)
- [91] James Hamilton. “On Designing and Deploying Internet-Scale Services.” At *21st Large Installation System Administration Conference (LISA)*, November 2007.

- [92] Dotan Horovits. “Open Source for Better Observability.” *horovits.medium.com*, October 2021. Archived at perma.cc/R2HD-U2ZT
- [93] Brian Foote and Joseph Yoder. “Big Ball of Mud.” At *4th Conference on Pattern Languages of Programs (PLoP)*, September 1997. Archived at perma.cc/4GUP-2PBV
- [94] Marc Brooker. “What Is a Simple System?” *brooker.co.za*, May 2022. Archived at perma.cc/U72T-BFVE
- [95] Frederick P. Brooks. “No Silver Bullet—Essence and Accident in Software Engineering.” In *The Mythical Man-Month*, Anniversary edition, Addison-Wesley, 1995. ISBN: 9780201835953
- [96] Dan Luu. “Against Essential and Accidental Complexity.” *danluu.com*, December 2020. Archived at perma.cc/H5ES-69KC
- [97] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 9780201633610
- [98] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 9780321125217
- [99] Hongyu Pei Breivold, Ivica Crnkovic, and Peter J. Eriksson. “Analyzing Software Evolvability.” At *32nd Annual IEEE International Computer Software and Applications Conference (COMPSAC)*, July 2008. [doi:10.1109/COMPSAC.2008.50](https://doi.org/10.1109/COMPSAC.2008.50)
- [100] Enrico Zaninotto. “From X Programming to the X Organisation.” At *XP Conference*, May 2002. Archived at perma.cc/R9AR-QCKZ

The Trouble with Distributed Systems

They're funny things, Accidents. You never have them till you're having them.

—A.A. Milne, *The House at Pooh Corner* (1928)

As discussed in “**Reliability and Fault Tolerance**” on page 43, making a system reliable means ensuring that the system as a whole continues working, even when things go wrong (i.e., when there is a fault). However, anticipating all the possible faults and handling them is not that easy. As a developer, it is very tempting to focus mostly on the happy path (after all, most of the time things work fine!) and to neglect faults, since they introduce a lot of edge cases.

If you want your system to be reliable in the presence of faults, you have to radically change your mindset and focus on what could go wrong, even though it may be unlikely. It doesn't matter whether there is only a one-in-a-million chance; in a large enough system, one-in-a-million events happen every day. Experienced systems operators will tell you that anything that *can* go wrong *will* go wrong.

Working with distributed systems is also fundamentally different from writing software on a single computer—the main difference being that things can go wrong in lots of new and exciting ways [1, 2]. In this chapter, you will get a taste of the problems that arise in practice and an understanding of what you can and cannot rely on.

To understand the challenges we are up against, we will turn our pessimism up to the maximum and explore the many types of things that may go wrong in a distributed system, including problems with networks as well as clocks and timing issues. The consequences of all these issues are disorienting, so we'll also explore how to think about the state of a distributed system and how to reason about things that have happened. In Chapter 10, we will look at some examples of how we can achieve fault tolerance in the face of these events.

Faults and Partial Failures

When you are writing a program on a single computer, it normally behaves in a fairly predictable way: either it works or it doesn't. Buggy software may give the appearance that the computer is sometimes “having a bad day” (a problem that is often fixed by a reboot), but that is mostly just a consequence of badly written software.

There is no fundamental reason that software on a single computer should be flaky. When the hardware is working correctly, the same operation always produces the same result (it is *deterministic*). If there is a hardware problem (e.g., memory corruption or a loose connector), the consequence is usually a total system failure (e.g., kernel panic, “blue screen of death,” failure to start up). An individual computer with good software is usually either fully functional or entirely broken, but not something in between.

This is a deliberate choice in the design of computers. If an internal fault occurs, we prefer a computer to crash completely rather than returning a wrong result, because wrong results are difficult and confusing to deal with. Thus, computers hide the fuzzy physical reality on which they are implemented and present an idealized system model that operates with mathematical perfection. A CPU instruction always does the same thing; if you write some data to memory or disk, that data remains intact and doesn't get randomly corrupted. As discussed in “[Hardware and Software Faults](#)” on page 44, this is not actually true—in reality, data does get silently corrupted and CPUs do sometimes silently return the wrong result—but it happens rarely enough that we can get away with ignoring it.

When you are writing software that runs on several computers, connected by a network, the situation is fundamentally different. In distributed systems, faults occur much more frequently, so we can no longer ignore them—we have no choice but to confront the messy reality of the physical world. And in the physical world, a remarkably wide range of things can go wrong, as illustrated by this anecdote [3]:

In my limited experience I've dealt with long-lived network partitions in a single data center (DC), PDU [power distribution unit] failures, switch failures, accidental power cycles of whole racks, whole-DC backbone failures, whole-DC power failures, and a hypoglycemic driver smashing his Ford pickup truck into a DC's HVAC [heating, ventilation, and air conditioning] system. And I'm not even an ops guy.

—Coda Hale

In a distributed system, there may well be some parts of the system that are broken in an unpredictable way, even though other parts of the system are working fine. This is known as a *partial failure*. The difficulty is that partial failures are *nondeterministic*: if you try to do anything involving multiple nodes and the network, it may sometimes work and sometimes unpredictably fail. As we shall see, you may not even *know* whether something succeeded!

This nondeterminism and possibility of partial failures is what makes distributed systems hard to work with [4]. On the other hand, if a distributed system can tolerate partial failures, that opens up powerful possibilities—for example, it means we can perform a rolling upgrade, rebooting one node at a time to install software updates while the system as a whole continues working uninterrupted. Fault tolerance therefore allows us to make distributed systems more reliable than single-node systems; we can build a reliable system from unreliable components.

But before we can implement fault tolerance, we need to know more about the faults that we’re supposed to tolerate. It is important to consider a wide range of possible faults—even fairly unlikely ones—and to artificially create such situations in your testing environment to see what happens. In distributed systems, suspicion, pessimism, and paranoia pay off.

Unreliable Networks

In the past, older computers such as mainframes were made reliable by ensuring that individual components were redundant—for example, by employing RAID to sustain individual disk failures. As discussed in “[Shared-Memory, Shared-Disk, and Shared-Nothing Architectures](#)” on page 51, the distributed systems we focus on in this book are mostly *shared-nothing systems*: a bunch of machines connected by a network. Instead of having redundancy of components within a single machine, shared-nothing systems use replication across separate machines for redundancy. The network is the only way these machines can communicate. We assume that each machine has its own memory and disk, and one machine cannot access another machine’s memory or disk (except by making requests to a service over the network). Even when storage is shared, such as with object storage, machines communicate with shared storage services over the network.

The internet and most internal networks in datacenters (often Ethernet) are *asynchronous packet networks*. In this kind of network, one node can send a message (a packet) to another node, but the network gives no guarantees as to when it will arrive or whether it will arrive at all. If you send a request and expect a response, many things could go wrong (some of which are illustrated in [Figure 9-1](#)):

- Your request may have been lost (perhaps someone unplugged a network cable).
- Your request may be waiting in a queue and will be delivered later (perhaps the network or the recipient is overloaded).
- The remote node may have failed (perhaps it crashed or it was powered down).
- The remote node may have temporarily stopped responding (perhaps it is experiencing a long GC pause; see “[Process Pauses](#)” on page 86), but it will start responding again later.

- The remote node may have processed your request, but the response has been lost on the network (perhaps a network switch has been misconfigured).
- The remote node may have processed your request, but the response has been delayed and will be delivered later (perhaps the network or your own machine is overloaded).

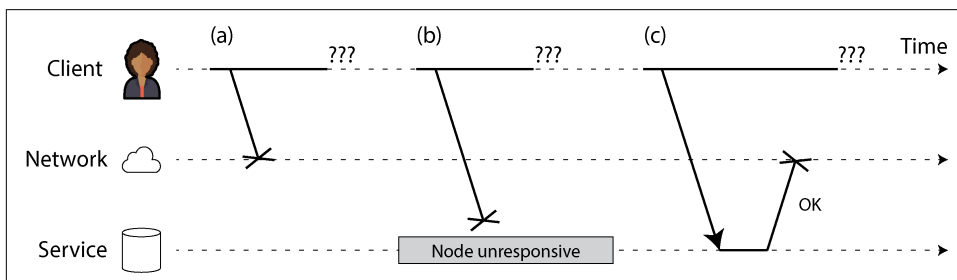


Figure 9-1. If you send a request and don't get a response, it's not possible to distinguish whether (a) the request was lost, (b) the remote node is down, or (c) the response was lost.

The sender can't even tell whether the packet was delivered. The only option is for the recipient to send a response message, which may in turn be lost or delayed. These issues are indistinguishable in an asynchronous network; the only information you have is that you haven't received a response yet. If you send a request to another node and don't receive a response, it is *impossible* to tell why.

The usual way of handling this issue is a *timeout*: after some time, you give up waiting and assume that the response is not going to arrive. However, when a timeout occurs, you still don't know whether the remote node got your request (and if the request is still queued somewhere, it may still be delivered to the recipient, even if you've given up on that possibility).

The Limitations of TCP

Network packets have a maximum size (generally a few kilobytes), but many applications need to send messages (requests, responses) that are too big to fit in one packet. These applications most often use TCP, the Transmission Control Protocol, to establish a *connection* that breaks large data streams into individual packets and puts them back together again on the receiving side.



Most of what we say about TCP applies also to its more recent alternative, QUIC, as well as the Stream Control Transmission Protocol (SCTP) used in WebRTC, the BitTorrent uTP protocol, and other transport protocols. For a comparison to UDP, see “**TCP Versus UDP**” on page 74.

TCP is often described as providing “reliable” delivery, in the sense that it detects and retransmits dropped packets, it detects reordered packets and puts them back in the correct order, and it detects packet corruption by using a simple checksum. It also figures out how fast it can send data so that it is transferred as quickly as possible, but without overloading the network or the receiving node; this is known as *congestion control*, *flow control*, or *backpressure* [5].

When you “send” data by writing it to a socket, it doesn’t get sent immediately; it’s placed in a buffer managed by your operating system. When the congestion control algorithm decides that it has capacity to send a packet, it takes the next packet’s worth of data from that buffer and passes it to the network interface. The packet passes through several switches and routers, and eventually the receiving node’s operating system places the packet’s data in a receive buffer and sends an acknowledgment packet back to the sender. Only then does the receiving operating system notify the application that more data has arrived [6].

So, if TCP provides “reliability,” does that mean we no longer need to worry about networks being unreliable? Unfortunately not. TCP decides that a packet must have been lost if no acknowledgment arrives within a certain timeout, but it can’t tell whether it was the outbound packet or the acknowledgment that was lost. Although it can resend the packet, it can’t guarantee that the new packet will get through (if the network cable is unplugged, for example, TCP can’t plug it back in for you). Eventually, after a configurable timeout, it gives up and signals an error to the application. TCP’s deduplication and retransmission capabilities apply to only a single connection, so if the application reconnects and retransmits, data could be duplicated.

If a TCP connection is closed with an error—perhaps because the remote node crashed or the network was interrupted—you unfortunately have no way of knowing how much data was actually processed by the remote node [6]. Even if you receive an acknowledgment that a packet was delivered, this means only that the operating system kernel on the remote node received it; the application may have crashed before it handled that data. If you want to be sure that a request was successful, you need a positive response from the application itself [7].

Nevertheless, TCP is very useful because it provides a convenient way of sending and receiving messages that are too big to fit in one packet. Once a TCP connection is established, you can also use it to send multiple requests and responses. This is usually done by first sending a header that indicates the length of the following message in bytes, followed by the actual message. HTTP and many RPC protocols (see Chapter 5) work like this.

Network Faults in Practice

We have been building computer networks for decades—one might hope that by now we would have figured out how to make them reliable. Unfortunately, we have not yet succeeded. Some systematic studies and plenty of anecdotal evidence show that network problems are surprisingly common, even in controlled environments like a datacenter operated by one company [8]:

- One study in a medium-sized datacenter found about 12 network faults per month, of which half disconnected a single machine and half disconnected an entire rack [9].
- Another study measured the failure rates of components like top-of-rack switches, aggregation switches, and load balancers [10]. It found that adding redundant networking gear doesn't reduce faults as much as you might expect, since it doesn't guard against human error (e.g., misconfigured switches), which is a major cause of outages.
- Interruptions of wide-area fiber links have been blamed on cows [11], beavers [12], and sharks [13] (though shark bites have become rarer with better shielding of submarine cables [14]). Humans are also often at fault, via accidental misconfiguration [15], scavenging [16], or sabotage [17].
- Across cloud regions, round-trip times of up to several *minutes* have been observed at high percentiles [18, Table 3]. Even within a single datacenter, packet delay of more than a minute can occur during a network topology reconfiguration, triggered by a problem during a software upgrade for a switch [19]. Thus, we have to assume that messages might be delayed arbitrarily.
- Sometimes communications are partially interrupted, depending on who you're talking to—for example, A and B can communicate, and B and C can communicate, but A and C cannot [20, 21]. Other surprising faults include a network interface that sometimes drops all inbound packets but sends outbound packets successfully [22]. Just because a network link works in one direction doesn't guarantee it's also working in the opposite direction.
- Even a brief network interruption can have repercussions that last for much longer than the original issue [8, 20, 23].

Even if network faults are rare in your environment, the fact that faults *can* occur means that your software needs to be able to handle them. Whenever any communication happens over a network, it may fail—there is no way around it.



Network partitions

The term *network partition*, or *netsplit*, is sometimes used when one part of the network is cut off from the rest because of a network fault. However, this is not fundamentally different from other kinds of network interruption. Network partitions are not related to sharding of a storage system, which is sometimes also called *partitioning* (see Chapter 7).

If the error handling of network faults is not defined and tested, arbitrarily bad things could happen—for example, the cluster could become deadlocked and permanently unable to serve requests, even when the network recovers [24], or it could potentially delete all of your data [25]. If software is put in an unanticipated situation, it may do arbitrary unexpected things.

Handling network faults doesn't necessarily mean *tolerating* them. If your network is normally fairly reliable, a valid approach may be to simply show an error message to users while your network is experiencing problems. However, you do need to know how your software reacts to network problems and ensure that the system can recover from them. It may make sense to deliberately trigger network problems and test the system's response (see “[Fault injection](#)” on page 105).

Fault Detection

Many systems need to automatically detect faulty nodes. For example:

- A load balancer needs to stop sending requests to a node that is dead (i.e., take it *out of rotation*).
- In a distributed database with single-leader replication, if the leader fails, one of the followers needs to be promoted to be the new leader (see Chapter 6).

Unfortunately, the uncertainty about the network makes it difficult to tell whether a node is working. In specific circumstances, you might get feedback to explicitly tell you that something is not working:

- If you can reach the machine on which the node should be running, but no process is listening on the destination port (e.g., because the process crashed), the operating system will helpfully close or refuse TCP connections by sending an RST or FIN packet in reply.
- If a node process crashed (or was killed by an administrator) but the node's operating system is still running, a script can notify other nodes about the crash so that another node can take over quickly without having to wait for a timeout to expire. For example, HBase does this [26].

- If you have access to the management interface of the network switches in your datacenter, you can query them to detect link failures at a hardware level (e.g., if the remote machine is powered down). This option is ruled out if you're connecting via the internet, if you're in a shared datacenter with no access to the switches themselves, or if you can't reach the management interface because of a network problem.
- If a router is sure that the IP address you're trying to connect to is unreachable, it may reply to you with an ICMP Destination Unreachable packet. However, the router doesn't have a magic failure detection capability either; it is subject to the same limitations as other participants in the network.

Rapid feedback about a remote node being down is useful, but you can't count on it. If something has gone wrong, you may get an error response at some level of the stack, but in general you have to assume that you will get no response at all. You can retry a few times, wait for a timeout to elapse, and eventually declare the node dead if you don't hear back within the timeout. Since the node could actually be alive, you need to strike a balance between false positives and false negatives: too short a timeout causes alive nodes to be incorrectly suspected to be dead, and too long a timeout causes unnecessary delays waiting for dead nodes.

Timeouts and Unbounded Delays

If a timeout is the only sure way of detecting a fault, then how long should the timeout be? There is unfortunately no simple answer.

A long timeout means a long wait until a node is declared dead (and during this time, users may have to wait or see error messages). A short timeout detects faults faster but carries a higher risk of incorrectly declaring a node dead when in fact it has only suffered a temporary slowdown (e.g., because of a load spike on the node or the network).

Prematurely declaring a node dead is problematic. If the node is actually alive and in the middle of performing an action (e.g., sending an email), and another node takes over, the action may end up being performed twice. We will discuss this issue in more detail in [“Knowledge, Truth, and Lies” on page 91](#) and in Chapters 10 and 12.

When a node is declared dead, its responsibilities need to be transferred to other nodes, which places additional load on those nodes and the network. If the system is already struggling with high load, declaring nodes dead prematurely can make the problem worse. In particular, it could happen that the node actually wasn't dead but only slow to respond because of overload; transferring its load to other nodes can cause a cascading failure (in the extreme case, all nodes declare each other dead, and everything stops working—see [“When an Overloaded System Won't Recover” on page 38](#)).

Imagine a fictitious system with a network that guarantees a maximum delay for packets. Every packet is either delivered within time d or lost, and delivery never takes longer than d . Furthermore, assume that you can guarantee that a non-failed node always handles a request within time r . In this case, you could guarantee that every successful request receives a response within time $2d + r$ —and if you don't receive a response within that time, you know that either the network or the remote node is not working. If this was true, $2d + r$ would be a reasonable timeout to use.

Unfortunately, most systems we work with have neither of those guarantees. Asynchronous networks have *unbounded delays* (they try to deliver packets as quickly as possible, but there is no upper limit on the time it may take for a packet to arrive), and most server implementations cannot guarantee that they can handle requests within a maximum time (see “[Providing response time guarantees](#)” on page 89). For failure detection, it's not sufficient for the system to be fast most of the time: if your timeout is low, it takes only a transient spike in round-trip times to throw the system off balance.

Network congestion and queueing

When driving a car, travel times on road networks often vary most because of traffic congestion. Similarly, the variability of packet delays on computer networks is most often due to queueing [27]:

- If several nodes simultaneously try to send packets to the same destination, the network switch must queue them up and feed them into the destination network link one by one (as illustrated in [Figure 9-2](#)). On a busy network link, a packet may have to wait a while until it can get a slot (this is called *network congestion*). If there is so much incoming data that the switch queue fills up, the packet is dropped, so it needs to be resent—even though the network is functioning fine.
- When a packet reaches the destination machine, if all CPU cores or application threads are currently busy, the incoming request from the network is queued by the operating system until the application is ready to handle it. Depending on the load on the machine, this may take an arbitrary length of time [28].
- In virtualized environments, a running operating system is often paused for tens of milliseconds while another virtual machine (VM) uses a CPU core. During this time, the VM cannot consume any data from the network, so the incoming data is queued (buffered) by the VM monitor [29], further increasing the variability of network delays.
- As mentioned earlier, to avoid overloading the network, TCP limits the rate at which it sends data. This means additional queueing at the sender before the data even enters the network.

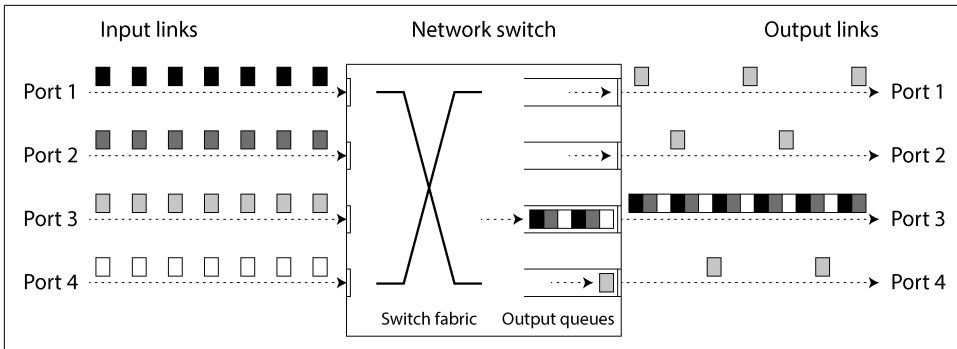


Figure 9-2. If several machines send network traffic to the same destination, its switch queue can fill up. Here, ports 1, 2, and 4 are all trying to send packets to port 3.

Moreover, when TCP detects and automatically retransmits a lost packet, the application does not see the packet loss directly but does see the resulting delay (waiting for the timeout to expire, and then waiting for the retransmitted packet to be acknowledged).

TCP Versus UDP

Some latency-sensitive applications, such as videoconferencing and Voice over IP (VoIP), use UDP rather than TCP. This choice is a trade-off between reliability and variability of delays: as UDP does not perform flow control and does not retransmit lost packets, it avoids some of the reasons for variable network delays (although it is still susceptible to switch queues and scheduling delays).

UDP is a good choice when delayed data is worthless. For example, in a VoIP phone call, there probably isn't enough time to retransmit a lost packet before its data is due to be played over the speaker. In this case, there's no point in retransmitting the packet—the application must instead fill the missing packet's time slot with silence (causing a brief interruption in the sound) and move on in the stream. The retry happens at the human layer instead. (“Could you repeat that please? The sound just cut out for a moment.”)

Variability of network delays

All these factors contribute to the variability of network delays. Queueing delays have an especially wide range when a system is close to its maximum capacity. A system with plenty of spare capacity can easily drain queues, whereas in a highly utilized system, long queues can build up very quickly.

In public clouds and multitenant datacenters, resources are shared among many customers. The network links and switches, and even each machine's network interface

and CPUs (when running on VMs), are shared. Processing large amounts of data can use the entire capacity of network links (*saturate* them). Because you have no control over or insight into other customers' usage of the shared resources, network delays can be highly variable if someone near you (a *noisy neighbor*) is using a lot of resources [30, 31].

In such environments, you can choose timeouts only experimentally: measure the distribution of network round-trip times over an extended period, and over many machines, to determine the expected variability of delays. Then, taking into account your application's characteristics, you can determine an appropriate trade-off between failure detection delay and risk of premature timeouts.

Even better, rather than using configured constant timeouts, systems can continually measure response times and their variability (*jitter*) and automatically adjust timeouts according to the observed response time distribution. The Phi Accrual failure detector [32] (which is used, for example, in Akka and Cassandra [33]), is one way of doing this. TCP retransmission timeouts work similarly [5].

Synchronous Versus Asynchronous Networks

Distributed systems would be a lot simpler if we could rely on the network to deliver packets with a fixed maximum delay and to not drop packets. Why can't we solve this at the hardware level and make the network reliable so that the software doesn't need to worry about it?

To answer this question, it's interesting to compare datacenter networks to the traditional fixed-line telephone network (non-cellular, non-VoIP), which is extremely reliable; delayed audio frames and dropped calls are very rare. A phone call requires a constantly low end-to-end latency and enough bandwidth to transfer the audio samples of your voice. Wouldn't it be nice to have similar reliability and predictability in computer networks?

When you make a call over the telephone network, it establishes a *circuit*: a fixed, guaranteed amount of bandwidth is allocated for the call, along the entire route between the two callers. This circuit remains in place until the call ends [34]. For example, an ISDN network runs at a fixed rate of 4,000 frames per second. When a call is established, it is allocated 16 bits of space within each frame (in each direction). Thus, for the duration of the call, each side is guaranteed to be able to send exactly 16 bits of audio data every 250 microseconds [35].

This kind of network is *synchronous*: even as data passes through several routers, it does not suffer from queueing, because the 16 bits of space for the call have already been reserved in the next hop of the network. And because there is no queueing, the maximum end-to-end latency of the network is fixed. We call this a *bounded delay*.

Can we not simply make network delays predictable?

Note that a circuit in a telephone network is very different from a TCP connection. A circuit has a fixed amount of reserved bandwidth that nobody else can use as long as it is established, whereas the packets of a TCP connection opportunistically use whatever network bandwidth is available. You can give TCP a variable-sized block of data (e.g., an email or a web page), and it will try to transfer it in the shortest time possible. While a TCP connection is idle, it doesn't use any bandwidth (except perhaps for an occasional keepalive packet).

If datacenter networks and the internet were circuit-switched networks, it would be possible to establish a guaranteed maximum round-trip time when a circuit was set up. However, they are not. Ethernet and IP are packet-switched protocols, which suffer from queueing and thus unbounded delays in the network. These protocols do not have the concept of a circuit.

Why do datacenter networks and the internet use packet switching? The answer is that they are optimized for *bursty traffic*. A circuit is good for an audio or video call, which needs to transfer a fairly constant number of bits per second for the duration of the call. On the other hand, requesting a web page, sending an email, or transferring a file doesn't have any particular bandwidth requirement—we just want it to complete as quickly as possible.

If you wanted to transfer a file over a circuit, you would have to guess a bandwidth allocation. If you guess too low, the transfer will be unnecessarily slow, leaving network capacity unused. If you guess too high, the circuit cannot be set up (because the network cannot allow a circuit to be created if its bandwidth allocation cannot be guaranteed). By contrast, TCP dynamically adapts the rate of data transfer to the available network capacity.

Latency and Resource Utilization

More generally, you can think of variable delays as a consequence of dynamic resource partitioning.

Say you have a wire (or fiber) between two telephone switches that can carry up to 10,000 simultaneous calls. Each circuit that is switched over this wire occupies one of those call slots. Thus, you can think of the wire as a resource that can be shared by up to 10,000 simultaneous users. The resource is divided in a *static* way: even if you're the only call on the wire right now, and all other 9,999 slots are unused, your circuit is still allocated the same fixed amount of bandwidth as when the wire is fully utilized.

By contrast, the internet shares network bandwidth *dynamically*. Senders push and jostle with one another to get their packets over the wire as quickly as possible, and the network switches decide which packet to send (i.e., the bandwidth allocation) from one moment to the next. This approach has the downside of queueing, but the

advantage is that it maximizes utilization of the wire. The wire has a fixed cost, so if you utilize it better, each byte you send over it is cheaper.

A similar situation arises with CPUs. If you share each CPU core dynamically among several threads, one thread sometimes has to wait in the operating system's run queue while another thread is running, so a thread can be paused for varying lengths of time [36]. However, this utilizes the hardware better than if you allocated a static number of CPU cycles to each thread (see [“Providing response time guarantees” on page 89](#)). Better hardware utilization is also why cloud platforms run several VMs from different customers on the same physical machine.

Latency guarantees are achievable in certain environments, if resources are statically partitioned (e.g., dedicated hardware and exclusive bandwidth allocations). However, these guarantees come at the cost of reduced utilization—in other words, it is more expensive. On the other hand, multitenancy with dynamic resource partitioning provides better utilization, so it is cheaper, but it has the downside of variable delays.

Variable delays in networks are not a law of nature but simply the result of a cost/benefit trade-off.

Combining circuit switching and packet switching

There have been some attempts to build hybrid networks that support both circuit switching and packet switching. *Asynchronous Transfer Mode* (ATM) was a competitor to Ethernet in the 1980s, but it didn't gain much adoption outside of telephone network core switches. InfiniBand has some similarities [37]: it implements end-to-end flow control at the link layer, which reduces the need for queueing in the network, although it can still suffer from delays due to link congestion [38].

With careful use of *quality of service* (QoS) mechanisms such as prioritization and scheduling of packets and *admission control* (rate-limiting senders), it is possible to emulate circuit switching on packet networks, or to provide statistically bounded delay [27, 34]. New network algorithms like *Low Latency, Low Loss, and Scalable Throughput* (L4S) attempt to mitigate some of the queuing and congestion control problems both at the client and the router level. Linux's traffic controller (TC) also allows applications to reprioritize packets for QoS purposes.

Such QoS mechanisms, however, are not currently enabled in multitenant datacenters and public clouds, or when communicating via the internet. Currently deployed technology does not allow us to make any guarantees about delays or reliability of the network; we have to assume that network congestion, queueing, and unbounded delays will happen. Consequently, there's no “correct” value for timeouts—they need to be determined experimentally.

Peering agreements between internet service providers and the establishment of routes through the Border Gateway Protocol (BGP) bear a closer resemblance to

circuit switching than typical IP packet routing does. At this level, it is possible to buy dedicated bandwidth. That said, internet routing operates at the level of networks, not individual connections between hosts, and on a much longer timescale.

Unreliable Clocks

Clocks and time are important. Applications depend on clocks in various ways, to answer questions like the following:

1. Has this request timed out yet?
2. What's the 99th percentile response time of this service?
3. How many queries per second did this service handle on average in the last five minutes?
4. How long did the user spend on our site?
5. When was this article published?
6. At what date and time should the reminder email be sent?
7. When does this cache entry expire?
8. What is the timestamp on this error message in the log file?

Questions 1–4 measure *durations* (e.g., the time interval between a request being sent and a response being received), whereas questions 5–8 describe *points in time* (events that occur on a particular date, at a particular time).

In a distributed system, time is a tricky business, because communication is not instantaneous; it takes time for a message to travel across the network from one machine to another. The time when a message is received is always later than the time when it is sent, but because of variable delays in the network, we don't know how much later. This fact sometimes makes it difficult to determine the order in which things happened when multiple machines are involved.

Moreover, each machine on the network has its own clock, which is a hardware device—usually a quartz crystal oscillator. These devices are not perfectly accurate, so each machine has its own notion of time, which may be slightly faster or slower than on other machines. It is possible to synchronize clocks to some degree; the most commonly used mechanism is the Network Time Protocol (NTP), which allows the computer clock to be adjusted according to the time reported by a group of servers [39]. The servers in turn get their time from a more accurate time source, such as a GPS receiver.

Monotonic Versus Time-of-Day Clocks

Modern computers have at least two kinds of clocks: a time-of-day clock and a monotonic clock. Although both measure time, it is important to distinguish between them, since they serve different purposes.

Time-of-day clocks

A *time-of-day clock* does what you intuitively expect of a clock: it returns the current date and time according to a calendar (also known as *wall-clock time*). For example, `clock_gettime(CLOCK_REALTIME)` on Linux and `System.currentTimeMillis` in Java return the number of seconds (or milliseconds) since the *epoch*, defined as midnight UTC on January 1, 1970, according to the Gregorian calendar, not counting leap seconds. Some systems use other dates as their reference point. (Although the Linux clock is called *real-time*, it has nothing to do with real-time operating systems, as discussed in “[Providing response time guarantees](#)” on page 89.)

Time-of-day clocks are usually synchronized with NTP, which means that a time-stamp from one machine (ideally) means the same as a timestamp on another machine. However, time-of-day clocks also have various oddities, as described in the next section. In particular, if the local clock is too far ahead of the NTP server, it may be forcibly reset and appear to jump back to a previous point in time. These jumps, as well as similar jumps caused by leap seconds, make time-of-day clocks unsuitable for measuring elapsed time [40].

Time-of-day clocks can also experience jumps due to the start and end of Daylight Saving Time (DST), although these can be avoided by always using UTC as the time zone, which does not have DST. Historically, these clocks had quite a coarse-grained resolution (e.g., moving forward in steps of 10 ms on older Windows systems [41]). On recent systems, this is less of a problem.

Monotonic clocks

A *monotonic clock* is suitable for measuring a duration (time interval), such as a time-out or a service’s response time; `clock_gettime(CLOCK_MONOTONIC)` or `clock_gettime(CLOCK_BOOTTIME)` on Linux [42] and `System.nanoTime` in Java, for example, measure time using a monotonic clock. The name comes from the fact that this type of clock is guaranteed to always move forward (whereas a time-of-day clock may jump back in time).

You can check the value of a monotonic clock at one point in time, do something, and then check the clock again at a later time. The *difference* between the two values tells you how much time elapsed between the two checks—more like a stopwatch than a wall clock. However, the *absolute* value of the clock is meaningless; it might be the number of nanoseconds since the computer was booted up, or something similarly

arbitrary. In particular, it makes no sense to compare monotonic clock values from two computers, because they don't mean the same thing.

On a server with multiple CPU sockets, there may be a separate timer per CPU, which is not necessarily synchronized with other CPUs [43]. Operating systems compensate for any discrepancy and try to present a monotonic view of the clock to application threads, even as they are scheduled across multiple CPUs. However, it is wise to take this guarantee of monotonicity with a pinch of salt [44].

NTP may adjust the frequency at which the monotonic clock moves forward (this is known as *slewing* the clock) if it detects that the computer's local quartz is moving faster or slower than the NTP server. By default, NTP allows the clock rate to be sped up or slowed down by up to 0.05%, but it cannot cause the monotonic clock to jump forward or backward. The resolution of monotonic clocks is usually quite good: on most systems they can measure time intervals in microseconds or less.

In a distributed system, using a monotonic clock for measuring elapsed time (e.g., timeouts) is usually fine, because it doesn't assume any synchronization between different nodes' clocks and is not sensitive to slight inaccuracies of measurement.

Clock Synchronization and Accuracy

Monotonic clocks don't require synchronization, but time-of-day clocks need to be set according to an NTP server or other external time source in order to be useful. Unfortunately, our methods for getting a clock to tell the correct time aren't nearly as reliable or accurate as you might hope—hardware clocks and NTP can be fickle beasts. To give just a few examples:

- The quartz clock in a typical computer is not very accurate: it *drifts* (runs faster or slower than it should). Clock drift varies depending on the temperature of the machine. Google assumes a clock drift of up to 200 ppm (parts per million) for its servers [45], which is equivalent to a 6 ms drift for a clock that is resynchronized with a server every 30 seconds, or a 17-second drift for a clock that is resynchronized once a day. This drift limits the best possible accuracy you can achieve, even if everything is working correctly.
- If a computer's clock differs too much from an NTP server, it may refuse to synchronize or be forcibly reset [39]. Any applications observing the time before and after this reset may see time go backward or suddenly jump forward.
- If a node is accidentally firewalled off from NTP servers, the misconfiguration may go unnoticed for some time, during which the drift may add up to large discrepancies between that and other nodes' clocks. Anecdotal evidence suggests that this does happen in practice.

- NTP synchronization can be only as good as the network delay, so its accuracy is limited when you're on a congested network with variable packet delays. One experiment showed that a minimum error of 35 ms is achievable when synchronizing over the internet [46], though occasional spikes in network delay can lead to errors of around a second. Depending on the configuration, large network delays can cause the NTP client to give up entirely.
- Some NTP servers are wrong or misconfigured, reporting time that is off by hours [47, 48]. NTP clients mitigate such errors by querying several servers and ignoring outliers. Nevertheless, it's somewhat worrying to bet the correctness of your systems on the time that you were told by a stranger on the internet.
- Leap seconds result in a minute that is 59 seconds or 61 seconds long, which messes up timing assumptions in systems that are not designed with leap seconds in mind [49]. The fact that leap seconds have crashed many large systems [40, 50] shows how easy it is for incorrect assumptions about clocks to sneak into a system. The best way of handling leap seconds may be to make NTP servers "lie," by performing the leap second adjustment gradually over the course of a day (this is known as *smearing*) [51, 52], although actual NTP server behavior varies in practice [53]. Leap seconds will no longer be used from 2035 on, so this problem will fortunately go away.
- In VMs, the hardware clock is virtualized, which raises additional challenges for applications that need accurate timekeeping [54]. When a CPU core is shared among VMs, each VM is paused for tens of milliseconds while another VM is running. From an application's point of view, this pause manifests itself as the clock suddenly jumping forward when the VM continues running [29]. An NTP client running inside the VM does not know when a pause occurs, so it may report the clock accuracy incorrectly [55].
- If you run software on devices that you don't fully control (e.g., mobile or embedded devices), you probably cannot trust their hardware clocks at all. Some users deliberately set their device's hardware clock to an incorrect date and time, for example, to cheat in games [56]. As a result, the clock might be set to a wildly inaccurate time.

Achieving very good clock accuracy is possible if you care about it sufficiently to invest significant resources. For example, the MiFID II European regulation for financial institutions requires all high-frequency trading funds to synchronize their clocks to within 100 microseconds of UTC, in order to help debug market anomalies such as "flash crashes" and to help detect market manipulation [57].

Such accuracy can be achieved with special hardware (GPS receivers and/or atomic clocks), the Precision Time Protocol (PTP), and careful deployment and monitoring [58, 59]. Relying on GPS alone can be risky because GPS signals can easily be jammed. In some locations (e.g., close to military facilities), this happens frequently

[60]. Some cloud providers have begun offering high-accuracy clock synchronization for their virtual machines [61], but clock synchronization still requires a lot of care. If your NTP daemon is misconfigured or a firewall is blocking NTP traffic, the clock error due to drift can quickly become large.

Relying on Synchronized Clocks

The problem with clocks is that while they seem simple and easy to use, they have a surprising number of pitfalls. A day may not have exactly 86,400 seconds, time-of-day clocks may move backward in time, and the time according to one node's clock may be quite different from another node's clock.

Earlier in this chapter we discussed networks dropping and arbitrarily delaying packets. Even though networks are well behaved most of the time, software must be designed on the assumption that the network will occasionally be faulty, and the software must handle such faults gracefully. The same is true with clocks: although they work quite well most of the time, robust software needs to be prepared to deal with incorrect clocks.

Part of the problem is that incorrect clocks easily go unnoticed. If a machine's CPU is defective or its network is misconfigured, it most likely won't work at all, so the issue will quickly be spotted and fixed. On the other hand, if its quartz clock is defective or its NTP client is misconfigured, most things will seem to work fine, even though its clock gradually drifts further and further away from reality. If a piece of software is relying on an accurately synchronized clock, the result is more likely to be silent and subtle data loss than a dramatic crash [62, 63].

Thus, if you use software that requires synchronized clocks, it is essential that you carefully monitor the clock offsets between all the machines in your cluster. Any node whose clock drifts too far from the others should be declared dead and removed. Such monitoring ensures that you notice the faulty clocks before they can cause too much damage.

Timestamps for ordering events

Let's consider one particular situation in which it is tempting, but dangerous, to rely on clocks: ordering of events across multiple nodes [64]. For example, if two clients write to a distributed database, who got there first? Which write is the more recent one?

Figure 9-3 illustrates a dangerous use of time-of-day clocks in a database with multi-leader replication (the example is similar to Figure 6-8). Client A writes $x = 1$ on node 1; the write is replicated to node 3; client B increments x on node 3 (we now have $x = 2$); and finally, both writes are replicated to node 2. As this figure shows, when a write is replicated to other nodes, it is tagged with a timestamp according to the time-of-day clock on the node where the write originated. The clock synchronization

is very good in this example; the skew between node 1 and node 3 is less than 3 ms, which is probably better than you can expect in practice.

Since the increment builds upon the earlier write of $x = 1$, we might expect that the write of $x = 2$ should have the greater timestamp of the two. Unfortunately, that is not what happens in **Figure 9-3**. The write $x = 1$ has a timestamp of 42.004 seconds, but the write $x = 2$ has a timestamp of 42.003 seconds. In other words, the write by client B is causally later than the write by client A, but B's write has an earlier timestamp.

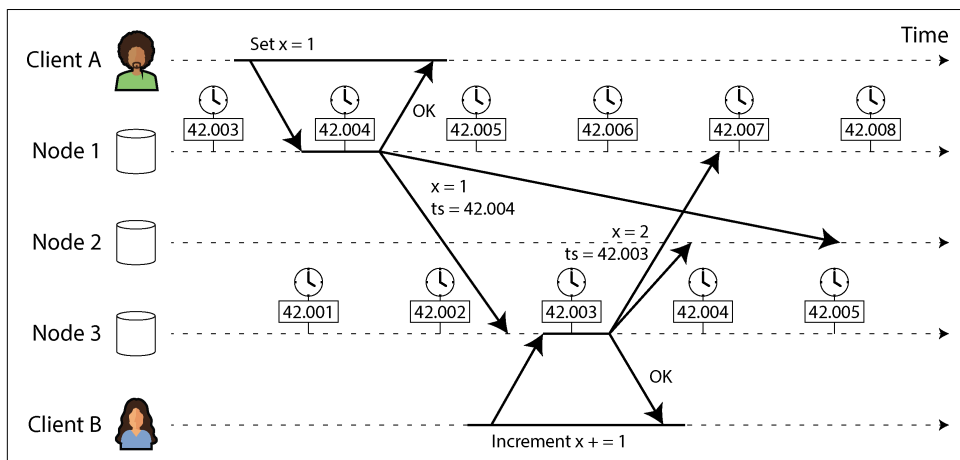


Figure 9-3. Relying on timestamps for ordering events can cause problems when time-of-day clocks are not perfectly synchronized.

As discussed in Chapter 6, one way of resolving conflicts between concurrently written values on different nodes is *last write wins* (LWW), which means keeping the write with the greatest timestamp for a given key and discarding all writes with older timestamps. In **Figure 9-3**, when node 2 receives these two events, it will incorrectly conclude that $x = 1$ is the more recent value and drop the write $x = 2$, so the increment is lost.

This problem can be prevented by ensuring that when a value is overwritten, the new value always has a higher timestamp than the overwritten value, even if that timestamp is ahead of the writer's local clock. However, that incurs the cost of an additional read to find the greatest existing timestamp. Some systems, including Cassandra and ScyllaDB, are designed to avoid this additional round trip and simply use the client clock's timestamp along with a LWW policy [62]. This approach has some serious problems:

- Database writes can mysteriously disappear. A node with a lagging clock is unable to overwrite values previously written by a node with a faster clock until the clock skew between the nodes has elapsed [63, 65]. This scenario can

cause arbitrary amounts of data to be silently dropped without any error being reported to the application.

- LWW cannot distinguish between writes that occurred sequentially in quick succession (in [Figure 9-3](#), client B's increment definitely occurs *after* client A's write) and writes that were truly concurrent (neither writer was aware of the other). Additional causality tracking mechanisms, such as version vectors, are needed in order to prevent violations of causality (see Chapter 6).
- Two nodes could independently generate writes with the same timestamp, especially when the clock has only millisecond resolution. An additional tiebreaker value (which can simply be a large random number) is required to resolve such conflicts, but this approach can also lead to violations of causality [62].

Thus, even though it is tempting to resolve conflicts by keeping the most “recent” value and discarding others, it is important to be aware that the definition of “recent” depends on a local time-of-day clock, which may well be incorrect. Even with tightly NTP-synchronized clocks, you could send a packet at timestamp 100 ms (according to the sender's clock) and have it arrive at timestamp 99 ms (according to the recipient's clock)—so it appears as though the packet arrived before it was sent, which is impossible.

Could NTP synchronization be made accurate enough that such incorrect orderings cannot occur? Probably not, because NTP's synchronization accuracy is itself limited by the network round-trip time, in addition to other sources of error such as quartz drift. To guarantee a correct ordering, you would need the clock error to be significantly lower than the network delay, which is not possible.

So-called *logical clocks* [66], which are based on incrementing counters rather than an oscillating quartz crystal, are a safer alternative for ordering events (see Chapter 6). Logical clocks do not measure the time of day or the number of seconds elapsed, only the relative ordering of events (whether one event happened before or after another). In contrast, time-of-day and monotonic clocks, which measure actual elapsed time, are known as *physical clocks*. We'll look at logical clocks in more detail in Chapter 10.

Clock readings with a confidence interval

You may be able to read a machine's time-of-day clock with microsecond or even nanosecond resolution. But even if you can get such a fine-grained measurement, that doesn't mean the value is actually accurate to such precision. In fact, it most likely is not. As mentioned previously, the drift in an imprecise quartz clock can easily be several milliseconds, even if you synchronize with an NTP server on the local network every minute. With an NTP server on the public internet, the best possible accuracy is probably to the tens of milliseconds, and the error may easily spike to over 100 ms with network congestion.

Thus, it doesn't make sense to think of a clock reading as a point in time. It is more like a range of times, within a confidence interval—for example, a system may be 95% confident that the time now is between 10.3 and 10.5 seconds past the minute, but it doesn't know any more precisely than that [67]. If we know only the time ± 100 ms, the microsecond digits in the timestamp are essentially meaningless.

The uncertainty bound can be calculated based on your time source. If you have a GPS receiver or atomic clock directly attached to your computer, the expected error range is determined by the device and, in the case of GPS, by the quality of the signal from the satellites. If you're getting the time from a server, the uncertainty is based on the expected quartz drift since your last sync with the server, plus the NTP server's uncertainty, plus the network round-trip time to the server (to a first approximation, and assuming you trust the server).

Unfortunately, most systems don't expose this uncertainty; for example, when you call `clock_gettime`, the return value doesn't tell you the expected error of the timestamp, so you don't know whether its confidence interval is five milliseconds or five years.

There are exceptions. The *TrueTime* API in Google Spanner [45] and Amazon ClockBound explicitly report the confidence interval on the local clock. When you ask it for the current time, you get back two values: [*earliest*, *latest*], which are the *earliest possible* and the *latest possible* timestamp. Based on its uncertainty calculations, the clock knows that the actual current time is somewhere within that interval. The width of the interval depends, among other factors, on how long it has been since the local quartz clock was last synchronized with a more accurate clock source.

Synchronized clocks for global snapshots

In Chapter 8 we discussed *multiversion concurrency control* (MVCC), which is a very useful feature in databases that need to support both small, fast read/write transactions and large, long-running read-only transactions (e.g., for backups or analytics). It allows read-only transactions to see a *snapshot* of the database, a consistent state at a particular point in time, without locking and interfering with read/write transactions.

Generally, MVCC requires a monotonically increasing transaction ID. If a write happened later than the snapshot (i.e., the write has a greater transaction ID than the snapshot), that write is invisible to the snapshot transaction. On a single-node database, a simple counter is sufficient for generating transaction IDs.

However, when a database is distributed across many machines, potentially in multiple datacenters, a global, monotonically increasing transaction ID (across all shards) is difficult to generate, because it requires coordination. The transaction ID must reflect causality: if transaction B reads or overwrites a value that was previously writ-

ten by transaction A, then B must have a higher transaction ID than A—otherwise, the snapshot would not be consistent. With lots of small, rapid transactions, creating transaction IDs in a distributed system becomes an untenable bottleneck. (We will discuss such ID generators in Chapter 10.)

Can we use the timestamps from synchronized time-of-day clocks as transaction IDs? If we could get the synchronization good enough, they would have the right properties because later transactions have a higher timestamp. The problem is the uncertainty about clock accuracy.

Spanner implements snapshot isolation across datacenters in this way [68, 69]. It uses the clock's confidence interval as reported by the TrueTime API and is based on the following observation: if you have two confidence intervals, each consisting of an earliest and latest possible timestamp ($A = [A_{\text{earliest}}, A_{\text{latest}}]$ and $B = [B_{\text{earliest}}, B_{\text{latest}}]$), and those two intervals do not overlap (i.e., $A_{\text{earliest}} < A_{\text{latest}} < B_{\text{earliest}} < B_{\text{latest}}$), then B definitely happened after A—there can be no doubt. Only if the intervals overlap are we unsure in which order A and B happened.

To ensure that transaction timestamps reflect causality, Spanner deliberately waits for the length of the confidence interval before committing a read/write transaction. By doing so, it ensures that any transaction that may read the data is at a sufficiently later time that their confidence intervals do not overlap. To keep the wait time as short as possible, Spanner needs to keep the clock uncertainty as small as possible; for this purpose, Google deploys a GPS receiver or atomic clock in each datacenter, allowing clocks to be synchronized to within about 7 ms [45].

The atomic clocks and GPS receivers are not strictly necessary in Spanner. The important thing is to have a confidence interval—accurate clock sources only help keep that interval small. Other systems are beginning to adopt similar approaches—for example, YugabyteDB can leverage ClockBound when running on AWS [70], and several other systems now also rely on clock synchronization to various degrees [71, 72].

Process Pauses

Let's consider another example of dangerous clock use in a distributed system. Say you have a database with a single leader per shard. Only the leader is allowed to accept writes. How does a node know that it is still leader (that it hasn't been declared dead by the others) and that it may safely accept writes?

One option is for the leader to obtain a *lease* from the other nodes, which is similar to a lock with a timeout [73]. Only one node can hold the lease at any one time. Thus, when a node obtains a lease, it knows that it is the leader for a certain amount of time, until the lease expires. To remain leader, the node must periodically renew the lease before it expires. If the node fails, it stops renewing the lease, so another node can take over when it expires.

You can imagine the request-handling loop looking something like this:

```
while (true) {
    request = getIncomingRequest();

    // Ensure that the lease always has at least 10 seconds remaining
    if (lease.expiryTimeMillis - System.currentTimeMillis() < 10000) {
        lease = lease.renew();
    }

    if (lease.isValid()) {
        process(request);
    }
}
```

What's wrong with this code? First, it's relying on synchronized clocks: the expiry time on the lease is set by a different machine (where the expiry may be calculated as the current time plus 30 seconds, for example), and it's being compared to the local system clock. If the clocks are out of sync by more than a few seconds, the code will start doing strange things.

Second, even if we change the protocol to use only the local monotonic clock, there is another problem: the code assumes that very little time passes between the point that it checks the time (`System.currentTimeMillis()`) and the time when the request is processed (`process(request)`). Normally this code runs very quickly, so the 10-second buffer is more than enough to ensure that the lease doesn't expire in the middle of processing a request.

However, what if an unexpected pause occurs in the execution of the program? For example, imagine the thread stops for 15 seconds around the line that calls `lease.isValid` before finally continuing. In that case, the lease will likely have expired by the time the request is processed, and another node will already have taken over as leader. However, there is nothing to tell this thread that it was paused for so long, so the code won't notice that the lease has expired until the next iteration of the loop—by which time it may have already done something unsafe by processing the request.

Is it reasonable to assume that a thread might be paused for so long? Unfortunately, yes. There are various reasons this could happen:

- Contention among threads accessing a shared resource, such as a lock or queue, can cause threads to spend a lot of their time waiting. Such problems are often worse on machines with more CPU cores, and contention problems can be difficult to diagnose [74].
- Many programming language runtimes (such as the JVM) have a garbage collector that occasionally needs to stop all running threads. In the past, such “*stop-the-world*” *garbage collection pauses* would sometimes last for several minutes [75]!

With modern GC algorithms this is less of a problem, but GC pauses can still be noticeable (see “[Limiting the impact of garbage collection](#)” on page 90).

- In virtualized environments, a VM can be *suspended* (pausing the execution of all processes and saving the contents of memory to disk) and *resumed* (restoring the contents of memory and continuing execution). This pause can occur at any time in a process’s execution and can last for an arbitrary length of time. This feature is sometimes used for *live migration* of VMs from one host to another without a reboot, in which case the length of the pause depends on the rate at which processes are writing to memory [76].
- On end-user devices such as laptops and phones, execution may also be suspended and resumed arbitrarily (e.g., when the user closes the lid of their laptop).
- When the operating system context-switches to another thread, or when the hypervisor switches to a different VM (when running in a VM), the currently running thread can be paused at any arbitrary point in the code. In the case of a VM, the CPU time spent in other VMs is known as *steal time*. If the machine is under heavy load—that is, if a long queue of threads is waiting to run—it may take some time before the paused thread gets to run again.
- If the application performs synchronous disk access, a thread may be paused waiting for a slow disk I/O operation to complete [77]. In many languages, disk access can happen surprisingly, even if the code doesn’t explicitly mention file access—for example, the Java classloader lazily loads class files when they are first used, which could happen at any time in the program execution. I/O pauses and GC pauses may even conspire to combine their delays [78]. If the disk is actually a network filesystem or network block device (such as Amazon EBS), the I/O latency is further subject to the variability of network delays [31].
- If the operating system is configured to allow *swapping to disk* (*paging*), a simple memory access may result in a page fault that requires a page from disk to be loaded into memory. The thread is paused while this slow I/O operation takes place. If memory pressure is high, this may in turn require a different page to be swapped out to disk. In extreme circumstances, the operating system may spend most of its time swapping pages in and out of memory and get little actual work done (this is known as *thrashing*). To avoid this problem, paging is often disabled on server machines (if you would rather kill a process to free up memory than risk thrashing).

- A Unix process can be paused by sending it the SIGSTOP signal—for example, by pressing Ctrl-Z in a shell. This signal immediately stops the process from getting any more CPU cycles until it is resumed with SIGCONT, at which point it continues running where it left off. Even if your environment does not normally use SIGSTOP, it might be sent accidentally by an operations engineer.

All these occurrences can *preempt* the running thread at any point and resume it at a later time, without the thread even noticing. The problem is similar to making multithreaded code on a single machine thread-safe; you can't assume anything about timing, because arbitrary context switches and parallelism may occur.

When writing multithreaded code on a single machine, we have fairly good tools for making it thread-safe: mutexes, semaphores, atomic counters, lock-free data structures, blocking queues, and so on. Unfortunately, these tools don't directly translate to distributed systems, because a distributed system has no shared memory—only messages sent over an unreliable network.

A node in a distributed system must assume that its execution can be paused for a significant length of time at any point, even in the middle of a function. During the pause, the rest of the world keeps moving and may even declare the paused node dead because it's not responding. Eventually, the paused node may continue running, without even noticing that it was asleep until it checks its clock sometime later.

Providing response time guarantees

As we've just discussed, in many programming languages and operating systems, threads and processes may pause for an unbounded amount of time. However, those reasons for pausing *can* be eliminated if you try hard enough.

Some software runs in environments where a failure to respond within a specified time can cause serious damage. Computers that control the movements of aircraft, rockets, robots, cars, and other physical objects, for example, must respond quickly and predictably to their sensor inputs. In these so-called hard *real-time* systems, the software must respond by a specified deadline; failure to meet the deadline may cause a failure of the entire system.



In embedded systems, *real-time* means that a system is carefully designed and tested to meet specified timing guarantees in all circumstances. This meaning is in contrast to the more vague use of the term *real-time* on the web, where it describes servers pushing data to clients and stream processing without hard response-time constraints (see Chapter 12).

For example, if your car’s onboard sensors detect that you are currently experiencing a crash, you wouldn’t want the release of the airbag to be delayed because of an inopportune GC pause in the airbag release system.

Providing real-time guarantees in a system requires support from all levels of the software stack. A *real-time operating system* (RTOS) that allows processes to be scheduled with a guaranteed allocation of CPU time in specified intervals is needed; library functions must document their worst-case execution times; dynamic memory allocation may be restricted or disallowed entirely (real-time garbage collectors exist, but the application must still ensure that it doesn’t give the garbage collector too much work to do); and an enormous amount of testing and measurement must be done to ensure that guarantees are being met.

All of this requires a large amount of additional work and severely restricts the range of programming languages, libraries, and tools that can be used (since most languages and tools do not provide real-time guarantees). For these reasons, developing real-time systems is very expensive, and they are most commonly used in safety-critical embedded devices. Also, “real-time” is not the same as “high-performance”—in fact, real-time systems may have lower throughput, since they have to prioritize timely responses above all else (see “[Latency and Resource Utilization](#)” on page 76).

For most server-side data processing systems, real-time guarantees are simply not economical or appropriate. Consequently, these systems must suffer the pauses and clock instability that come from operating in a non-real-time environment.

Limiting the impact of garbage collection

Garbage collection used to be one of the biggest reasons for process pauses [79], but fortunately GC algorithms have improved a lot. A properly tuned collector will now usually pause processes for no more than a few milliseconds. The Java runtime offers collectors such as concurrent mark sweep (CMS), garbage-first (G1), the Z garbage collector (ZGC), Epsilon, and Shenandoah. Each is optimized for different memory profiles, such as high-frequency object creation, large heaps, and so on. By contrast, Go offers a simpler concurrent mark and sweep garbage collector that attempts to optimize itself.

If you need to avoid GC pauses entirely, one option is to use a language that doesn’t have a garbage collector. For example, Swift uses automatic reference counting to determine when memory can be freed, while Rust and Mojo track object lifetimes via the type system so the compiler can determine how long memory must be allocated for.

It’s also possible to use a garbage-collected language while mitigating the impact of pauses. For example, objects can be stored and reused in pools rather than discarded, or data can be allocated off-heap. A more extreme approach is to treat GC pauses like brief planned outages of a node and let other nodes handle requests from clients while one node is collecting its garbage. If the runtime can warn the application that

a node soon requires a GC pause, the application can stop sending new requests to that node, wait for it to finish processing outstanding requests, and then perform the GC while no requests are in progress. This trick hides GC pauses from clients and reduces the high percentiles of the response time [80, 81].

A variant of this idea is to use the garbage collector for only short-lived objects (which are fast to collect) and to restart processes periodically, before they accumulate enough long-lived objects to require a full GC of long-lived objects [79, 82]. One node can be restarted at a time, and traffic can be shifted away from the node before the planned restart, as in a rolling upgrade (see Chapter 5).

These measures cannot fully prevent GC pauses, but they can usefully reduce their impact on the application.

Knowledge, Truth, and Lies

So far in this chapter we have explored the ways in which distributed systems are different from programs running on a single computer. Distributed systems have no shared memory, only message passing via an unreliable network with variable delays, and the systems may suffer from partial failures, unreliable clocks, and processing pauses.

The consequences of these issues are profoundly disorienting if you're not used to distributed systems. A node in the network cannot *know* anything for sure about other nodes—it can only make guesses based on the messages it receives (or doesn't receive). A node can find out another node's state (what data it has stored, whether it is correctly functioning, etc.) only by exchanging messages with it. If a remote node doesn't respond, there is no way of knowing its state, because problems in the network cannot reliably be distinguished from problems at a node.

Discussions of these systems border on the philosophical: What do we know to be true or false in our system? How sure can we be of that knowledge, if the mechanisms for perception and measurement are unreliable [83]? Should software systems obey the laws that we expect of the physical world, such as cause and effect?

Fortunately, we don't need to go as far as figuring out the meaning of life. In a distributed system, we can state the assumptions we are making about the behavior (the *system model*) and design the actual system in such a way that it meets those assumptions. Algorithms can be proved to function correctly within a certain system model. This means that reliable behavior is achievable, even if the underlying system model provides very few guarantees.

However, although it is possible to make software well behaved in an unreliable system model, doing so is not straightforward. In the rest of this chapter we will further explore the notions of knowledge and truth in distributed systems, which will help us

think about the kinds of assumptions we can make and the guarantees we may want to provide. In Chapter 10 we will proceed to look at some examples of distributed algorithms that provide particular guarantees under particular assumptions.

The Majority Rules

Imagine a network with an asymmetric fault: a node is able to receive all messages sent to it, but any outgoing messages from that node are dropped or delayed [22]. Even though that node is working perfectly well and is receiving requests from other nodes, the other nodes cannot hear its responses. After a timeout, the other nodes declare it dead, because they haven't heard from the node. The situation unfolds like a nightmare—the semi-disconnected node is dragged to the graveyard, kicking and screaming “I’m not dead!”—but since nobody can hear its screaming, the funeral procession continues with stoic determination.

In a slightly less nightmarish scenario, the semi-disconnected node may notice that the messages it is sending are not being acknowledged by other nodes and realize that there must be a fault in the network. Nevertheless, the node is wrongly declared dead by the other nodes, and it's unable to do anything about it.

As a third scenario, imagine a node that pauses execution for one minute. During that time, no requests are processed and no responses are sent. The other nodes wait, retry, grow impatient, and eventually declare the node dead and load it onto the hearse. Finally, the pause finishes, and the node's threads continue as if nothing had happened. The other nodes are surprised as the supposedly dead node suddenly raises its head out of the coffin, in full health, and starts cheerfully chatting with bystanders. At first, the paused node doesn't even realize that an entire minute has passed and that it was declared dead—from its perspective, hardly any time has passed since it was last talking to the other nodes.

The moral of these stories is that a node cannot necessarily trust its own judgment of a situation. A distributed system cannot exclusively rely on a single node, because a node may fail at any time, potentially leaving the system stuck and unable to recover. Instead, many distributed algorithms rely on a *quorum* (i.e., voting among the nodes; see Chapter 6): decisions require a minimum number of votes from several nodes in order to reduce the dependence on any one particular node.

That includes decisions about declaring nodes dead. If a quorum of nodes declares another node dead, then it must be considered dead, even if that node still very much feels alive. The individual node must abide by the quorum decision and step down.

Most commonly, the quorum is an absolute majority of more than half the nodes (although other kinds of quorums are possible). A majority quorum allows the system to continue working if a minority of nodes are faulty (with three nodes, one faulty node can be tolerated; with five nodes, two faulty nodes can be tolerated). It

is also safe, because there can be only one majority in the system—there cannot be two majorities with conflicting decisions at the same time. We will discuss the use of quorums in more detail when we get to consensus algorithms in Chapter 10.

Distributed Locks and Leases

Locks and leases in distributed applications are prone to misuse and are a common source of bugs [84]. Let's look at one particular case of how they can go wrong.

In “[Process Pauses](#)” on page 86 we saw that a lease is a kind of lock that times out and can be assigned to a new owner if the old owner stops responding (perhaps because it crashed, it paused for too long, or it was disconnected from the network). You can use leases when a system requires there to be only one of some thing. For example:

- Only one node is allowed to be the leader for a database shard, to avoid split brain (see Chapter 6).
- Only one transaction or client is allowed to update a particular resource or object, to prevent it from being corrupted by concurrent writes.
- Only one node should process a given input file to a big processing job, to avoid wasted effort due to multiple nodes redundantly doing the same work.

It is worth thinking carefully about what happens if several nodes simultaneously believe that they hold the lease, perhaps because of a process pause. In the third example, the consequence is only some wasted computational resources, which is not a big deal. But in the first two cases, the consequence could be lost or corrupted data, which is much more serious.

For example, [Figure 9-4](#) shows a data corruption bug due to an incorrect implementation of locking. (The bug is not theoretical; HBase used to have this problem [85, 86].) Say you want to ensure that a file in a storage service can be accessed by only one client at a time, because if multiple clients tried to write to it, the file would become corrupted. You try to implement this by requiring a client to obtain a lease from a lock service before accessing the file. Such a lock service is often implemented using a consensus algorithm, discussed further in Chapter 10.

The problem is an example of what we discussed in “[Process Pauses](#)” on page 86. If the client holding the lease is paused for too long, its lease expires. Another client can then obtain a lease for the same file and start writing to the file. When the paused client comes back, it believes (incorrectly) that it still has a valid lease and continues writing to the file. We now have a split-brain situation: the clients' writes clash and corrupt the file.

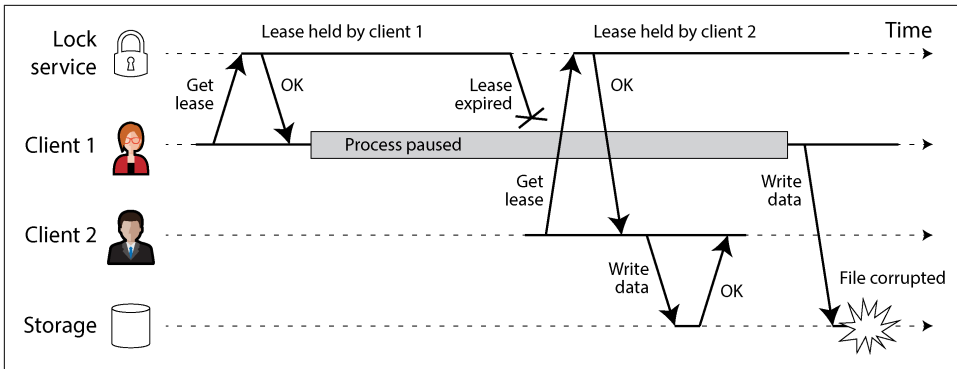


Figure 9-4. An incorrect implementation of a distributed lock: client 1 believes that it still has a valid lease, even though it has expired, and thus corrupts a file in storage

Figure 9-5 shows a different problem that has similar consequences. In this example there is no process pause, only a crash by client 1. Just before client 1 crashes, it sends a write request to the storage service, but this request is delayed for a long time in the network. (Remember from “[Network Faults in Practice](#)” on page 70 that packets can sometimes be delayed by a minute or more.) By the time the write request arrives at the storage service, client 1’s lease has already timed out, allowing client 2 to acquire it and issue a write of its own. The result is corruption similar to that in Figure 9-4.

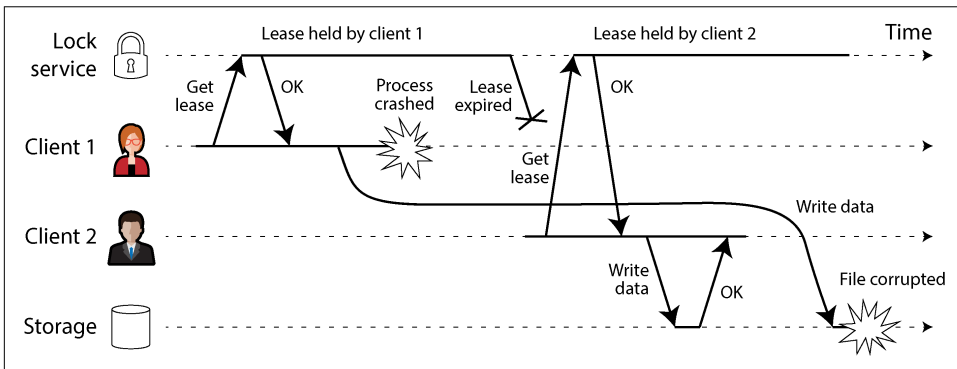


Figure 9-5. A message from a former leaseholder might be delayed for a long time and arrive after another node has taken over the lease.

Fencing off zombies and delayed requests

The term *zombie* is sometimes used to describe a former leaseholder that has not yet found out that it lost the lease and is still acting as if it were the current leaseholder. Since we cannot rule out zombies entirely, we have to instead ensure that they can’t do any damage in the form of split brain. This is called *fencing off* the zombie.

Some systems attempt to fence off zombies by shutting them down—for example, by disconnecting them from the network [9], shutting down the VM via the cloud provider’s management interface, or even physically powering down the machine [87]. This approach is sometimes known as *shoot the other node in the head* (STO-NITH), though we prefer not to use such violent terminology. In any case, it is not particularly effective: this approach does not protect against the large network delays depicted in [Figure 9-5](#); all the nodes could shut one another down [19]; and by the time a zombie has been detected and shut down, it may be too late and data may already have been corrupted.

A more robust fencing solution, which protects against both zombies and delayed requests, is illustrated in [Figure 9-6](#).

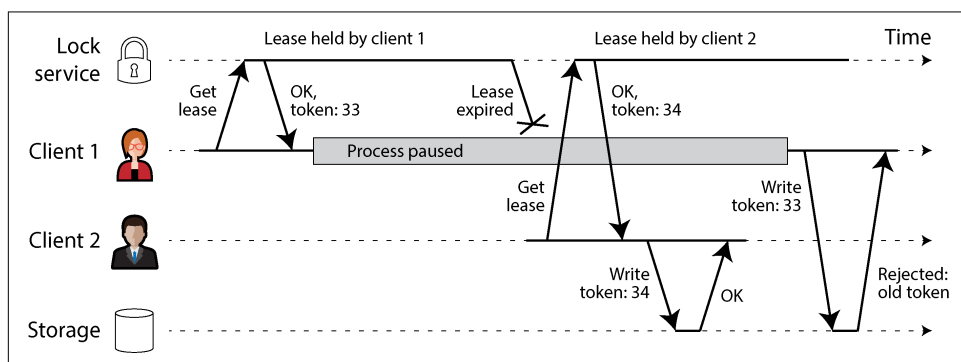


Figure 9-6. Making access to storage safe by allowing writes only in the order of increasing fencing tokens

Let’s assume that every time the lock service grants a lock or lease, it also returns a *fencing token*, which is a number that increases every time a lock is granted (e.g., incremented by the lock service). We can then require that every time a client sends a write request to the storage service, it must include its current fencing token.



There are several alternative names for fencing tokens. In Chubby, Google’s lock service, they are called *sequencers* [88], and in Kafka they are called *epoch numbers*. In consensus algorithms, which we will discuss in Chapter 10, the *ballot number* (Paxos) or *term number* (Raft) serves a similar purpose.

In [Figure 9-6](#), client 1 acquires the lease with a token of 33, but then it goes into a long pause and the lease expires. Client 2 acquires the lease with a token of 34 (the number always increases) and sends its write request to the storage service, including the token. Later, client 1 comes back to life and sends its write to the storage service, including its token value, 33. However, the storage service remembers that it has

already processed a write with a higher token number (34), so it rejects the request with token 33. A client that has just acquired the lease must immediately make a write to the storage service, and once that write has completed, any zombies are fenced off. These operations are similar to the optimistic concurrency control (OCC) technique we saw in Chapter 8, except that fencing is permanent, while concurrency control failures can be retried.

If ZooKeeper is your lock service, you can use the transaction ID `zxid` or the node version `cversion` as a fencing token [85]. With `etcd`, the revision number along with the lease ID serves a similar purpose [89]. The `FencedLock` API in Hazelcast explicitly generates a fencing token [90].

This mechanism requires that the storage service has some way of checking whether a write is based on an outdated token. Alternatively, it's sufficient for the service to support a write that succeeds only if the object has not been written by another client since the current client last read it, similarly to an atomic CAS operation. For example, object storage services support such a check; Amazon S3 calls it *conditional writes*, Azure Blob Storage calls it *conditional headers*, and Google Cloud Storage calls it *request preconditions*.

Fencing with multiple replicas

If your clients need to write to only one storage service that supports conditional writes, the lock service is somewhat redundant [91, 92], since the lease assignment could have been implemented directly based on that storage service [93]. However, once you have a fencing token, you can use it with multiple services or replicas and ensure that the old leaseholder is fenced off on all of them.

For example, imagine the storage service is a leaderless replicated key-value store with LWW conflict resolution (see Chapter 6). In such a system, the client sends writes directly to each replica, and each replica independently decides whether to accept a write based on a timestamp assigned by the client.

As illustrated in [Figure 9-7](#), you can put the writer's fencing token in the most significant bits or digits of the timestamp. You can then be sure that any timestamp generated by the new leaseholder will be greater than any timestamp from the old leaseholder, even if the old leaseholder's writes happened later.

In [Figure 9-7](#), client 2 has a fencing token of 34, so all its timestamps starting with 34... are greater than any timestamps starting with 33... that are generated by client 1. Client 2 writes to a quorum of replicas, but it can't reach replica 3. This means that when the zombie client 1 later tries to write, its write may succeed at replica 3 even though it is ignored by replicas 1 and 2. This is not a problem, since a subsequent quorum read will prefer the write from client 2 with the greater timestamp, and read repair or anti-entropy will eventually overwrite the value written by client 1.

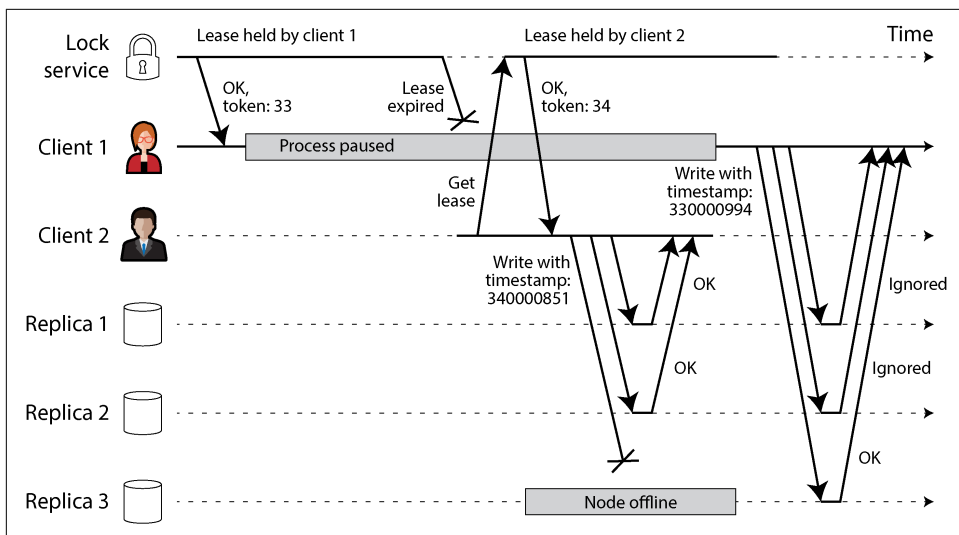


Figure 9-7. Using fencing tokens to protect writes to a leaderless replicated database

As you can see from these examples, it is not safe to assume that only one node is holding a lease at any one time. Fortunately, with a bit of care you can use fencing tokens to prevent zombies and delayed requests from doing any damage.

Byzantine Faults

Fencing tokens can detect and block a node that is *inadvertently* acting in error (e.g., because it hasn't yet found out that its lease has expired). However, if the node deliberately wanted to subvert the system's guarantees, it could easily do so by sending messages with a fake fencing token.

In this book we assume that nodes are unreliable but honest. They may be slow or never respond (because of a fault), and their state may be outdated (because of a GC pause or network delays), but we assume that if a node *does* respond, it is telling the "truth." To the best of its knowledge, it is playing by the rules of the protocol.

Distributed systems problems become much harder if there is a risk that nodes may "lie" (send arbitrary faulty or corrupted responses)—for example, a node might cast multiple contradictory votes in the same election. Such behavior is known as a *Byzantine fault*, and the problem of reaching consensus in this untrusting environment is known as the *Byzantine Generals Problem* [94].

The Byzantine Generals Problem

The Byzantine Generals Problem is a generalization of the *two generals problem* [95], which imagines two army generals needing to agree on a battle plan. As they have set up camp at different sites, they can communicate only by messenger, and the messengers sometimes get delayed or lost (like packets in a network). We will discuss this problem of *consensus* in Chapter 10.

In the Byzantine version of the problem, n generals need to agree, and their endeavor is hampered by traitors in their midst. Most of the generals are loyal and thus send truthful messages, but the traitors may try to deceive and confuse the others by sending fake or untrue messages. It is not known in advance who the traitors are.

Byzantium was an ancient Greek city that later became Constantinople, in the place which is now Istanbul in Turkey. There isn't any historic evidence that the generals of Byzantium were any more prone to intrigue and conspiracy than those elsewhere. Rather, the name is derived from *Byzantine* in the sense of *excessively complicated, bureaucratic, devious*, which was used in politics long before computers [96]. Lamport wanted to choose a nationality that would not offend any readers, and he was advised that calling it *The Albanian Generals Problem* was not such a good idea [97].

Uses of Byzantine fault tolerance

A system is *Byzantine fault-tolerant* if it continues to operate correctly even if some of the nodes are malfunctioning and not obeying the protocol, or if malicious attackers are interfering with the network. This concern is relevant in certain specific circumstances. For example:

- In aerospace environments, the data in a computer's memory or CPU register could become corrupted by radiation, leading it to respond to other nodes in arbitrarily unpredictable ways. Since a system failure would be very expensive (e.g., an aircraft crashing and killing everyone on board, or a rocket colliding with the International Space Station), flight control systems must tolerate Byzantine faults [98, 99].
- In a system with multiple participating parties, some participants may attempt to cheat or defraud others. In such circumstances, it is not safe for a node to simply trust another node's messages, since they may be sent with malicious intent. The consensus mechanisms underlying cryptocurrencies like Bitcoin and other blockchain-based systems can be considered to be a way of getting mutually untrusting parties to agree on whether a transaction happened, without relying on a central authority [100].

However, in the kinds of systems we discuss in this book, we can usually safely assume that there are no Byzantine faults. In a datacenter, all the nodes are controlled by your organization (so they can hopefully be trusted), and radiation levels are low enough that memory corruption is not a major problem (although datacenters in orbit are being considered [101]). Multitenant systems have mutually untrusting tenants, but they are isolated from one another via firewalls, virtualization, and access control policies, not using Byzantine fault tolerance. Protocols for making systems Byzantine fault-tolerant are quite expensive [102], and fault-tolerant embedded systems rely on support from the hardware level [98]. In most server-side data systems, the cost of deploying Byzantine fault-tolerant solutions makes them impracticable.

Web applications do need to expect arbitrary and malicious behavior from clients that are under end-user control, such as web browsers. This is why input validation, sanitization, and output escaping are so important: to prevent SQL injection and cross-site scripting, for example. However, we typically don't use Byzantine fault-tolerant protocols here, but simply make the server the authority on what client behavior is and isn't allowed. In peer-to-peer networks, where there is no such central authority, Byzantine fault tolerance is more relevant [103, 104].

A bug in the software could be regarded as a Byzantine fault, but if you deploy the same software to all nodes, then a Byzantine fault-tolerant algorithm cannot save you. Most Byzantine fault-tolerant algorithms require a supermajority of more than two-thirds of the nodes to be functioning correctly (e.g., if you have four nodes, at most one may malfunction). To use this approach against bugs, you would have to have four independent implementations of the same software and hope that a given bug appears in only one of the four implementations.

Similarly, it would be appealing if a protocol could protect us from vulnerabilities, security compromises, and malicious attacks. Unfortunately, this is not realistic either. In most systems, if an attacker can compromise one node, they can probably compromise all of them, because the nodes are probably running the same software. Thus, traditional mechanisms (authentication, access control, encryption, firewalls, and so on) continue to be the main protection against attackers.

Weak forms of lying

Although we assume that nodes are generally honest, it can be worth adding mechanisms to software that guard against weak forms of “lying”—for example, invalid messages due to hardware issues, software bugs, and misconfiguration. Such protection mechanisms are not full-blown Byzantine fault tolerance, as they would not withstand a determined adversary, but they are nevertheless simple and pragmatic steps toward better reliability. For example:

- Network packets do sometimes get corrupted because of hardware issues or bugs in operating systems, drivers, routers, etc. Usually, corrupted packets are caught

by the checksums built into TCP and UDP, but sometimes they evade detection [105, 106, 107]. Simple measures are usually sufficient protection against such corruption, such as checksums in the application-level protocol. TLS-encrypted connections also offer protection against corruption.

- A publicly accessible application must carefully sanitize any user input—for example, by escaping certain characters to prevent SQL injection attacks, checking that a value is within a reasonable range, and limiting the size of strings to prevent denial of service through large memory allocations. An internal service behind a firewall may be able to get away with less strict checks on inputs, but basic checks in protocol parsers are still a good idea [105].
- NTP clients can be configured with multiple server addresses. When synchronizing, the client contacts all of them, estimates their errors, and checks that a majority of servers agree on a time range. As long as most of the servers are OK, a misconfigured NTP server that is reporting an incorrect time is detected as an outlier and is excluded from synchronization [39]. The use of multiple servers makes NTP more robust than if it uses only a single server.

System Model and Reality

Many algorithms have been designed to solve distributed systems problems—for example, we will examine solutions for the consensus problem in Chapter 10. In order to be useful, these algorithms need to tolerate the various faults of distributed systems that we discussed in this chapter.

Algorithms must be written in a way that does not depend too heavily on the details of the hardware and software configuration on which they are run. This in turn requires that we somehow formalize the kinds of faults that we expect to happen in a system. We do this by defining a *system model*, which is an abstraction that describes an algorithm's assumptions.

With regard to timing assumptions, three system models are in common use:

Synchronous model

The synchronous model assumes bounded network delay, bounded process pauses, and bounded clock error. This does not imply exactly synchronized clocks or zero network delay; it just means you know that network delay, pauses, and clock drift will never exceed a fixed upper bound [108]. The synchronous model is not a realistic model of most practical systems because (as discussed in this chapter) unbounded delays and pauses do occur.

Partially synchronous model

Partial synchrony means that a system behaves like a synchronous system *most of the time*, but it sometimes exceeds the bounds for network delay, process pauses, and clock drift [108]. This is a realistic model of many systems. Most of the time,

networks and processes are quite well behaved—otherwise, we would never be able to get anything done—but we have to reckon with the fact that any timing assumptions may be shattered occasionally. When this happens, network delay, pauses, and clock error may become arbitrarily large.

Asynchronous model

In this model, an algorithm is not allowed to make any timing assumptions—in fact, it does not even have a clock (so it cannot use timeouts). Some algorithms can be designed for the asynchronous model, but it is very restrictive.

Besides timing issues, we also have to consider node failures. Some common system models for nodes are as follows:

Crash-stop faults

In the *crash-stop* (or *fail-stop*) model, an algorithm may assume that a node can fail in only one way—namely, by crashing [109]. The node may suddenly stop responding at any moment, and thereafter that node is gone forever—it never comes back.

Crash-recovery faults

In the crash-recovery model, we assume that nodes may crash at any moment, and perhaps start responding again after an unknown time. Nodes are assumed to have stable storage (i.e., nonvolatile disk storage) that is preserved across crashes, while the in-memory state is assumed to be lost.

Degraded performance and partial functionality

In addition to crashing and restarting, nodes may slow down. They may still be able to respond to health check requests, while being too slow to get any real work done. For example, a Gigabit network interface could suddenly drop to 1 Kb/s throughput because of a driver bug [110]; a process that is under memory pressure may spend most of its time performing garbage collection [111]; worn-out SSDs can have erratic performance; and hardware can be affected by high temperature, loose connectors, mechanical vibration, power supply problems, firmware bugs, and more [112]. Such a situation is called a *limping node*, *gray failure*, or *fail-slow* [113], and it can be even more difficult to deal with than a cleanly failed node. A related problem occurs when a process stops doing some of the things it is supposed to do while other aspects continue working—for example, because a background thread is crashed or deadlocked [114].

Byzantine (arbitrary) faults

Nodes may do absolutely anything, including trying to trick and deceive other nodes, as described in the previous section.

For modeling real systems, the partially synchronous model with crash-recovery faults is generally the most useful. It allows for unbounded network delay, process pauses, and slow nodes. But how do distributed algorithms cope with that model?

Defining the correctness of an algorithm

To define what it means for an algorithm to be *correct*, we can describe its *properties*. For example, the output of a sorting algorithm has the property that for any two distinct elements of the output list, the element further to the left is smaller than the element further to the right. That is simply a formal way of defining what it means for a list to be sorted—an invariant of a sorted list.

Similarly, we can write down the properties we want of a distributed algorithm to define what it means to be correct. For example, if we are generating fencing tokens for a lock (see “[Fencing off zombies and delayed requests](#)” on page 94), we may require the algorithm to have the following properties:

Uniqueness

No two requests for a fencing token return the same value.

Monotonic sequence

If request x returned token t_x , and request y returned token t_y , and x completed before y began, then $t_x < t_y$.

Availability

A node that requests a fencing token and does not crash eventually receives a response.

An algorithm is correct in a system model if it always satisfies its properties in all situations that we assume may occur in that system model. However, if all nodes crash, or all network delays suddenly become infinitely long, then no algorithm will be able to get anything done. How can we still make useful guarantees even in a system model that allows complete failures?

Distinguishing between safety and liveness

To clarify the situation, it is worth distinguishing between two kinds of properties: *safety* and *liveness*. In the example just given, *uniqueness* and *monotonic sequence* are safety properties, but *availability* is a liveness property.

What distinguishes the two kinds of properties? A giveaway is that liveness properties often include the word “eventually” in their definition. (And yes, you guessed it—*eventual consistency* is a liveness property [115].)

Safety is often informally defined as *nothing bad happens*, and liveness as *something good eventually happens*. However, it’s best to not read too much into those informal definitions, because “good” and “bad” are value judgments that don’t apply well to algorithms. The actual definitions of safety and liveness are more precise [116]:

- If a safety property is violated, we can point to the particular point in time that it was broken (e.g., if the uniqueness property was violated, we can identify the

particular operation in which a duplicate fencing token was returned). After a safety property has been violated, the violation cannot be undone—the damage is already done.

- A liveness property works the other way around. It may not hold at a certain point in time (e.g., a node may have sent a request but not yet received a response), but there is always hope that it may be satisfied in the future (namely, by receiving a response).

An advantage of distinguishing between safety and liveness properties is that it helps us deal with difficult system models. For distributed algorithms, it is common to require that safety properties *always* hold, in all possible situations of a system model [108]. Even if all nodes crash, or the entire network fails, the algorithm must nevertheless ensure that it does not return a wrong result (i.e., that the safety properties remain satisfied).

However, with liveness properties we are allowed to make caveats—for example, we could say that a request needs to receive a response only if a majority of nodes have not crashed, and only if the network eventually recovers from an outage. The definition of the partially synchronous model requires that eventually the system returns to a synchronous state—that is, any period of network interruption lasts only for a finite duration and is then repaired.

Mapping system models to the real world

Safety and liveness properties and system models are very useful for reasoning about the correctness of a distributed algorithm. However, when implementing an algorithm in practice, the messy facts of reality come back to bite you again, and it becomes clear that the system model is a simplified abstraction of reality.

For example, algorithms in the crash-recovery model generally assume that data in stable storage survives crashes. However, what happens if the data on disk is corrupted or wiped out by hardware error or misconfiguration [117]? What happens if a server has a firmware bug and fails to recognize its hard drives on reboot even though the drives are correctly attached to the server [118]?

Quorum algorithms (see Chapter 6) rely on a node remembering the data that it claims to have stored. If a node may suffer from amnesia and forget previously stored data, that breaks the quorum condition and thus breaks the correctness of the algorithm. Perhaps a new system model is needed, in which we assume that stable storage mostly survives crashes but may sometimes be lost. But that model then becomes harder to reason about.

The theoretical description of an algorithm can declare that certain things are simply assumed not to happen—and in non-Byzantine systems, we do have to make assumptions about faults that can and cannot happen. However, a real implementation

may still have to include code to handle the case of something happening that was assumed to be impossible, even if that handling boils down to `printf("Sucks to be you")` and `exit(666)`—that is, letting a human operator clean up the mess [119]. (This is one difference between computer science and software engineering.)

That is not to say that theoretical, abstract system models are worthless—quite the opposite. They are incredibly helpful for distilling down the complexity of real systems to a manageable set of faults that we can reason about so that we can understand the problem and try to solve it systematically.

Formal Methods and Randomized Testing

How do we know that an algorithm satisfies the required properties? Because of concurrency, partial failures, and network delays, there are a huge number of potential states. We need to guarantee that the properties hold in every possible state and ensure that we haven't forgotten about any edge cases.

One approach is to formally verify an algorithm by describing it mathematically and using proof techniques to show that it satisfies the required properties in all situations that the system model allows. Proving an algorithm correct does not mean its *implementation* on a real system will necessarily always behave correctly. But it's a very good first step, because the theoretical analysis can uncover problems in an algorithm that might remain hidden for a long time in a real system and that come to bite you only when your assumptions (e.g., about timing) are defeated by unusual circumstances.

It is prudent to combine theoretical analysis with empirical testing to verify that implementations behave as expected. Techniques such as property-based testing, fuzzing, and deterministic simulation testing use randomization to test a system in a wide range of situations. Organizations such as Amazon Web Services, FoundationDB, and TigerBeetle have successfully used a combination of these techniques on many of their products [120, 121, 122, 123].

Model checking and specification languages

Model checkers are tools that help verify that an algorithm or system behaves as expected. An algorithm specification is written in a purpose-built language such as TLA+, Gallina, or FizzBee. These languages make it easier to focus on an algorithm's behavior without worrying about code implementation details. Model checkers then use these models to verify that invariants hold across all of an algorithm's states by systematically trying all the things that could happen.

Model checking can't actually prove that an algorithm's invariants hold for every possible state, since most real-world algorithms have an infinite state space. A true verification of all states would require a formal proof, which can be done, but which

is typically more difficult than running a model checker. Instead, model checkers encourage you to reduce the algorithm's model to an approximation that can be fully verified, or to limit the execution to an upper bound (e.g., by setting a maximum number of messages that can be sent). Any bugs that occur only with longer executions would then not be found.

Still, model checkers strike a nice balance between ease of use and the ability to find nonobvious bugs. CockroachDB, TiDB, Kafka, and many other distributed systems use model specifications to find and fix bugs [124, 125, 126]. For example, using TLA+, researchers were able to demonstrate the potential for data loss in viewstamped replication (VR) caused by ambiguity in the prose description of the algorithm [127].

By design, model checkers don't run your actual code, but rather a simplified model that specifies only the core ideas of your protocol. This makes it more tractable to systematically explore the state space, but it risks that your specification and your implementation go out of sync with each other [128]. It is possible to check whether the model and the real implementation have equivalent behavior, but this requires instrumentation in the real implementation [129].

Fault injection

Many bugs are triggered when machine and network failures occur. *Fault injection* is an effective (and sometimes scary) technique that verifies whether a system's implementation works as expected when things go wrong. The idea is simple: inject faults into a running system's environment and see how it behaves. Faults can be network failures, machine crashes, disk corruption, paused processes—anything you can imagine going wrong with a computer.

Fault injection tests are typically run in an environment that closely resembles the production environment where the system will run. Some even inject faults directly into their production environment. Netflix popularized this approach with its Chaos Monkey tool [130]. Production fault injection is often referred to as *chaos engineering*, which we discussed in “[Reliability and Fault Tolerance](#)” on page 43.

To run fault injection tests, the system under test is first deployed along with fault injection coordinators and scripts. Coordinators are responsible for deciding what faults to execute and when to execute them. Local or remote scripts are responsible for injecting failures into individual nodes or processes. Injection scripts use many tools to trigger faults. A Linux process can be paused or killed using Linux's `kill` command, a disk can be unmounted with `umount`, and network connections can be disrupted through firewall settings. You can inspect system behavior during and after faults are injected to make sure things work as expected.

The myriad of tools required to trigger failures make fault injection tests cumbersome to write. It's common to adopt a fault injection framework like Jepsen to run fault injection tests to simplify the process. Such frameworks come with integrations for various operating systems and many prebuilt fault injectors [131]. Jepsen has been remarkably effective at finding critical bugs in many widely used systems [132, 133].

Deterministic simulation testing

Another formalization technique, which has become a popular complement to model checking and fault injection, is *deterministic simulation testing* (DST). It uses a similar state space exploration process to a model checker, but it tests your actual code, not a model.

In DST, a simulation automatically runs through a large number of randomized executions of the system. Network communication, I/O, and clock timing during the simulation are all replaced with mocks that allow the simulator to control the exact order in which things happen, including various timings and failure scenarios. This allows the simulator to explore many more situations than handwritten tests or fault injection could. If a test fails, it can be rerun since the simulator knows the exact order of operations that triggered the failure—in contrast to fault injection, which does not have such fine-grained control over the system.

DST requires the simulator to be able to control all sources of nondeterminism, such as network delays or thread scheduling in multithreaded code. One of three strategies is generally adopted to make code deterministic:

Application-level

Some systems are built from the ground up to make it easy to execute code deterministically. For example, FoundationDB, one of the pioneers in the DST space, is built using an asynchronous communication library called Flow. Flow provides a point for developers to inject a deterministic network simulation into the system [134]. Similarly, TigerBeetle is an OLTP database with first-class DST support. The system's state is modeled as a state machine, with all mutations occurring within a single event loop. When combined with mock deterministic primitives such as clocks, such an architecture is able to run deterministically [135].

Runtime-level

Languages with asynchronous runtimes and commonly used libraries provide an insertion point to introduce determinism. A single-threaded runtime is used to force all asynchronous code to run sequentially. FrostDB, for example, patches Go's runtime to execute goroutines sequentially [136]. Rust's MadSim library works in a similar manner. It provides deterministic implementations of Tokio's asynchronous runtime API, Amazon's S3 library, Kafka's Rust library, and many

others; applications can swap in deterministic libraries and runtimes to get deterministic test executions without changing their code.

Machine-level

Rather than patching code at runtime, an entire machine can be made deterministic. This is a delicate process that requires a machine to respond to all normally nondeterministic calls with deterministic responses. Tools such as Antithesis do this by building a custom hypervisor that replaces normally nondeterministic operations with deterministic ones. Everything from clocks to network and storage needs to be accounted for. Once that's done, though, developers can run their entire distributed system in a collection of containers within the hypervisor and get a completely deterministic distributed system.

DST provides several advantages beyond replayability. For example, Antithesis attempts to explore many paths in application code by branching a test execution into multiple subexecutions when it discovers less common behavior. And because deterministic tests often use mocked clocks and network calls, such tests can run faster than wall clock time. TigerBeetle's time abstraction, for instance, allows simulations to simulate network latency and timeouts without actually taking the full length of time to trigger the timeout. Such techniques allow the simulator to explore more code paths faster.

The Power of Determinism

Nondeterminism is at the core of all the distributed systems challenges we discussed in this chapter: concurrency, network delay, process pauses, clock jumps, and crashes all happen in unpredictable ways that vary from one run of a system to the next. Conversely, if you can make a system deterministic, that can hugely simplify things.

In fact, making things deterministic is a simple but powerful idea that arises again and again in distributed system design. Besides deterministic simulation testing, we have seen several ways of using determinism over the past chapters:

- A key advantage of event sourcing (see Chapter 3) is that you can deterministically replay a log of events to reconstruct derived materialized views.
- Workflow engines (see Chapter 5) rely on workflow definitions being deterministic to provide durable execution semantics.
- *State machine replication*, which we will discuss in Chapter 10, replicates data by independently executing the same sequence of deterministic transactions on each replica. We have already seen two variants of that idea: statement-based replication (see Chapter 6) and serial transaction execution using stored procedures (see Chapter 8).

However, making code fully deterministic requires care. Even once you have removed all concurrency and replaced I/O, network communication, clocks, and random number generators with deterministic simulations, elements of nondeterminism may remain. For example, in some programming languages, the order in which you iterate over the elements of a hash table may be nondeterministic. Whether you run into a resource limit (memory allocation failure, stack overflow) is also nondeterministic.

Summary

In this chapter we have discussed a wide range of problems that can occur in distributed systems. For example:

- Whenever you try to send a packet over the network, it may be lost or arbitrarily delayed. Likewise, the reply may be lost or delayed, so if you don't get a reply, you have no idea whether the message got through.
- A node's clock may be significantly out of sync with other nodes (despite your best efforts to set up NTP), it may suddenly jump forward or back in time, and relying on it is dangerous because you most likely don't have a good measure of your clock's confidence interval.
- A process may pause for a substantial amount of time at any point in its execution, be declared dead by other nodes, and then come back to life again without realizing that it was paused.

The fact that such *partial failures* can occur is the defining characteristic of distributed systems. Whenever software tries to do anything involving other nodes, there is the possibility that it may occasionally fail, or randomly go slow, or not respond at all (and eventually time out). In distributed systems, we try to build tolerance of partial failures into software so that the system as a whole may continue functioning even when some of its constituent parts are broken.

To tolerate faults, the first step is to *detect* them, but even that is hard. Most systems don't have an accurate mechanism for detecting whether a node has failed, so most distributed algorithms rely on timeouts to determine whether a remote node is still available. However, timeouts can't distinguish between network and node failures, and variable network delay sometimes causes a node to be falsely suspected of crashing. Handling limping nodes, which are responding but are too slow to do anything useful, is even harder.

Once a fault is detected, making a system tolerate it is not easy either; there is no global variable, no shared memory, no common knowledge or any other kind of shared state between the machines [83]. Nodes can't even agree on what time it is, let alone on anything more profound. The only way information can flow from one node to another is by sending it over the unreliable network. Major decisions cannot

be safely made by a single node, so we require protocols that enlist help from other nodes and try to get a quorum to agree.

If you're used to writing software in the idealized mathematical perfection of a single computer, where the same operation always deterministically returns the same result, then moving to the messy physical reality of distributed systems can be a bit of a shock. Conversely, distributed systems engineers will often regard a problem as trivial if it can be solved on a single computer [4], and indeed a single computer can do a lot nowadays. If you can avoid opening Pandora's box and simply keep things on a single machine—for example, by using an embedded storage engine (see Chapter 4)—it is generally worth doing so.

However, as discussed in “[Distributed Versus Single-Node Systems](#)” on page 19, scalability is not the only reason for wanting to use a distributed system. Fault tolerance and low latency (by placing data geographically close to users) are equally important goals, and they cannot be achieved with a single node. The power of distributed systems is that in principle, they can run forever without being interrupted at the service level, because all faults and maintenance can be handled at the node level. (In practice, if a bad configuration change is rolled out to all nodes, that will still bring a distributed system to its knees.)

In this chapter we also went on some tangents to explore whether the unreliability of networks, clocks, and processes is an inevitable law of nature. We saw that it isn't: it is possible to give hard real-time response guarantees and bounded delays in networks, but doing so is very expensive and results in lower utilization of hardware resources. Most non-safety-critical systems choose cheap and unreliable over expensive and reliable.

This chapter has been all about problems, and it has presented us a bleak outlook. We gain a lot by using extensively tested, production-grade distributed systems that manage these problems. In the next chapter we will move on to solutions and discuss some algorithms such systems employ to cope with these issues.

References

- [1] Mark Cavage. “[There's Just No Getting Around It: You're Building a Distributed System.](#)” *ACM Queue*, volume 11, issue 4, pages 80–89, April 2013. [doi:10.1145/2466486.2482856](#)
- [2] Jay Kreps. “[Getting Real About Distributed System Reliability.](#)” *blog.empathy-box.com*, March 2012. Archived at [perma.cc/9B5Q-AEBW](#)
- [3] Coda Hale. “[You Can't Sacrifice Partition Tolerance.](#)” *codahale.com*, October 2010. Archived at [perma.cc/6GJU-X4G5](#)

- [4] Jeff Hodges. “Notes on Distributed Systems for Young Bloods.” *somethingsimilar.com*, January 2013. Archived at perma.cc/B636-62CE
- [5] Van Jacobson. “Congestion Avoidance and Control.” At *ACM Symposium on Communications Architectures and Protocols (SIGCOMM)*, August 1988. [doi:10.1145/52324.52356](https://doi.org/10.1145/52324.52356)
- [6] Bert Hubert. “The Ultimate SO_LINGER Page, or: Why Is My TCP Not Reliable.” *blog.netherlabs.nl*, January 2009. Archived at perma.cc/6HDX-L2RR
- [7] Jerome H. Saltzer, David P. Reed, and David D. Clark. “End-To-End Arguments in System Design.” *ACM Transactions on Computer Systems*, volume 2, issue 4, pages 277–288, November 1984. [doi:10.1145/357401.357402](https://doi.org/10.1145/357401.357402)
- [8] Peter Bailis and Kyle Kingsbury. “The Network Is Reliable.” *ACM Queue*, volume 12, issue 7, pages 48–55, July 2014. [doi:10.1145/2639988.2639988](https://doi.org/10.1145/2639988.2639988)
- [9] Joshua B. Leners, Trinabh Gupta, Marcos K. Aguilera, and Michael Wal-fish. “Taming Uncertainty in Distributed Systems with Help from the Net-work.” At *10th European Conference on Computer Systems (EuroSys)*, April 2015. [doi:10.1145/2741948.2741976](https://doi.org/10.1145/2741948.2741976)
- [10] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. “Understanding Network Failures in Data Centers: Measurement, Analysis, and Implications.” At *ACM SIGCOMM Conference*, August 2011. [doi:10.1145/2018436.2018477](https://doi.org/10.1145/2018436.2018477)
- [11] Urs Hölzle. “But recently a farmer had started grazing a herd of cows nearby. And whenever they stepped on the fiber link, they bent it enough to cause a blip.” *x.com*, May 2020. Archived at perma.cc/WX8X-ZZA5
- [12] CBC News. “Hundreds Lose Internet Service in Northern B.C. After Beaver Chews Through Cable.” *cbc.ca*, April 2021. Archived at perma.cc/UW8C-H2MY
- [13] Will Oremus. “The Global Internet Is Being Attacked by Sharks, Google Con-firms.” *slate.com*, August 2014. Archived at perma.cc/P6F3-C6YG
- [14] Jess Auerbach Jahajeeah. “Down to the Wire: The Ship Fixing Our Internet.” *continent.substack.com*, November 2023. Archived at perma.cc/DP7B-EQ7S
- [15] Santosh Janardhan. “More Details About the October 4 Outage.” *engineering.fb.com*, October 2021. Archived at perma.cc/WW89-VSXH
- [16] Tom Parfitt. “Georgian Woman Cuts off Web Access to Whole of Armenia.” *theguardian.com*, April 2011. Archived at perma.cc/KMC3-N3NZ
- [17] Antonio Voce, Tural Ahmedzade and Ashley Kirk. “‘Shadow Fleets’ and Sub-aquatic Sabotage: Are Europe’s Undersea Internet Cables Under Attack?” *theguardian.com*, March 2025. Archived at perma.cc/HA7S-ZDBV

- [18] Shengyun Liu, Paolo Viotti, Christian Cachin, Vivien Quéma, and Marko Vukolić. “XFT: Practical Fault Tolerance Beyond Crashes.” At *12th USENIX Symposium on Operating Systems Design and Implementation* (OSDI), November 2016.
- [19] Mark Imbriaco. “Downtime Last Saturday.” *github.blog*, December 2012. Archived at perma.cc/M7X5-E8SQ
- [20] Tom Lianza and Chris Snook. “A Byzantine Failure in the Real World.” *blog.cloudflare.com*, November 2020. Archived at perma.cc/83EZ-ALCY
- [21] Mohammed Alfatafta, Basil Alkhatib, Ahmed Alquraan, and Samer Al-Kiswany. “Toward a Generic Fault Tolerance Technique for Partial Network Partitioning.” At *14th USENIX Symposium on Operating Systems Design and Implementation* (OSDI), November 2020.
- [22] Marc A. Donges. “Re: bnx2 Cards Intermittantly Going Offline.” Message to Linux *netdev* mailing list, *spinics.net*, September 2012. Archived at perma.cc/TXP6-H8R3
- [23] Troy Toman. “Inside a CODE RED: Network Edition.” *signalvnoise.com*, September 2020. Archived at perma.cc/BET6-FY25
- [24] Kyle Kingsbury. “Jepsen: Elasticsearch.” *aphyr.com*, June 2014. Archived at perma.cc/JK47-S89J
- [25] Salvatore Sanfilippo. “A Few Arguments About Redis Sentinel Properties and Fail Scenarios.” *antirez.com*, October 2014. Archived at perma.cc/8XEU-CLM8
- [26] Nicolas Liochon. “CAP: If All You Have Is a Timeout, Everything Looks Like a Partition.” *blog.thislongrun.com*, May 2015. Archived at perma.cc/FS57-V2PZ
- [27] Matthew P. Grosvenor, Malte Schwarzkopf, Ionel Gog, Robert N. M. Watson, Andrew W. Moore, Steven Hand, and Jon Crowcroft. “Queues Don’t Matter When You Can JUMP Them!” At *12th USENIX Symposium on Networked Systems Design and Implementation* (NSDI), May 2015.
- [28] Theo Julienne. “Debugging Network Stalls on Kubernetes.” *github.blog*, November 2019. Archived at perma.cc/K9M8-XVGL
- [29] Guohui Wang and T. S. Eugene Ng. “The Impact of Virtualization on Network Performance of Amazon EC2 Data Center.” At *29th IEEE International Conference on Computer Communications* (INFOCOM), March 2010. [doi:10.1109/INFOCOM.2010.5461931](https://doi.org/10.1109/INFOCOM.2010.5461931)
- [30] Brandon Philips. “etcd: Distributed Locking and Service Discovery.” At *Strange Loop*, September 2014.
- [31] Steve Newman. “A Systematic Look at EC2 I/O.” *blog.scalyr.com*, October 2012. Archived at perma.cc/FL4R-H2VE

- [32] Naohiro Hayashibara, Xavier Défago, Rami Yared, and Takuya Katayama. “The ϕ Accrual Failure Detector.” Japan Advanced Institute of Science and Technology, School of Information Science, Technical Report IS-RR-2004-010, May 2004. Archived at perma.cc/NSM2-TRYA
- [33] Jeffrey Wang. “Phi Accrual Failure Detector.” *ternarysearch.blogspot.co.uk*, August 2013. Archived at perma.cc/L452-AMLV
- [34] Srinivasan Keshav. *An Engineering Approach to Computer Networking: ATM Networks, the Internet, and the Telephone Network*. Addison-Wesley Professional, 1997. ISBN: 9780201634426
- [35] Othmar Kyas. *ATM Networks*. International Thomson Publishing, 1995. ISBN: 9781850321286
- [36] Jialin Li, Naveen Kr. Sharma, Dan R. K. Ports, and Steven D. Gribble. “Tales of the Tail: Hardware, OS, and Application-level Sources of Tail Latency.” At *ACM Symposium on Cloud Computing (SOCC)*, November 2014. [doi:10.1145/2670979.2670988](https://doi.org/10.1145/2670979.2670988)
- [37] Mellanox Technologies. “InfiniBand FAQ, Rev 1.3.” *network.nvidia.com*, December 2014. Archived at perma.cc/LQJ4-QZVK
- [38] Jose Renato Santos, Yoshio Turner, and G. (John) Janakiraman. “End-to-End Congestion Control for InfiniBand.” At *22nd Annual Joint Conference of the IEEE Computer and Communications Societies (INFOCOM)*, April 2003. Also published by HP Laboratories Palo Alto, Tech Report HPL-2002-359. [doi:10.1109/INFCOM.2003.1208949](https://doi.org/10.1109/INFCOM.2003.1208949)
- [39] Ulrich Windl, David Dalton, Marc Martinec, and Dale R. Worley. “The NTP FAQ and HOWTO.” *ntp.org*, November 2006. Archived at archive.org
- [40] John Graham-Cumming. “How and Why the Leap Second Affected Cloudflare DNS.” *blog.cloudflare.com*, January 2017. Archived at archive.org
- [41] David Holmes. “Inside the Hotspot VM: Clocks, Timers and Scheduling Events —Part I—Windows.” *blogs.oracle.com*, October 2006. Archived at archive.org
- [42] Joran Dirk Greef. “Three Clocks Are Better than One.” *tigerbeetle.com*, August 2021. Archived at perma.cc/5RXG-EU6B
- [43] Oliver Yang. “Pitfalls of TSC Usage.” *oliveryang.net*, September 2015. Archived at perma.cc/Z2QY-5FRA
- [44] Steve Loughran. “Time on Multi-Core, Multi-Socket Servers.” *stevelloughran.blogspot.co.uk*, September 2015. Archived at perma.cc/7M4S-D4U6
- [45] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, JJ Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander

Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Dale Woodford, Yasushi Saito, Christopher Taylor, Michal Szymaniak, and Ruth Wang. “[Spanner: Google’s Globally-Distributed Database](#).” At *10th USENIX Symposium on Operating System Design and Implementation (OSDI)*, October 2012.

[46] M. Caporloni and R. Ambrosini. “[How Closely Can a Personal Computer Clock Track the UTC Timescale Via the Internet?](#)” *European Journal of Physics*, volume 23, issue 4, pages L17–L21, June 2012. [doi:10.1088/0143-0807/23/4/103](#)

[47] Nelson Minar. “[A Survey of the NTP Network](#).” *alumni.media.mit.edu*, December 1999. Archived at [perma.cc/EV76-7ZV3](#)

[48] Viliam Holub. “[Synchronizing Clocks in a Cassandra Cluster Pt. 1—The Problem](#).” *blog.rapid7.com*, March 2014. Archived at [perma.cc/N3RV-5LNL](#)

[49] Poul-Henning Kamp. “[The One-Second War \(What Time Will You Die?\)](#)” *ACM Queue*, volume 9, issue 4, pages 44–48, April 2011. [doi:10.1145/1966989.1967009](#)

[50] Nelson Minar. “[Leap Second Crashes Half the Internet](#).” *somebits.com*, July 2012. Archived at [perma.cc/2WB8-D6EU](#)

[51] Christopher Pascoe. “[Time, Technology and Leaping Seconds](#).” *googleblog.blogspot.co.uk*, September 2011. Archived at [perma.cc/U2JL-7E74](#)

[52] Mingxue Zhao and Jeff Barr. “[Look Before You Leap—The Coming Leap Second and AWS](#).” *aws.amazon.com*, May 2015. Archived at [perma.cc/KPE9-XMFM](#)

[53] Darryl Veitch and Kanthaiah Vijayalayan. “[Network Timing and the 2015 Leap Second](#).” At *17th International Conference on Passive and Active Measurement (PAM)*, April 2016. [doi:10.1007/978-3-319-30505-9_29](#)

[54] VMware, Inc. “[Timekeeping in VMware Virtual Machines](#).” *vmware.com*, October 2008. Archived at [perma.cc/HM5R-T5NF](#)

[55] Victor Yodaiken. “[Clock Synchronization in Finance and Beyond](#).” *yodaiken.com*, November 2017. Archived at [perma.cc/9XZD-8ZZN](#)

[56] Mustafa Emre Acer, Emily Stark, Adrienne Porter Felt, Sascha Fahl, Radhika Bhargava, Bhanu Dev, Matt Braithwaite, Ryan Sleevi, and Parisa Tabriz. “[Where the Wild Warnings Are: Root Causes of Chrome HTTPS Certificate Errors](#).” At *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, October 2017. [doi:10.1145/3133956.3134007](#)

[57] European Securities and Markets Authority. “[MiFID II / MiFIR: Regulatory Technical and Implementing Standards—Annex I](#).” *esma.europa.eu*, Report ESMA/2015/1464, September 2015. Archived at [perma.cc/ZLX9-FGQ3](#)

[58] Luke Bigum. “[Solving MiFID II Clock Synchronisation with Minimum Spend \(Part 1\)](#).” *catnach.blogspot.com*, November 2015. Archived at [perma.cc/4J5W-FNM4](#)

- [59] Oleg Obleukhov and Ahmad Byagowi. “How Precision Time Protocol Is Being Deployed at Meta.” *engineering.fb.com*, November 2022. Archived at perma.cc/29G6-UJNW
- [60] John Wiseman. “GPSJAM: Daily Maps of GPS Interference.” *gpsjam.org*
- [61] Josh Levinson, Julien Ridoux, and Chris Munns. “It’s About Time: Microsecond-Accurate Clocks on Amazon EC2 Instances.” *aws.amazon.com*, November 2023. Archived at perma.cc/56M6-5VMZ
- [62] Kyle Kingsbury. “Jepsen: Cassandra.” *aphyr.com*, September 2013. Archived at perma.cc/4MBR-J96V
- [63] John Daily. “Clocks Are Bad, or, Welcome to the Wonderful World of Distributed Systems.” *riak.com*, November 2013. Archived at perma.cc/4XB5-UCXY
- [64] Marc Brooker. “It’s About Time!” *brooker.co.za*, November 2023. Archived at perma.cc/N6YK-DRPA
- [65] Kyle Kingsbury. “The Trouble with Timestamps.” *aphyr.com*, October 2013. Archived at perma.cc/W3AM-5VAV
- [66] Leslie Lamport. “Time, Clocks, and the Ordering of Events in a Distributed System.” *Communications of the ACM*, volume 21, issue 7, pages 558–565, July 1978. doi:10.1145/359545.359563
- [67] Justin Sheehy. “There Is No Now: Problems with Simultaneity in Distributed Systems.” *ACM Queue*, volume 13, issue 3, pages 36–41, March 2015. doi:10.1145/2733108
- [68] Murat Demirbas. “Spanner: Google’s Globally-Distributed Database.” *muratbuffalo.blogspot.co.uk*, July 2013. Archived at perma.cc/6VWR-C9WB
- [69] Dahlia Malkhi and Jean-Philippe Martin. “Spanner’s Concurrency Control.” *ACM SIGACT News*, volume 44, issue 3, pages 73–77, September 2013. doi:10.1145/2527748.2527767
- [70] Franck Pachot. “Achieving Precise Clock Synchronization on AWS.” *yuga-byte.com*, December 2024. Archived at perma.cc/UYM6-RNBS
- [71] Spencer Kimball. “Living Without Atomic Clocks: Where CockroachDB and Spanner Diverge.” *cockroachlabs.com*, January 2022. Archived at perma.cc/AWZ7-RXFT
- [72] Murat Demirbas. “Use of Time in Distributed Databases (Part 4): Synchronized Clocks in Production Databases.” *muratbuffalo.blogspot.com*, January 2025. Archived at perma.cc/9WNX-Q9U3

- [73] Cary G. Gray and David R. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” At *12th ACM Symposium on Operating Systems Principles (SOSP)*, December 1989. doi:10.1145/74850.74870
- [74] Daniel Sturman, Scott Delap, Max Ross, et al. “Roblox Return to Service.” *corp.roblox.com*, January 2022. Archived at perma.cc/8ALT-WAS4
- [75] Todd Lipcon. “Avoiding Full GCs with MemStore-Local Allocation Buffers.” *slideshare.net*, February 2011. Archived at perma.cc/CH62-2EWJ
- [76] Christopher Clark, Keir Fraser, Steven Hand, Jacob Gorm Hansen, Eric Jul, Christian Limpach, Ian Pratt, and Andrew Warfield. “Live Migration of Virtual Machines.” At *2nd USENIX Symposium on Symposium on Networked Systems Design & Implementation (NSDI)*, May 2005.
- [77] Mike Shaver. “fsyncers and Curveballs.” *shaver.off.net*, May 2008. Archived at archive.org
- [78] Zhenyun Zhuang and Cuong Tran. “Eliminating Large JVM GC Pauses Caused by Background IO Traffic.” *engineering.linkedin.com*, February 2016. Archived at perma.cc/ML2M-X9XT
- [79] Martin Thompson. “Java Garbage Collection Distilled.” *mechanical-sympathy.blogspot.co.uk*, July 2013. Archived at perma.cc/DJT3-NQLQ
- [80] David Terei and Amit Levy. “Blade: A Data Center Garbage Collector.” *arXiv:1504.02578*, April 2015.
- [81] Martin Maas, Tim Harris, Krste Asanović, and John Kubiataowicz. “Trash Day: Coordinating Garbage Collection in Distributed Systems.” At *15th USENIX Workshop on Hot Topics in Operating Systems (HotOS)*, May 2015.
- [82] Martin Fowler. “The LMAX Architecture.” *martinfowler.com*, July 2011. Archived at perma.cc/5AV4-N6RJ
- [83] Joseph Y. Halpern and Yoram Moses. “Knowledge and Common Knowledge in a Distributed Environment.” *Journal of the ACM (JACM)*, volume 37, issue 3, pages 549–587, July 1990. doi:10.1145/79147.79161
- [84] Chuzhe Tang, Zhaoguo Wang, Xiaodong Zhang, Qianmian Yu, Binyu Zang, Haibing Guan, and Haibo Chen. “Ad Hoc Transactions in Web Applications: The Good, the Bad, and the Ugly.” At *ACM International Conference on Management of Data (SIGMOD)*, June 2022. doi:10.1145/3514221.3526120
- [85] Flavio P. Junqueira and Benjamin Reed. *ZooKeeper: Distributed Process Coordination*. O’Reilly Media, 2013. ISBN: 9781449361303
- [86] Enis Söztutar. “HBase and HDFS: Understanding Filesystem Usage in HBase.” At *HBaseCon*, June 2013. Archived at perma.cc/4DXR-9P88

- [87] SUSE LLC. “SUSE Linux Enterprise High Availability 15 SP6 Administration Guide, Section 12: Fencing and STONITH.” *documentation.suse.com*, March 2025. Archived at perma.cc/8LAR-EL9D
- [88] Mike Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” At *7th USENIX Symposium on Operating System Design and Implementation* (OSDI), November 2006.
- [89] Kyle Kingsbury. “etcd 3.4.3.” *jepsen.io*, January 2020. Archived at perma.cc/2P3Y-MPWU
- [90] Ensar Basri Kahveci. “Distributed Locks Are Dead; Long Live Distributed Locks!” *hazelcast.com*, April 2019. Archived at perma.cc/7FS5-LDXE
- [91] Martin Kleppmann. “How to Do Distributed Locking.” *martin.kleppmann.com*, February 2016. Archived at perma.cc/Y24W-YQ5L
- [92] Salvatore Sanfilippo. “Is Redlock Safe?” *antirez.com*, February 2016. Archived at perma.cc/B6GA-9Q6A
- [93] Gunnar Morling. “Leader Election with S3 Conditional Writes.” *morling.dev*, August 2024. Archived at perma.cc/7V2N-J78Y
- [94] Leslie Lamport, Robert Shostak, and Marshall Pease. “The Byzantine Generals Problem.” *ACM Transactions on Programming Languages and Systems* (TOPLAS), volume 4, issue 3, pages 382–401, July 1982. [doi:10.1145/357172.357176](https://doi.org/10.1145/357172.357176)
- [95] Jim N. Gray. “Notes on Data Base Operating Systems.” In *Operating Systems: An Advanced Course*, Lecture Notes in Computer Science, volume 60, edited by R. Bayer, R. M. Graham, and G. Seegmüller, pages 393–481, Springer-Verlag, 1978. ISBN: 9783540087557. Archived at perma.cc/7S9M-2LZU
- [96] Brian Palmer. “How Complicated Was the Byzantine Empire?” *slate.com*, October 2011. Archived at perma.cc/AN7X-FL3N
- [97] Leslie Lamport. “My Writings.” *lamport.azurewebsites.net*, December 2014. Archived at perma.cc/5NNM-SQGR
- [98] John Rushby. “Bus Architectures for Safety-Critical Embedded Systems.” At *1st International Workshop on Embedded Software* (EMSOFT), October 2001. [doi:10.1007/3-540-45449-7_22](https://doi.org/10.1007/3-540-45449-7_22)
- [99] Jake Edge. “ELC: SpaceX Lessons Learned.” *lwn.net*, March 2013. Archived at perma.cc/AYX8-QP5X
- [100] Shehar Bano, Alberto Sonnino, Mustafa Al-Bassam, Sarah Azouvi, Patrick McCorry, Sarah Meiklejohn, and George Danezis. “SoK: Consensus in the Age of Blockchains.” At *1st ACM Conference on Advances in Financial Technologies* (AFT), October 2019. [doi:10.1145/3318041.3355458](https://doi.org/10.1145/3318041.3355458)

- [101] Ezra Feilden, Adi Oltean, and Philip Johnston. “Why We Should Train AI in Space.” White Paper, *starcloud.com*, September 2024. Archived at perma.cc/7Y3S-8UB6
- [102] James Mickens. “The Saddest Moment.” *USENIX ;login*, May 2013. Archived at perma.cc/T7BZ-XCFR
- [103] Martin Kleppmann and Heidi Howard. “Byzantine Eventual Consistency and the Fundamental Limits of Peer-to-Peer Databases.” *arXiv:2012.00472*, December 2020.
- [104] Martin Kleppmann. “Making CRDTs Byzantine Fault Tolerant.” At *9th Workshop on Principles and Practice of Consistency for Distributed Data* (PaPoC), April 2022. [doi:10.1145/3517209.3524042](https://doi.org/10.1145/3517209.3524042)
- [105] Evan Gilman. “The Discovery of Apache ZooKeeper’s Poison Packet.” *pagerduty.com*, May 2015. Archived at perma.cc/RV6L-Y5CQ
- [106] Jonathan Stone and Craig Partridge. “When the CRC and TCP Checksum Disagree.” At *ACM Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication* (SIGCOMM), August 2000. [doi:10.1145/347059.347561](https://doi.org/10.1145/347059.347561)
- [107] Evan Jones. “How Both TCP and Ethernet Checksums Fail.” *evanjones.ca*, October 2015. Archived at perma.cc/9T5V-B8X5
- [108] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM*, volume 35, issue 2, pages 288–323, April 1988. [doi:10.1145/42282.42283](https://doi.org/10.1145/42282.42283)
- [109] Richard D. Schlichting and Fred B. Schneider. “Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems.” *ACM Transactions on Computer Systems* (TOCS), volume 1, issue 3, pages 222–238, August 1983. [doi:10.1145/357369.357371](https://doi.org/10.1145/357369.357371)
- [110] Thanh Do, Mingzhe Hao, Tanakorn Leesatapornwongsa, Tiratat Patana-anake, and Haryadi S. Gunawi. “Limplock: Understanding the Impact of Limpware on Scale-out Cloud Systems.” At *4th ACM Symposium on Cloud Computing* (SoCC), October 2013. [doi:10.1145/2523616.2523627](https://doi.org/10.1145/2523616.2523627)
- [111] Josh Snyder and Joseph Lynch. “Garbage Collecting Unhealthy JVMs, a Proactive Approach.” *netflixtechblog.medium.com*, November 2019. Archived at perma.cc/8BTA-N3YB
- [112] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, Gary Grider, Parks M. Fields, Kevin Harms, Robert B. Ross, Andree Jacobson, Robert Ricci, Kirk Webb, Peter Alvaro, H. Biralı Runesha, Mingzhe Hao, and Huaicheng Li. “Fail-Slow at Scale: Evidence of Hardware Performance Faults in

Large Production Systems.” At *16th USENIX Conference on File and Storage Technologies*, February 2018.

[113] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. “**Gray Failure: The Achilles’ Heel of Cloud-Scale Systems.**” At *16th Workshop on Hot Topics in Operating Systems (HotOS)*, May 2017. [doi:10.1145/3102980.3103005](https://doi.org/10.1145/3102980.3103005)

[114] Chang Lou, Peng Huang, and Scott Smith. “**Understanding, Detecting and Localizing Partial Failures in Large System Software.**” At *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, February 2020.

[115] Peter Bailis and Ali Ghodsi. “**Eventual Consistency Today: Limitations, Extensions, and Beyond.**” *ACM Queue*, volume 11, issue 3, pages 55–63, March 2013. [doi:10.1145/2460276.2462076](https://doi.org/10.1145/2460276.2462076)

[116] Bowen Alpern and Fred B. Schneider. “**Defining Liveness.**” *Information Processing Letters*, volume 21, issue 4, pages 181–185, October 1985. [doi:10.1016/0020-0190\(85\)90056-0](https://doi.org/10.1016/0020-0190(85)90056-0)

[117] Flavio P. Junqueira. “**Dude, Where’s My Metadata?**” *fpj.me*, May 2015. Archived at perma.cc/D2EU-Y9S5

[118] Scott Sanders. “**January 28th Incident Report.**” *github.com*, February 2016. Archived at perma.cc/5GZR-88TV

[119] Jay Kreps. “**A Few Notes on Kafka and Jepsen.**” *blog.empathybox.com*, September 2013. Archived at perma.cc/XJ5C-F583

[120] Marc Brooker and Ankush Desai. “**Systems Correctness Practices at AWS.**” *ACM Queue*, volume 22, issue 6, pages 79–96, November/December 2024. [doi:10.1145/3712057](https://doi.org/10.1145/3712057)

[121] Andrey Satarin. “**Testing Distributed Systems: Curated list of Resources on Testing Distributed Systems.**” *asatarin.github.io*. Archived at perma.cc/U5V8-XP24

[122] Phil Eaton and Joran Dirk Greef. “**We Put a Distributed Database in the Browser—And Made a Game of It!**” *tigerbeetle.com*, June 2023. Archived at perma.cc/L7M7-X4HD

[123] Apple, Inc. and the FoundationDB project authors. “**FoundationDB—Simulation and Testing.**” *apple.github.io*. Archived at perma.cc/4C4L-AUH3

[124] Jack Vanlightly. “**Verifying Kafka Transactions—Diary Entry 2—Writing an Initial TLA+ Spec.**” *jack-vanlightly.com*, December 2024. Archived at perma.cc/NSQ8-MQ5N

- [125] Siddon Tang. “From Chaos to Order—Tools and Techniques for Testing TiDB, A Distributed NewSQL Database.” *pingcap.com*, April 2018. Archived at perma.cc/5EJB-R29F
- [126] Nathan VanBenschoten. “Parallel Commits: An Atomic Commit Protocol for Globally Distributed Transactions.” *cockroachlabs.com*, November 2019. Archived at perma.cc/5FZ7-QK6J
- [127] Jack Vanlightly. “Paper: VR Revisited—State Transfer (Part 3).” *jack-vanlightly.com*, December 2022. Archived at perma.cc/KNK3-K6WS
- [128] Hillel Wayne. “What If the Spec Doesn’t Match the Code?” *buttondown.com*, March 2024. Archived at perma.cc/8HEZ-KHER
- [129] Lingzhi Ouyang, Xudong Sun, Ruize Tang, Yu Huang, Madhav Jivrajani, Xiaoxing Ma, Tianyin Xu. “Multi-Grained Specifications for Distributed System Model Checking and Verification.” At *20th European Conference on Computer Systems (EuroSys)*, March 2025. [doi:10.1145/3689031.3696069](https://doi.org/10.1145/3689031.3696069)
- [130] Yury Izrailevsky and Ariel Tseitlin. “The Netflix Simian Army.” *netflixtechblog.com*, July, 2011. Archived at perma.cc/M3NY-FJW6
- [131] Kyle Kingsbury. “Jepsen: On the Perils of Network Partitions.” *aphyr.com*, May, 2013. Archived at perma.cc/W98G-6HQP
- [132] Kyle Kingsbury. *Analyses*. jepsen.io, 2024. Archived at perma.cc/8LDN-D2T8
- [133] Rupak Majumdar and Filip Niksic. “Why Is Random Testing Effective for Partition Tolerance Bugs?” *Proceedings of the ACM on Programming Languages (PACMPL)*, volume 2, issue POPL, article no. 46, December 2017. [doi:10.1145/3158134](https://doi.org/10.1145/3158134)
- [134] FoundationDB project authors. “Simulation and Testing.” apple.github.io. Archived at perma.cc/NQ3L-PM4C
- [135] Alex Kladov. “Simulation Testing for Liveness.” *tigerbeetle.com*, July 2023. Archived at perma.cc/RKD4-HGCR
- [136] Alfonso Subiotto Marqués. “(Mostly) Deterministic Simulation Testing in Go.” *polarsignals.com*, May 2024. Archived at perma.cc/ULD6-TSA4

About the Authors

Martin Kleppmann is an associate professor at the University of Cambridge, UK, where he lectures on distributed systems and cryptographic protocols. The first edition of *Designing Data-Intensive Applications* in 2017 established him as an authority on data systems, and through his research on distributed systems he helped start the local-first software movement. Previously he was a software engineer and entrepreneur at internet companies including LinkedIn and Rapportive, where he worked on large-scale data infrastructure.

Chris Riccomini is a software engineer, startup investor, and author with 15+ years of experience at PayPal, LinkedIn, and WePay. He runs Materialized View Capital, where he invests in infrastructure startups. He is also the cocreator of Apache Samza and SlateDB and coauthor of *The Missing README: A Guide for the New Software Engineer*.

Colophon

The animal on the cover of *Designing Data-Intensive Applications* is an Indian wild boar (*Sus scrofa cristatus*), a subspecies from India, Myanmar, Nepal, Sri Lanka, and Thailand that has higher back bristles, no woolly undercoat, and a larger, straighter skull than European boars. The Indian wild boar has gray or black hair with stiff bristles along the spine. Males have protruding canine teeth called *tushes* and are larger than females. The species averages 33–35 inches tall at the shoulder and weighs 200–300 pounds.

Predators of Indian wild boars include bears, tigers, and various big cats. Boars are nocturnal and omnivorous; their varied diet includes roots, insects, carrion, nuts, berries, and small animals. They root through garbage and crop fields, to the dismay of farmers, and need to eat 4,000–4,500 calories a day. Their well-developed sense of smell helps them forage underground, but their eyesight is poor.

In Hindu lore, the boar is an avatar of the god Vishnu. In ancient Greek funerary monuments, it is a symbol of a gallant loser. The aggressive animal decorates armor and weaponry of Scandinavian, Germanic, and Anglo-Saxon warriors. In the Chinese zodiac, it symbolizes determination and impetuosity.

Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Monica Kamsvaag, based on an antique line engraving from Shaw's *Zoology*. The series design is by Edie Freedman, Ellie Volckhausen, and Karen Montgomery. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.